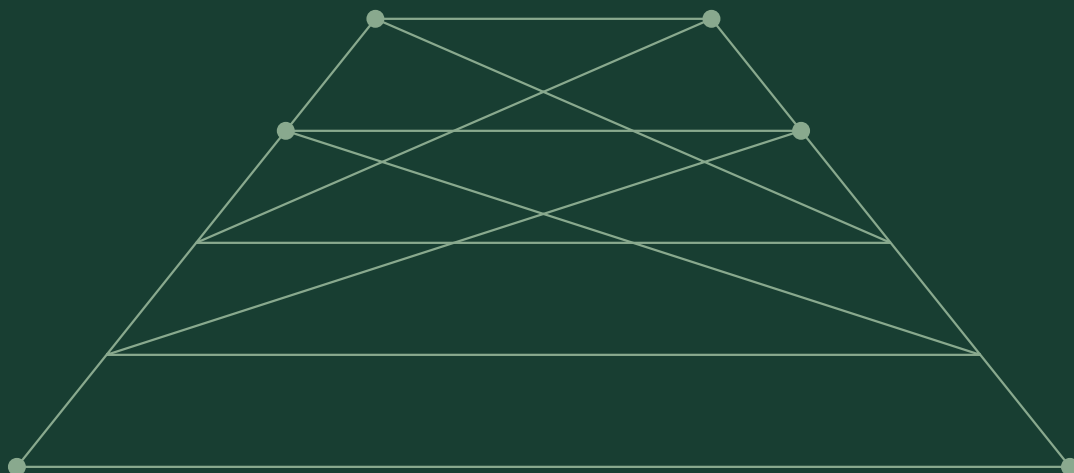


HARNESS ENGINEERING / FORMAL FOUNDATIONS

Harness 工程的 形式化基础

模型论 · 系统论双框架

锵与D老师 著



三篇 · 八章

原稿：约 2026 年 5 月

最后排版：2026 年 9 月 10 日



ABOUT THIS EDITION

阅读与版本说明

模型论提供形式化的分析语言，系统论提供宏观的架构原则。本书将二者结合，用统一的微观分析接口，理解编码智能体 Harness 的组织与运行。

全书分三篇：第一篇建立形式语言；第二篇依整体性、层次性、开放性与目的性展开架构分析；第三篇进行比较综合与展望。

版本与编排

本书由八章正文与原始导读编排而成，采用原稿章节顺序。正文中的论证、程序代码与数值沿用原稿，跨章引用已按本书结构整理；公式已统一为带定界符的 TeX，并按语义规范上下标分组。公式型代码块已改为独立公式或正文示意。封面、目录、分篇页和本说明为新增编辑内容。本版属于出版式排版交付，不代表已完成学术审定。

引用与版本整理

已对照早期十一章稿核查第七章的旧引用。原“11.1 节”的统一定义实际位于旧稿 11.1.7 节，本版将其概述补入第七章；原“11.4.2 节”的负反馈原则改为直接陈述，并指向本书第六章的反馈机制说明。题记与逐子系统比较的旧章号也已整理。

源码版本基线

omp: v16 系列。版本归档说明明确将本稿标为 v16 冻结；本稿所用的精确小版本及提交号未记录。DS-TUI: 原始分析采用的版本历史截至 v0.8.37（2026 年 5 月 14 日）；这一日期是版本历史的截止点，不能据此确认全部源码均锁定于该版本，精确提交号未记录。

上述信息依据原始分析的版本历史与文档归档记录。本书中的实现细节应结合这一历史范围阅读；“最后排版”日期仅表示文档排版更新时间，不表示源码已按该日期重新核验。

后续内容审订

正式出版前，建议核对经典模型论术语与工程类比的边界、定理表述及适用条件，并为具体系统版本、性能数值与实现细节补充可追溯来源。本文中的统计性判断与形式化类比沿用原稿，不作为本次排版新增的核验结论。

作者：镞与D老师。原稿时间据最早入库记录（2026 年 5 月 27 日）估计为 2026 年 5 月。原始导读收入书末。最后排版：2026 年 9 月 10 日。

CONTENTS

目录

第一篇 形式语言

第一章 模型论基础：语言、理论与模型	5
--------------------	---

第二章 Harness 即模型论：Agent 系统的公理化设计	18
---------------------------------	----

第二篇 系统论架构

第三章 整体性：理论的相容性与合并	34
-------------------	----

第四章 层次性：理论塔	41
-------------	----

第五章 开放性：语言扩展	61
--------------	----

第六章 目的性：目标函数驱动理论选择	74
--------------------	----

第三篇 综合

第七章 比较分析与统一定义	93
---------------	----

第八章 展望：模型论视角的 Harness 工程未来	105
----------------------------	-----

附录：原始导读	118
---------	-----

I

第一篇

形式语言

建立语言、理论与模型之间的关系。

CHAPTER 01

第一章 模型论基础：语言、理论与模型

模型论是数理逻辑的一个分支，研究形式语言与其解释之间的关系。在编码智能体 harness 工程中，模型论提供了一个精确的分析框架：系统提示词是一个一阶理论 T ，LLM 的每一次会话行为是一个结构 M ，满足关系 $M \models T$ 是衡量行为是否"正确"的数学基准。本章建立这套分析语言的全部核心概念，并揭示它在 LLM 系统中的三个根本性偏离。

模型论的基本概念

一阶语言 L : 符号的宇宙

数理逻辑中的一阶语言（first-order language） L 由两类符号构成：**逻辑符号**（逻辑连接词、量词、等词、变元）和**非逻辑符号**（常元符号、函数符号、关系符号）。逻辑符号对所有一阶语言是通用的；非逻辑符号决定了具体语言的"词汇量"——Signature $\Sigma = (C, F, R)$ ，其中 C 是常元符号集、 F 是函数符号集（每个附有元数 n ）、 R 是关系符号集（每个附有元数 n ）。

语言 L 的语法是递归定义的：从项（terms）开始——常元符号和变元是最简项，用函数符号作用在其他项上生成复合项；项上施加关系符号得到原子公式；逻辑连接词（ $\neg$ 、 $\wedge$ 、 $\vee$ 、 $\rightarrow$ 、 $\leftrightarrow$ ）和量词（ $\forall$ 、 $\exists$ ）组合原子公式得到公式。这一套递归定义保证了语言的良基性（well-foundedness）：每个表达式有唯一的构造树。

在模型论语境中，**语言 L 本身不含意义**。符号是在解释中获得意义的。

L -结构 M : 符号的执行

给定一阶语言 L ，一个 L -结构（L-structure） M 提供以下要素：

- **论域（universe）** $|M|$ ：一个非空集合，结构中的一切都在其中取值。
- **常元解释**：对每个常元符号 $c \in C$ ， M 赋予一个具体元素 $c^M \in |M|$ 。
- **函数解释**：对每个 n 元函数符号 $f \in F$ ， M 赋予一个映射 $f^M : |M|^n \rightarrow |M|$ 。
- **关系解释**：对每个 n 元关系符号 $R \in R$ ， M 赋予一个子集 $R^M \subseteq |M|^n$ 。

L -结构是"意义"的载体。语言 L 中的符号在这里获得具体指称：常元成为数据、函数成为操作、关系成为约束条件。

在 harness 工程中， L -结构的映射既直接又富有启发性：

逻辑概念	Harness 中的对应
语言 L 的常元符号	硬编码配置值：模型名称、上下文窗口容量、工作空间根路径
语言 L 的函数符号	工具（tools）：每个工具是一个 n 元函数，输入参数 $\rightarrow$ 输出结果
语言 L 的关系符号	约束条件：MUST/NEVER 规则、许可策略、契约块中的不变量
项（terms）	工具调用的参数构造： <code>read("src/main.rs:50-100")</code>
原子公式	单个工具调用的完成： <code>read</code> 返回文件内容
复合公式	工具调用序列：先 <code>search</code> 后 <code>read</code> 再 <code>edit</code>

论域 $|M|$ 则是会话期间 LLM 可触及的所有实体：消息文本、文件内容、命令输出、工具返回值、LLM 的思维链（reasoning content）、用户消息中的自由变量。这是一个动态的、异质的、不断增长的对象集合。

满足关系 $M \models T$

当且仅当结构 M 中的解释使理论 T 的所有句子（sentence——无自由变量的公式）为真时，称 M 是 T 的一个模型，记为 $M \models T$ 。满足关系（satisfaction relation）是用递归方式定义的：从原子公式的赋值开始，通过逻辑连接词和量词的语义规则上升到复合公式。这个递归定义保证了满足关系的良基性——每个公式的真值最终可以归结到原子层次的可判定检查。

满足关系的核心性质是：

1. **完全性**：对任何句子 φ ， $M \models \varphi$ 或 $M \models \neg\varphi$ 有且仅有一个成立。
2. **紧致性**：如果 T 的每个有限子集都有模型，则 T 有模型。
3. **洛文海姆-斯科伦定理**：如果 T 有无限模型，则 T 在任意大于语言基数的基数上都有模型。

这些性质为编码智能体 harness 设计提供了判断标准。系统提示词是一个理论 T ——它声明了"你"的身份、可用工具集、行为约束和验证规则。LLM 的会话行为——其输出的每一个 token、发起的每一个工具调用、遵循的每一条约束——构成了一个结构 M 。当会话正确运行时， $M \models T$ 。

这条判断标准精确地界定了系统提示词设计的目标：用有限长度的自然语言文本，构造一个理论 T ，使得在 harness 约束下的 LLM 生成的每个结构 M 都满足 T 。这是一个**理论选择问题**：在浩瀚的可能行为空间中，挑选出"正确"行为的结构类 K ，并用可传递的句子集 T 来刻画 K 。

理论与模型之间的张力

一个理论 T 可能有多个不同的模型（非范畴性），也可能完全没有模型（不一致）。前者的典型是算术的一阶理论——有标准模型 N ，也有无数非标准模型（包含无穷大元素）。后者的典型是包含矛盾的理论——没有结构能同时满足 φ 和 $\neg\varphi$ 。

在 harness 工程中，理论 T 的不一致是一个操作性风险：如果系统提示词中包含矛盾指令（例如，一段命令 LLM "总是详细解释每一步" 而另一段命令 "保持简洁"），则 LLM 的行为结构 M 在某种意义上试图同时满足两条矛盾的约束——结果通常是**模糊遵循**（部分满足、部分违反），即软满足。这引出了下一节的核心主题。

LLM 系统的三个偏离

经典模型论的核心假设有三个：语言是形式化的、满足是二值的、公式是封闭的。LLM 系统在每一个假设上都发生了根本性偏离。这些偏离并非"错误"——它们是 LLM 系统特性（灵活性、适应性、创造力）的来源——但它们确实要求我们在应用模型论框架时做出调整。

偏离一：非形式化语言

经典一阶语言 L 的语法是严格递归定义的：符号 $\rightarrow$ 项 $\rightarrow$ 原子公式 $\rightarrow$ 公式 $\rightarrow$ 句子。自然语言没有这种结构——它是一团涌动的、依赖上下文消歧的、充满模糊边界的"软语法"。

系统提示词使用的是自然语言（通常是英语），其中 MUST、NEVER、SHOULD 等关键词是**被强行标注为形式符号的自然语言片段**。系统提示词的开篇：

```
You are THE staff engineer the team trusts with load-bearing changes:  
- debugging across unfamiliar code,  
- refactors that touch many callers,  
- API decisions that other code will depend on for years.
```

这段文本试图定义一个实体（"你"）的性质。在形式语言中，这对应于一串公理：身份公理、能力范围公理、决策准则公理。但在自然语言中，这些"公理"的语义边界是模糊的——"load-bearing changes" 究竟包含哪些场景？"debugging across unfamiliar code" 包不包括仅仅是阅读代码而不修改的情况？

更关键的是，自然语言句子的逻辑形式（logical form）是不确定的。同一个句子可以被解析为不同的逻辑结构：

原句: "You MUST optimize for correctness first"

1. $\rightarrow \forall \text{action} : \text{priority}(\text{correctness}, \text{action}) > \text{priority}(\text{other}, \text{action})$
2. $\rightarrow \forall \text{action} : \text{Do}(\text{action}) \rightarrow \text{Ensure}(\text{correctness}(\text{action}))$
3. $\rightarrow \forall \text{action} : \text{recommended}(\text{action}) \wedge \text{correctness}(\text{action}) > \neg \text{recommended}(\text{action})$

这三种解析产生不同的理论 T 版本，进而可能产生不同的模型类。经典模型论中的符号与非逻辑符号之间有清晰界分：逻辑符号的含义由演绎系统固定，非逻辑符号的含义由 L -结构的解释函数固定。但自然语言提示词中，这层界分被消解了——MUST 被标记为“必然模态算子”，但仍然是通过模型权重学习到的语义，而非通过演绎规则定义的语义。

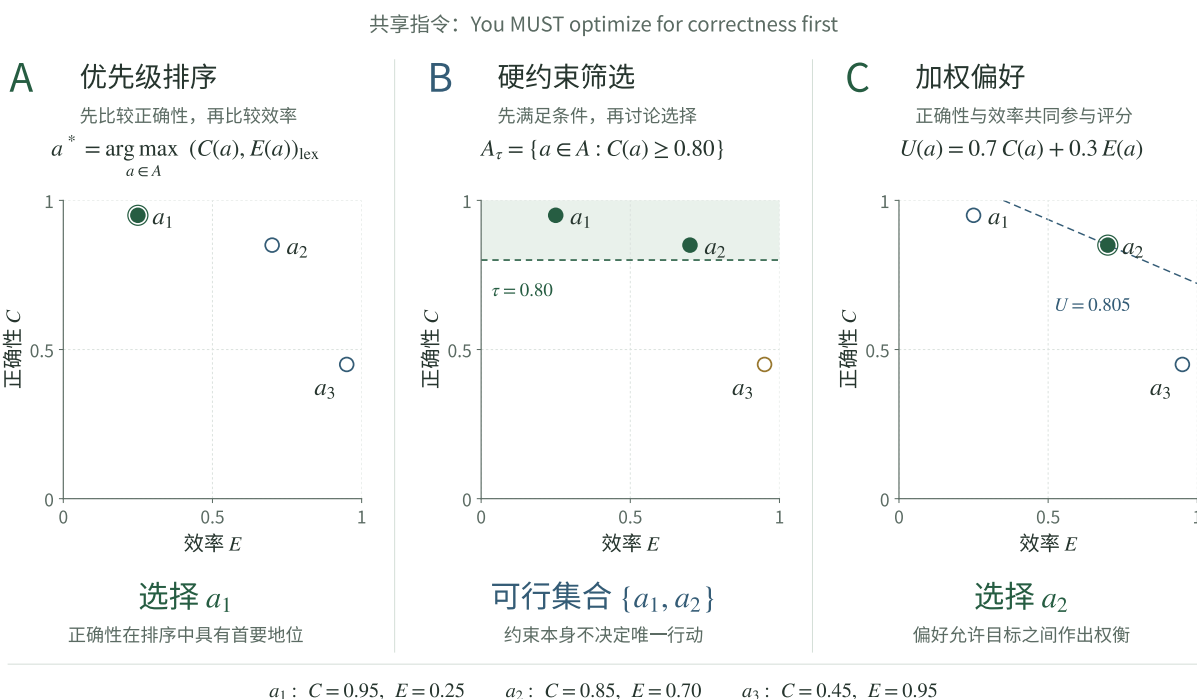


图 1.1 同一指令的三种决策模型（构造示例）

设候选集合 $A = \{a_1, a_2, a_3\}$, $C(a)$ 与 $E(a)$ 分别为正确性和效率的假设评分，取值范围为 $[0, 1]$ ，并非概率。分图 A 按 (C, E) 的字典序排序；分图 B 以 $C(a) \geq 0.80$ 筛选可行集合；分图 C 按 $0.7C(a) + 0.3E(a)$ 比较方案，三个得分依次为 0.740、0.805、0.600。

分数、门槛和权重仅用于示意；门槛是否代表充分正确性取决于具体任务。本图为补充构造，非原文三式的严格等价改写。原文第三式的运算语义尚未完整定义，分图 C 使用此处单独定义的加权模型。

omp 的 <system-conventions> 块试图在非形式系统中注入形式语义：

****RFC 2119 applies to MUST, REQUIRED, SHOULD, RECOMMENDED, MAY, OPTIONAL.**
`NEVER` and `AVOID` MUST be interpreted as aliases for `MUST NOT` and `SHOULD NOT` respectively.**

这条元语句试图用一个公理化定义将自然语言中的副词集映射到模态逻辑层次。这是一个"假装形式"的策略——它模拟了形式语言的语法约束，但底层仍然是统计语言模型。这个策略的工程有效性取决于语义与统计分布之间的一致性程度。

偏离二：软满足（连续而非二值的满足关系）

在经典模型论中，满足是二值的： $M \models \varphi$ 要么成立，要么不成立。没有"部分满足"或"近似满足"的中间状态。

LLM 对系统提示词的"遵守"是连续而非离散的。一个典型的场景：系统提示词明确要求使用 `read` 工具读取文件，但 LLM 在某些边缘情况下（如"只是查看文件是否存在"）使用了 `bash cat`。这种行为既不是完全遵守（它违反了工具选择公理），也不是完全违反（它实现了相同的功能）。在经典模型中，这是一个明确的 $M \not\models T$ 的情形。但在工程实践中，这种程度的偏离通常是可接受的——只要核心安全约束没有被破坏。

软满足的数学处理可以借助模糊模型论（fuzzy model theory）的概念。在模糊框架中，赋值函数 $v : \text{Formulae} \rightarrow [0,1]$ 给出公式的"真度"（degree of truth）。系统提示词中的约束可以按重要程度分层：

- **安全约束**（真度阈值 ~ 1.0 ）：不得执行破坏性操作、不得泄露凭证、不得违反用户明确的否定指令。这些约束几乎总是被 100% 满足——LLM 的安全训练（RLHF、DPO）使得违反这些约束的概率极低。
- **协议约束**（真度阈值 ~ 0.9 ）：工具选择指南、通信格式规则、分解模式。这些约束部分满足——LLM 可能在大多数情况下正确使用 `edit` 而非 `sed -i`，但在少数边缘情况下"滑坡"。
- **风格约束**（真度阈值 ~ 0.7 ）：语气、态度、习惯用语。这些约束最容易出现"滑动"——在长会话或高认知负载下，LLM 更可能恢复其默认的"通用助手"语调。

软满足的另一个维度是**时间演化**。在会话过程中， $M \models T$ 的程度随 token 计数增加而变化。早期（前 1000 tokens）满足度较高——LLM 更容易记住系统提示词中的约束。晚期（20K+ tokens）满足度降低——因为历史上下文稀释了系统提示词的信号，LLM 更依赖训练数据的默认行为。这对应于模型论中的**遗忘问题**：理论 T 的生效范围随结构 M 的增长而衰减。

偏离三：开公式参数（用户消息作为自由变量实例化）

在经典模型论中，一个理论 T 中的公式如果含有自由变量 x ，则称为开公式（open formula），记作 $\varphi(x)$ 。开公式本身不具有真值——只有在对自由变量进行赋值（assignment）后，它才获得真值。给定赋值 $s : \text{Var} \rightarrow |M|$ ， $M \models \varphi[s]$ 当且仅当在 M 中用 s 替换自由变量后 φ 为真。

编码智能体 harness 的核心机制就是**开公式实例化**：

1. **理论 T** = 系统提示词（全称量化约束 + 工具描述 + 通信规则）
2. **自由变量** = 用户消息的内容、会话历史的积累、环境状态的快照

3. 赋值 s = 用户在当前轮次输入的具体消息文本

在每一轮交互中，LLM 接收的并非封闭的理论 T ，而是 T 加上当前轮次的赋值 s ，形成一个被实例化的理论 $T[s]$ 。不同轮次的 s 不同，产生不同的理论实例和不同的模型：

(用户说"重构这个模块"→ LLM 生成重构方案的结构)

$$T[s_1] \rightarrow M_1$$

(用户说"系统响应太慢"→ LLM 生成性能分析的结构)

$$T[s_2] \rightarrow M_2$$

同一个 T 在不同实例化下可以产生结构上完全不同的模型。这正是适应性的来源——也是非确定性的根源。

更进一步：用户消息本身不是单一的自由变量，而是多个自由变量的复合赋值。一条消息可能包含问题（ $\varphi_{\text{question}}$ ）、约束（ $\varphi_{\text{constraint}}$ ）、上下文（ φ_{context} ）和示例（ φ_{example} ）。这些信息的混合赋值使得理论 T 的逻辑特征在每次实例化时都发生变化——有些约束被激活，有些被覆盖，有些被弱化。

从模型论看，这是一个**理论扩张**过程： $T[s] = T \cup \{\varphi_s\}$ ，其中 φ_s 是赋值 s 的公式编码。每次用户消息都是一次有限的理论扩张——它通过添加句子来改变 T 的模型类 $\text{Mod}(T)$ 。如果新添加的句子与 T 不一致（用户要求去做系统提示词禁止的事情），则 $\text{Mod}(T[s]) = \emptyset$ ——无模型状态。这是 LLM 通常表现为"抱歉，我无法执行这个操作"的情况。

解释函数 I : 从符号到执行

语法与语义之间的桥梁是解释函数 I 。在模型论中,解释函数是 L -结构的编译器：它将语言 L 中的每个符号映射到结构 M 中的一个具体实体。在 harness 工程中,解释函数 I 将工具名称（函数符号）映射到实际的系统操作（执行）。

工具调用的两层映射

工具调用涉及两层解释：

第一层：LLM 内部的语义解释。 LLM 将 token 序列 `read("src/main.rs")` 理解为一个函数符号 `read` 作用于参数 `"src/main.rs"`。这个语义解释发生在模型权重的编码空间中——它是隐式的、统计的、无法直接检查的。

第二层：运行时系统操作的解释。当 harness 接收到 LLM 发送的工具调用请求时，它将符号 `read` 映射到一个具体的运行时函数——读取文件系统、建立 I/O 通道、返回输出。这一层是显式的、类型化的、可以直接检查的。

解释函数 I 同时描述这两层——它是 LLM 的语义空间与操作系统的执行空间之间的同态映射：

$$I : \text{ToolName} \rightarrow (\text{InputSchema} \rightarrow \text{IOOperation})$$

在 DS-TUI 中，解释函数 I 通过 `ToolSpec` trait 实现。每个工具实现一个 `execute` 方法，其签名在类型层面保证了映射一致性：

```
// DS-TUI: ToolSpec trait 的 execute
trait ToolSpec {
  fn spec() -> ToolDefinition;          // 声明语言 L 中的签名
  async fn execute(args: Value) -> Result<Value>; // 结构 M 中的解释
}
```

`ToolDefinition` 是语言 L 中的声明——它定义了函数符号的名称、参数类型、返回类型和描述信息。`execute` 是结构 M 中的实现——它实际驱动了从输入到输出的转换。在 `omp` 中，等价结构是 `AgentTool` trait：

```
// omp: AgentTool 的 execute
interface AgentTool {
  readonly name: string;          // 语言 L 中的函数符号名
  readonly description: string;
  readonly schema: Schema;        // 参数类型的语法编码
  execute(input: Record<string, unknown>): Promise<string>; // 结构 M 中的实现
}
```

类型签名作为定义域约束

在两个系统中，解释函数 I 的核心约束来自**类型签名**。工具 `read` 声明的参数类型是 `{path: string}`——这是函数符号的定义域约束。在模型论中，每个函数符号 f 附带定义域约束 $\text{Dom}(f) \subseteq |M|^n$ 。 n 元函数的定义域是论域的 n 次笛卡尔积的一个子集。

JSON Schema（或 Rust 的类型系统）就是这些定义域约束的语法编码。当一个工具调用违反类型签名（传入 `null` 给一个需要 `string` 的参数）时，harness 应该拒绝调用——等价于模型 M 拒绝为该函数符号赋予一个落在定义域外的输入。DS-TUI 和 `omp` 都在工具调用处理前进行 JSON Schema 验证，正是为了强制遵守这一约束。

解释函数的多层次

解释函数 I 不是单层的——它在不同抽象层次上有不同的"含义"：

语法层 I_{syntax} ：符号名称 $\rightarrow$ AST 表示。LLM 将 `read` 理解为工具调用结构中一个特定的 AST 节点。

语义层 $I_{\text{semantics}}$ ：符号 $\rightarrow$ 功能描述。LLM 将 `read` 理解为"读取文件内容并返回"——这是从训练数据中学到的语义。这个解释与符号在系统提示词中的自然语言描述有关。

操作层 $I_{\text{operation}}$ ：符号 $\rightarrow$ IO 操作。运行时系统将 `read` 映射到文件系统的 `readFile` 系统调用。这个解释与符号的运行时实现有关。

结果层 I_{result} ：符号调用结果 $\rightarrow$ 新的论域实体。`read` 返回的文件内容成为后续推理的论域元素。这个解释改变了结构 M 的论域——它是一个**论域扩充**操作。

这三层解释的一致性程度决定了理论 T 的保真度。如果 LLM 将 `read` 理解为"读取并解析文件的内容并自动修复错误"（ $I_{\text{semantics}}$ 的偏离），但运行时系统只执行了文本读取（ $I_{\text{operation}}$ 的朴素解释），则 LLM 对后续推理的前提假设与实际情况之间产生了**解释鸿沟**。

解释与理论之间的关系

解释函数 I 不是独立于理论 T 存在的。系统提示词中的工具描述（每个工具的定义、使用指南、限制条件）本身就是理论 T 的一部分——它们为每个函数符号提供了公理化的语义描述。

DS-TUI 的工具选择指南相当于为每个函数符号添加了**使用条件公理**：

```
### `exec_shell`
Use `exec_shell` for shell-native diagnostics, pipelines, and bounded commands.
Use structured tools for structured operations when they map directly.
```

这是一条元公式： $\forall op \in \text{Operations}, (\text{isShellNative}(op) \vee \text{isPipeline}(op)) \rightarrow \text{use_exec_shell}(op)$ 。

它教导 LLM 在什么情境下应该调用这个函数符号——不仅是"工具可用"，而是"在给定条件下工具应该被使用"。

这种模式说明了一个深层结构：**解释函数 I 和理论 T 是相互定义的**。 T 中的公理约束了 I 应该将符号映射到哪些操作； I 的实现在结构 M 中创造了 T 中公式的真值条件。两者的交互构成了 harness 系统的逻辑动态。

Kategorizität 与确定性问题

Kategorizität（范畴性）是模型论中衡量理论确定性的核心概念。一个理论 T 称为 κ -范畴的（ κ -categorical），如果 T 的所有基数为 κ 的模型都是同构的（isomorphic）。直观地说：范畴理论"决定了"它的模型——在给定论域大小后，所有模型在结构上是不可区分的。

为什么 LLM 系统不是范畴的

给定固定的系统提示词 T 和固定的会话上下文 κ （包括用户消息和会话历史）， T 是否决定了唯一的模型（即唯一的 LLM 输出序列）？答案是否定的——原因有三个层次。

语言歧义是最根本的层次。 系统提示词使用自然语言，而自然语言的语义是多值的。omp 中有这样一条公理：

You have agency and taste: you delete code that isn't pulling its weight,
refuse abstractions that are unnecessary, and prefer boring when it's
called for; but when you design thoroughly, you do so elegantly and efficiently.

其中 "pulling its weight"、"boring"、"elegantly" 没有形式定义。不同 LLM 调用中，这些术语可以被"解释"为不同的具体行为集合。这相当于理论 T 中的多个谓词符号缺乏唯一解释——它们的含义被"悬置"了，直到在特定推理路径中被实例化。在模型论语境中，同一理论 T 因语义解释的不确定性，可以产生一组结构上不等价的模型 $\{M_1, M_2, \dots\}$ —— T 不是范畴的。

随机采样是操作层面的原因。 LLM 生成是概率性的。给定相同的 prompt，模型从分布 $P(\text{token}|\text{context})$ 中采样。即使所有系统提示词工具的使用完全确定了行为的"结构骨架"，采样过程中的少量随机性（temperature > 0 时的随机选择，top-k 采样中的排序波动）也可以导致不同的输出序列。不同的输出序列意味着不同的工具调用——进而导致不同的会话历史结构。这是最直接的多模型来源。

环境非确定性是第三个层次。 工具调用的结果不是确定性的。读取同一个文件可能产生不同内容（如果文件在执行中被外部修改），命令执行可能因网络延迟、CPU 调度、磁盘 I/O 而成功或失败。结构 M 中的函数解释 f^M 在两次运行中不同——这不是理论 T 的不确定性，而是结构 M 的动态性。在经典模型论中，结构 M 的论域和解释是固定的——它们不会在模型的使用过程中发生变化。但在 LLM 系统中，结构 M 的每个函数解释都是有副作用的：read 不仅返回内容，还可能改变文件系统的时间戳；bash 的执行可以改变系统状态。这种副作用本质使得 M 在每次函数调用后都不再是原来的结构——这是一种"可变结构"的模型论。

确定性困境与工程回应

非范畴性带来一个被低估的工程困境：**如何调试一个无法复现的行为？** 如果同一理论 T 在不同运行中产生不同模型，而某个模型违反了安全约束（例如，LLM 在某次运行中执行了破坏性命令），开发者是否可以在 T 中添加公理来防止这类行为？

理论上可以——添加一条禁止公理 $\neg\psi$ 可以排除包含该行为的模型。但由于三个偏离（语言歧义、概率采样、环境非确定性），添加公理后的理论 T' 仍然不是范畴的——它只是排除了一个模型，但没有决定随后的所有模型。这是一种**有限排除策略**：每次一个不良模型被观察到，就在 T 中添加一条相应的否定公理。随着公理数量的增加，模型类 $\text{Mod}(T)$ 逐步缩小，但永远无法缩小到单点——因为 T 的语言不是形式化的（偏离一），满足不是二值的（偏离二），而环境也不是确定性的（偏离三）。

DS-TUI 对非范畴性的工程回应是通过**行为分类**来减少不确定性。其 *Decomposition Philosophy* 提供了三个行为模式锚点（PREVIEW、CHUNK + map-reduce、RECURSIVE），将无限的行为可能性空间划分到少数几个"范型"中。这不是在概念层面定义这些范型，而是在**操作层面**将它们与具体的工具和流程绑定——每个范型对应一个具象的工具调用序列模板。这使得不同运行中的行为虽然不同，但属于相同的同构类——它们在模型论意义下是**结构相似的**（elementarily equivalent，但在 LLM 的"概率等价"意义上而非严格模型论的初等等价意义上）。

omp 的应对策略更侧重于**否定公理系统**。其系统提示词中包含约 15 条显式禁止公理，构成了一个排斥性约束网络。这些禁止公理不从正面定义良构行为的完整空间（那需要无限多的正例），而是从反面划定不良行为的边界——通过逐步削除搜索空间来逼近范畴性。这是一个**逐步排除法**，是经典模型论中"理论的否定理"的工程应用：少量精确的否定命题可以有效削减行为空间。

同构的退出：从模型论到概率

当非范畴性成为不可避免的前提，我们需要调整经典模型论的分析工具。在 LLM 系统中，"模型"不应当被理解为一个确定的 L -结构 M ，而应当被理解为一个**概率空间** (Ω, P) 上的随机结构 M_ω ——其中 $\omega \in \Omega$ 是随机种子（采样路径）， P 是 token 采样的概率分布。

模型的"满足"不再是二值的 $M \models T$ ，而是概率满足 $P(M \models T) \geq 1 - \varepsilon$ ——其中 ε 是容忍度。理论 T 的"确定性程度"可以度量为其模型类 $\text{Mod}(T)$ 的多样性：

$$\text{Det}(T) = 1 - \frac{|\text{Mod}(T)|_{\text{sim}}}{|\text{Mod}_{\text{total}}|}$$

其中 $|\text{Mod}(T)|_{\text{sim}}$ 可以用模型之间的"结构距离"来度量——例如编辑距离（消息序列的相似度）、语义距离（嵌入空间的相似度）或行为距离（工具调用序列的相似度）。这个度量框架将 *Kategorizität* 从离散概念（同构 vs 非同构）转化为连续概念（确定性程度），与 LLM 系统的概率本质自然对齐。

理论的一致性维护

如果一个理论 T 是**一致的**（consistent），存在至少一个结构 M 使得 $M \models T$ 。否则 T 是**不一致的**——它没有模型，是空的。

在编码智能体 harness 中，理论 T 的一致性不仅是逻辑要求，更是工程要求。不一致的约束集将导致 LLM 行为的不稳定——它会在矛盾的指令间摇摆，选择性地忽略部分约束，或产生不可预测的"解释"来调和不可调和的指令。

两种公理化策略

DS-TUI 和 omp 采用不同的策略来维护理论 T 的一致性。

DS-TUI：分层公理化。 其核心思想是将理论 T 分解为按优先级排序的子理论层：

$$T = T_{\text{base}} \cup T_{\text{personality}} \cup T_{\text{mode}} \cup T_{\text{approval}}$$

每层在组合时具有**优先级覆盖**语义：下层（较高优先级）可以限制上层（基础层）中的公理，但不可否定它们。 T_{mode} 中的"plan 模式限制工具执行权限"是对 T_{base} 中"所有工具可用"公理的限制（restriction），而非否定（negation）。这意味着 T 的推理闭包 $\text{Cn}(T)$ 始终是 T_{base} 的推理闭包的子集在受限条件下的投影——这是一种**保守限制**。

DS-TUI 的分层顺序在 `prompts.rs` 中被硬编码为确定性顺序——它通过编译期固定层间关系来避免运行时的冲突发现。这种策略的理论代价是**灵活性缺失**：新公理必须分配到已有层次中，无法动态创建新层。

omp：模块化公理化。 omp 的理论 T 通过正交维度组装：

$$T_{\text{omp}} = T_{\text{core}} \cup T_{\text{tools}} \cup T_{\text{skills}} \cup T_{\text{rules}} \cup T_{\text{memories}}$$

每个维度是一个独立的公理来源，通过技能加载器 `loadSkills()` 在会话启动时从文件系统扫描。正交维度意味着理论 T 的各部分之间没有预设的优先级关系——它们的交互通过**命名空间隔离**来避免冲突：不同技能、不同工具各自管理自己的公理，互不干扰。

当冲突确实发生时（同名工具、重叠的约束条件），omp 通过去重策略（`realPathSet`）处理——冲突技能产生警告而非错误。这是一种**宽容的一致性策略**：容忍局部不一致，只要系统整体仍能运作。

保守扩充与非保守扩充

在模型论中，理论扩充分为两类：

保守扩充 (conservative extension)：给定理论 T 和其语言 L ，在扩展语言 $L' \supseteq L$ 上添加新公理集 Δ ，如果对于原语言 L 中的每个句子 φ ， $T' \models \varphi$ 当且仅当 $T \models \varphi$ ，则称 T' 是 T 的保守扩充。

非保守扩充 (non-conservative extension)：添加新公理后，在原语言 L 中可能推导出原本不可推导的句子。

在 harness 工程中，保守扩充的分类有直接的工程意义：

1. **工具添加是保守扩充的吗？** 当 omp 在运行时动态添加一个新工具（一个新函数符号），它在语言 L 中添加了一个新符号及其使用公理。如果新工具的公理不与其他工具发生交互（即不修改已有工具的定义域或值域），则这是一个保守扩充——原有行为的推理不受影响。但实践中，新工具很可能改变 LLM 的工具选择行为（“现在有了新工具，我是否应该用新工具替代旧工具？”）——这就是非保守效应。
2. **技能添加可能是非保守的。** 当一个技能（如 accessibility 或 branding-identity）被注入系统提示词，它引入了新的常量符号（如 WCAG 准则中的 contrast_ratio）和新的义务公理（如“所有 UI 必须符合 WCAG 2.1 AA”）。这些新公理可能与 system-prompt.md 中的某些约束重叠甚至冲突——例如系统提示词要求“优先使用现成的 UI 库”而技能要求“使用原生组件以实现无障碍控制”。这构成非保守扩充。

DS-TUI 的编译时常量策略本质上要求所有扩充是**预先声明**的——每个插件的公理在编译时就被纳入 T 。这确保了 T 在编译后的所有运行中是固定的，从而容易检验一致性：如果编译时 T 是一致的，则运行时 T 是一致的（因为 T 不变）。代价是新插件需要重新编译。

omp 的运行时组装策略允许在运行时进行理论的保守扩充——但一致性验证不能在编译时完成。每一个新的技能、规则或记忆都可能引入非保守效应。omp 应对这一问题的工程策略是**增量验证**：每个技能被设计为自包含的、对其他公理依赖最小化的模块，通过命名空间隔离来避免跨模块交互。这是一种**设计时一致性**策略——从源头上减少非保守效应的可能。

理论的内模一致性

除了扩充时的一致性外，理论还存在**内模一致性**问题：系统提示词内部的公理之间是否矛盾？

一个高风险的矛盾模式是“全称约束” vs “模糊例外”。例如：

- 全称约束：You NEVER yield incomplete work.
- 模糊例外：When the user's intent is clear, you SHOULD proceed without asking.

在某种场景下，“用户的意图已经明确，但任务无法在本次调用中完整”——LLM 同时面对“必须完成”和“直接执行”的约束，产生了**操作不确定性**。这不是逻辑矛盾（两条规则可以同时为真），但在实际行为空间中产生了一个 LLM 难以导航的节点。

另一个矛盾模式是"刚性约束" vs "自反性约束"。例如：

- 刚性约束：You NEVER suppress tests to make code pass.
- 自反性约束：You have agency and taste: you delete code that isn't pulling its weight.

"Agentic"的 LLM 可能"有品位地"决定一个测试不重要而删除它——这是对前面的"永不压制测试"的潜在违反。这种矛盾的根源在于：**规则的严格程度与 LLM 的自主判断能力之间的张力**。给 LLM 越多"品味"和"判断力"， T 中刚性约束被覆盖的概率就越高。

一致性验证的边界

在经典模型论中，理论 T 的一致性是一个**不可判定问题**——对于足够丰富的语言，没有算法可以在有限时间内判断 T 是否一致。Harness 工程实践中的"一致性"只能是一个**可验证的近似**。

这种近似通过以下工程手段实现：

- **形式化边界**：在自然语言的边缘引入形式化元素——JSON Schema、类型签名、合约检查——来确保函数符号的定义域和值域的合法性。这些形式化元素构成了理论 T 的一个**可判定子集** $T_{\text{formal}} \subseteq T$ ，其一致性是可机械验证的。
- **运行时检查点**：在关键决策点（工具调用、操作审批）进行运行时验证。这些检查点相当于模型 M 在构建过程中被实时检验——如果 M 在某个点违反了 T 中的关键约束，系统可以中断操作。
- **统计一致性**：在 LLM 系统中，理论 T 的一致性不是定点的，而是被概率维护的。LLM 的推理机制（注意力机制、解码策略、RLHF 偏向）在统计上约束了行为分布——即使 T 在某些点上不一致，LLM 也可能因为训练数据的偏向而选择"合理"的路径。这是一种**统计的而非逻辑的一致性维持**。

DS-TUI 和 omp 都在这三个层次上维护理论一致性。DS-TUI 偏重形式化边界（通过 Rust 的类型系统和编译时检查），omp 偏重运行时检查点（通过工具调用的 schema 验证和审批策略）。两种策略的差异源于各自的语言范式（静态类型 vs 动态类型）和可扩展性需求（封闭系统 vs 开放系统），但它们在模型论框架下可以统一表述为：**理论 T 的一致性不在编译时一次性验证，而是在结构的构造过程中持续维护**。

CHAPTER 02

第二章 Harness 即模型论：Agent 系统的公理化设计

本章建立如下命题：Coding Agent 的 harness 设计，本质上是在构造一个形式系统。提示词工程师选择公理，工具接口定义签名，运行时路由提供解释函数，验证逻辑执行模型检查，扩展机制实现理论扩充。这不是比喻——这些术语在模型论中的精确含义直接映射到代码结构。

2.1 系统提示词作为理论 T

在一阶逻辑中，**理论 (theory)** T 是语言 L 中的一组句子 (sentences) 的闭包。每个句子通过逻辑连接词和量词约束着结构中谓词与函数的解释。在 Agent 系统中，系统提示词就是 T 的载体——它规定了 Agent 的身份公理、协议规则、工具偏好和约束条件。LLM 的每一次推理输出构成了一个结构 M ；当会话正确运行时，我们期望 $M \models T$ 。

2.1.1 DS-TUI：单体公理化与 264 行公理系

DeepSeek-TUI 的系统提示词集中在 `base.md` 中——约 264 行的单体文件，通过 `include_str!()` 在编译时嵌入二进制。这种单体策略的核心优势是**理论一致性**：所有公理在同一逻辑空间中定义，无跨文件依赖导致的隐含不一致风险。

`base.md` 的公理可按逻辑类型归类：

身份公理定义模型 M 的存在性前提：

```
You are DeepSeek TUI. You're already running inside it.  
Do not launch a nested interactive `deepseek` session  
unless the user explicitly asks.
```

这引入了约束 $\neg \exists x (\text{IsNestedSession}(x))$ ——模型必须承认自己已在 TUI 内部运行，禁止递归启动。

协议公理定义推理规则：

```
PREVIEW – Before diving into a large task, survey the terrain.
CHUNK + map-reduce – split into independent sub-tasks.
RECURSIVE – decompose recursively until each leaf is tractable.
```

三种分解模式、验证纪律（verify before proceeding）、路径安全检查（resolve_path）以条件语句的形式约束了推理路径的选择。

工具箱公理定义函数符号的偏好排序：

```
### `apply_patch`
Use `apply_patch` for structural edits, coordinated changes,
or cases where line context matters. Use `write_file` for
brand-new files, full-file rewrites...
```

这是一个**函数选择公理**：对于 $c \in \mathbf{Category}$ ，应使用 f_C 而非 g_C 。它定义了语言 L 中的**标准模型约束**——不是"什么可以做"，而是"做什么时应该用什么工具"。

验证公理施加证据门槛：

```
After every tool call that produces a result you'll act on,
verify before proceeding:
- File reads: confirm the line numbers...
- Shell commands: check stdout, not just exit code...
```

这是理论 T 中最强的约束之一：模型不能仅凭工具名就信任结果，而必须检查输出内容。它引入了一种**认知验证义务**—— M 必须在完成任务之前产生可观察的验证证据。

缓存感知公理是元理论层级的约束：

```
Prefix cache economics. V4 caches shared prefixes at 128-token
granularity with ~90% cost discount. Prefer appending to existing
messages over mutating old ones...
```

这是理论 T 中关于**理论执行成本**的命题。传统模型论不关心公理集的"执行效率"，但在 LLM 系统中，前缀缓存的命中率直接影响系统的经济可行性。 T 不仅控制了 M 的形式，还引导 M 选择成本最低的满足路径。

单体公理化的核心优势在于**前缀缓存效率**：264 行的 base.md 在所有同一版本的会话间共享相同字节前缀，DeepSeek V4 的 128-token 粒度缓存可以跨越用户和会话边界命中。代价是**用户扩展困难**：任何对 base.md 的修改都需要重新编译。

2.1.2 omp：模块化公理化与理论分片

Oh My Pi 以截然不同的方式组织理论 T 。其提示系统不是单体文件，而是通过**技能（skills）**、**规则（rules）**、**目标（goals）**、**记忆（memories）** 四个正交维度组装的。技能系统（`packages/coding-agent/src/extensibility/skills.ts`）从文件系统扫描 `SKILL.md` 文件，规则系统（`rules/` 目录）定义约束性指令，目标系统（`goals/`）描述行为模式，记忆系统（`memories/`）提供跨会话的事实知识。

在模型论中，这构成了**理论的分片（theory fragmentation）**：

$$T_{\text{omp}} = T_{\text{core}} \cup T_{\text{skills}} \cup T_{\text{rules}} \cup T_{\text{goals}} \cup T_{\text{memories}}$$

其中：

$$T_{\text{skills}} = \bigcup_{s \in \text{ActiveSkills}} T_s$$

$$T_{\text{rules}} = \bigcup_{r \in \text{ActiveRules}} T_r$$

- T_{skills} ：激活技能的理论分片。
- T_{rules} ：激活规则的分片。
- T_{goals} 依赖于当前正在执行的会话阶段。
- T_{memories} 是跨会话积累的初等扩充。

关键差异在于**可定义性（definability）**的运行时参数化。在 `system-prompt.md` 中，Handlebars 模板控制哪些部分被注入最终理论：

```
{{#if skills.length}}
# Skills
{{#each skills}}
- {{name}}: {{description}}
{{/each}}
{{/if}}
```

工具 t 在语言 L 中可定义当且仅当存在公式 $\varphi_t(x)$ 使得 $M \models \varphi_t[a] \Leftrightarrow a = t^M$ 。omp 的模板系统保证了：如果工具在当前配置中不存在，任何关于它的公式都不会被注入语言 L 。语言 L 的签名 Σ 在每次会话启动时被**重新构造**。

这种模块化的代价是**理论一致性维护的分布式问题**。 T_{skills} 中的一个文件修改可能打破与 T_{core} 中其他公理的隐含依赖——例如一个技能描述文件修改了某工具的推荐使用模式，但 `system-prompt.md` 中的工具优先级规则未同步更新，则 T 的内部一致性被破坏。从模型论角度看，不一致的公理集可以导出任意命题（ex contradictione quodlibet）。omp 通过**冲突检测和警告机制**（名称冲突产生警告而非崩溃）来缓解这一问题。

两个策略的比较：

维度	DS-TUI（单体）	omp（模块化）
公理源	单一 base.md	多个文件系统源 + 运行时装配
一致性保证	编译时固定	运行时扫描 + 冲突检测
用户可扩展性	有限（需重编译）	充分（安装技能即可）
前缀缓存效率	高（同版本全用户共享）	中（因人而异的前缀）
行为可预测性	高（同一二进制 T 确定）	中（取决于激活的模块集合）

2.2 工具集作为签名 Σ

在模型论中，**签名（signature） Σ** 是一个三元组 (F, R, C) ：函数符号集合、关系符号集合、常量符号集合。Agent 系统的工具集就是 Σ_F ——每个工具的名称、参数约束、返回值类型、副作用声明共同构成该函数符号的**类型结构（arity and sort）**。

2.2.1 DS-TUI：ToolSpec trait 作为编译时契约

DS-TUI 以 Rust trait 定义签名。核心是 ToolSpec (`crates/tui/src/tools/spec.rs`)，其方法精确映射了符号定义各个维度：

- `name()` $\rightarrow$ `&str`：函数符号的字符串标识符
- `description()` $\rightarrow$ `&str`：符号的预期语义描述
- `input_schema()` $\rightarrow$ `Value`：元数（arity）和排序约束（sort constraints），以 JSON Schema 编码
- `capabilities()` $\rightarrow$ `Vec<ToolCapability>`：副作用分类（`ReadOnly`、`WritesFiles`、`ExecutesCode`、`Network` 等），将函数符号映射到特定的权限域
- `approval_requirement()` $\rightarrow$ `ApprovalRequirement`：基于 `capabilities` 自动推导，`ExecutesCode` $\rightarrow$ `Required`，`WritesFiles` $\rightarrow$ `Suggest`——这是权限模型的编译时编码
- `execute(input, context)` $\rightarrow$ `Result<ToolResult, ToolError>`：函数的实现——符号在结构中的解释

ToolContext 结构构成了 Σ_C ——常量符号的编译时集合：

```
pub struct ToolContext {
    pub workspace: PathBuf,
    pub shell_manager: SharedShellManager,
    pub trust_mode: bool,
    pub sandbox_policy: SandboxPolicy,
    pub notes_path: PathBuf,
    pub mcp_config_path: PathBuf,
    pub trusted_external_paths: Vec<PathBuf>,
    pub network_policy: Option<NetworkPolicyDecider>,
    // ...
}
```

这些常量在签名层不可见（不会出现在给 LLM 的 JSON Schema 中），但在解释层可访问。这是编译时的选择：Rust 的类型系统保证所有实现 ToolSpec 的工具都能安全地访问这些常量，且访问路径在编译时确定。

ToolHandler trait (crates/tools/src/lib.rs) 提供了独立的执行接口：

```
#[async_trait]
pub trait ToolHandler: Send + Sync {
    fn kind(&self) -> ToolKind;
    fn is_mutating(&self) -> bool { false }
    async fn handle(&self, invocation: ToolInvocation)
        -> Result<ToolOutput, FunctionCallError>;
}
```

ToolSpec 与 ToolHandler 的分离反映了签名设计中“语言”与“结构”的分离——ToolSpec 定义了语言中的符号（名称、描述、类型约束），ToolHandler 定义了符号在具体结构中的解释（执行逻辑）。两个 trait 通过 ToolRegistry 关联，后者将函数符号名称映射到 handler 实例。

2.2.2 omp: AgentTool 接口与多协议运行时契约

omp 的签名以 TypeScript 接口定义。基础 Tool 接口 (packages/ai/src/types.ts)：

```
export interface Tool<TParameters extends TSchema = TSchema> {
    name: string;
    description: string;
    parameters: TParameters;
    strict?: boolean;
    customFormat?: { syntax: "lark" | "regex"; definition: string };
    customWireName?: string;
}
```

AgentTool 接口（packages/agent/src/types.ts:380-423）扩展了基础 Tool：

```
export type AgentToolExecFn<TParameters, TDetails, TTheme> = (
  this: AgentTool<TParameters, TDetails, TTheme>,
  toolCallId: string,
  params: Static<TParameters>,
  signal?: AbortSignal,
  onUpdate?: AgentToolUpdateCallback<TDetails, TParameters>,
  context?: AgentToolContext,
) => Promise<AgentToolResult<TDetails, TParameters>>;
```

this: AgentTool<...> 的绑定使 execute 函数可以访问工具自身的元数据，AgentToolContext 提供运行时上下文。这种设计将函数符号的实现（execute）、元数据（Tool 接口字段）和运行时上下文（AgentToolContext）清晰地分离到三个不同的类型层次中。

omp 签名设计的关键特性是**参数的模式验证（Schema validation）**和**多协议支持**：

- TSchema 是 Zod schema 和 JSON Schema 的联合类型。validateToolArguments() 函数（packages/ai/src/utils/validation.ts:968-1019）实现了多层次的 coercion：null 标准化 → schema 验证 → 类型修复（字符串"123" → 数字 123）→ 最多重试 MAX_COERCION_PASSES 次。这里的 coercion 机制承认了一个模型论事实：LLM 作为 T 的"证明器"是不完美的。每次 coercion 尝试都在搜索一个"接近"LLM 输出但仍在签名中的结构——这是一种**容错的解释函数**。
- customFormat 和 customWireName 字段允许同一签名 Σ 以不同的语法表示呈现给不同的 LLM。例如，有的模型使用 OpenAI 标准的 JSON 函数调用，有的使用 Anthropic 的 tool_use/tool_result 配对，有的通过 grammar-constrained API 以上下文无关文法表示工具。这是模型论中"不同语言描述同一结构"的工程实现。

2.2.3 必要多样性律的模型论表述

W. Ross Ashby 的必要多样性律指出：控制系统要有效调节其环境，内部状态的多样性必须至少等于环境扰动的多样性。在签名论中，这一规律被重构为：

签名 Σ_F 必须提供足够的函数符号，使得对于论域中每个可执行操作 $o \in \text{Operations}$ ，存在 $f \in \Sigma_F$ 能够表达 o 。

DS-TUI 通过模式驱动的签名选择来管理多样性。ToolRegistryBuilder（crates/tui/src/core/engine/tool_setup.rs:35-103）：


```
let mut builder = if mode == AppMode::Plan {
    ToolRegistryBuilder::new()
        .with_read_only_file_tools()
        .with_search_tools()
        .with_git_tools()
        // Plan 模式不暴露 shell 工具
} else {
    ToolRegistryBuilder::new()
        .with_agent_tools(self.session.allow_shell)
        // ...
};
```

Plan 模式将签名限制为 $\Sigma_{\text{Plan}} \subset \Sigma_{\text{Agent}}$ 。这种子签名关系确保模型 M 不能执行修改操作——不是因为 M 被"教导"不去做，而是因为语言中**没有表达这些操作的符号**。这是通过限制签名而非增加约束来实现的模型强制。

omp 的多样性覆盖更为极端。它支持 32+ 内置工具，加上通过 MCP（Model Context Protocol）动态发现的工具、通过插件系统注册的自定义工具、以及扩展注册的工具。CustomTool 接口（`packages/coding-agent/src/extensibility/custom-tools/types.ts:175-222`）允许任意 TypeScript 模块定义为工具——论域多样性的覆盖理论上无上限：

```
export interface CustomTool<TParams extends TSchema, TDetails> {
    name: string;
    description: string;
    parameters: TParams;
    execute: (
        params: Static<TParams>,
        signal?: AbortSignal,
        onUpdate?: AgentToolUpdateCallback<TDetails>,
    ) => Promise<CustomToolResult<TDetails>>;
    renderResult?: (result: TDetails, options: RenderResultOptions) => string;
    onSession?: (event: SessionEvent) => void;
}
```

onSession 回调允许自定义工具监听会话生命周期事件但不能拦截或修改它们——这是签名扩充时的保守性约束。

2.3 解释：从符号到运行时行为

在模型论中，给定签名 Σ ，一个 Σ -结构 A 包含论域 $|A|$ 以及将每个函数符号 $f \in \Sigma_F$ 映射到 $|A|$ 上运算的解释函数 f^A 。在 Agent 系统中，解释函数将工具名称（符号）映射到实际的计算机操作（运行时行为）。

2.3.1 DS-TUI：静态分发与 trait monomorphization

DS-TUI 的解释函数由 Rust 的静态分发决定。每个工具实现 `ToolSpec` trait，其 `execute` 方法就是解释函数。模型生成 JSON 调用 `{name: "read_file", arguments: {path: "src/main.rs"}}` 后，处理路径是确定性的：

1. **解析**：`tool_parser.rs` 将 LLM 输出解析为 `ToolCall` 结构体
2. **路由**：`tool_routing.rs` 在 `ToolRegistry` 中查找对应 handler
3. **上下文准备**：构建 `ToolContext`
4. **执行**：调用 `ToolSpec::execute(input, &context)`
5. **结果格式化**：`ToolResult { content, success, metadata }`

路径在编译时固定。Rust 的 trait 系统和单态化（monomorphization）意味着每个 `execute` 调用的目标函数在编译时就已知。不存在运行时动态发现新工具的机制（MCP 加载是一次性的，在会话启动时完成）。

静态解释的核心优势是**路径逃逸保护**。`ToolContext::resolve_path()`（`spec.rs:342-403`）在每次文件操作前验证路径是否在 `workspace` 内：

```
pub fn resolve_path(&self, raw: &str) -> Result<PathBuf, ToolError> {
    let candidate = if Path::new(raw).is_absolute() { PathBuf::from(raw) }
                    else { self.workspace.join(raw) };
    // canonicalize and validate against workspace bounds
    if !candidate_canonical.starts_with(&workspace_normalized)
        && !self.is_trusted_external_path(&candidate_canonical)
    { return Err(ToolError::PathEscape { path: candidate_canonical }); }
    Ok(candidate)
}
```

无论 LLM 请求读取什么路径，解释函数都会在执行前验证它是否在允许的论域内。在模型论语境中，这是对 f^A 的定义域约束：解释函数仅在参数满足条件时定义。

2.3.2 omp：动态派遣、coercion 与多供应商容错解释

omp 的解释函数是动态发现的。三个运行时决策路径使解释变得复杂：

MCP 动态发现：`packages/coding-agent/src/mcp/` 在运行时通过 MCP 协议发现外部工具服务器。工具签名在连接时获取而非编译时已知。

插件动态导入：`packages/coding-agent/src/extensibility/plugins/loader.ts` 展示了核心模式：

```
function resolveManifestEntryFile(joined: string): string | null {
  for (const name of ["index.ts", "index.js", "index.mjs", "index.cjs"]) {
    const candidate = path.join(joined, name);
    if (fs.existsSync(candidate)) return candidate;
  }
  // ...
}
```

运行时扫描文件系统，动态导入 TypeScript/JavaScript 模块。每个模块可以注册新的函数符号。

扩展的动态加载：packages/coding-agent/src/extensibility/extensions/loader.ts 展示了模块被评估为工厂函数或默认导出的灵活模式。

动态解释的代价是类型安全较弱，错误可能在运行时才暴露。但它的收益是**多供应商容错**——omp 支持 40+ LLM 供应商，每个供应商的消息格式不同。packages/agent/src/agent-loop.ts 中的 normalizeTools() 函数将工具定义统一为五种消息类型的归一化表示：

1. tool_use / tool_result - Anthropic 风格
2. function_call / function - OpenAI 风格 (旧版)
3. tool_calls / tool_call - OpenAI 风格 (新版)
4. custom wire format - 自定义协议 (grammar-constrained)
5. plain text parsing - 无元协议的模型 (依赖输出解析)

归一化层是一个**多协议解释函数** I_{multi} ，它将不同供应商的格式映射到内部统一的 ToolCall 结构。在模型论中，这相当于：给定一组不同的语法表示 $\{L_1, L_2, \dots, L_n\}$ ，存在解释函数 I_i 将每个 L_i 映射到共同的语义结构 A 。只要 $\forall i. I_i(L_i) = A$ ，就存在一个**共同的语义核心**，不同"语言"的描述指向同一结构。

2.3.3 解释策略对比

特性	DS-TUI (静态解释)	omp (动态解释)
工具发现时机	编译时 + 启动时一次 MCP 加载	启动时 + 运行时插件/扩展加载
类型安全	Rust 编译器保证	TypeScript + 运行时 schema 验证
新工具添加	需要重新编译	安装插件/技能/扩展即可
安全保证	编译时路径锁定	运行时 coercion + schema 检查
解释函数可变性	编译后不可变	运行时通过插件更新
多供应商支持	专为 DeepSeek 优化	40+ 供应商统一接口

2.4 理论修正：反馈作为模型校正

经典模型论中，理论 T 一旦选定就固定了。工程实践中，H(LLM) 系统的一个核心特征是其**反馈回路**：系统观察 LLM 的行为偏差，然后以两种方式修正：

- **收缩理论**：在不扩展语言的情况下增加约束语句，削除偏离的模型
- **扩充语言**：添加新的函数符号或约束类型，扩展可表达性的边界

两种修正策略对应不同的工程机制。

2.4.1 收缩理论：DS-TUI 的 capacity_controller

DS-TUI 的 capacity controller (crates/tui/src/core/engine/capacity.rs) 监视会话的 token 消耗模式，当检测到模型在超出上下文窗口限制的方向上运行时，增加约束语句来限制模型的思维广度。

收缩理论的逻辑形式：

$$T' = T \cup \{\varphi\}$$

其中 φ 是一个新约束语句（如 "Limit your analysis to the specific file changed"），不引入新符号。新理论 T' 的模型集是 T 的子集： $\text{Mod}(T') \subset \text{Mod}(T)$ 。约束完全通过排除不满足 φ 的模型来工作。

capacity_controller 在以下条件下执行收缩：

- **Token 预算预警**：当前轮次消耗接近阈值时，注入系统级的 token 约束
- **注意力扩散检测**：当模型的分析跨越过多文件时，施加减量约束
- **递归深度限制**：限制模型的自引用分解深度

```
// 核心逻辑：在系统提示词后追加约束
if should_restrict(current_depth, budget_remaining) {
    messages.insert(system_idx + 1,
        Message::system("IMPORTANT: Be concise. Focus on the specific task."));
}
```

收缩策略的优点是**理论一致性不受影响**： T' 只是 T 的特化，不改变原有函数符号的含义。代价是**可能过度限制**： φ 可能将合法模型排除在外。

2.4.2 扩充语言：omp 的 Harmony 泄漏检测

omp 的 Harmony 泄漏检测（`packages/ai/src/utils/harmony-leak.ts`）处理的是语言层次混淆。GPT-5 在特定条件下会在输出文本中泄漏内部协议标记——`to=functions.<tool_name>` 模式、`<|start|>/<|end|>/<|channel|>` 标记、以及幽灵 token 如 `changedFiles`、`RTL`、`Jsii`。

检测器定义了七种信号类别：

```
const MARKER_RE = /\bto=functions\.[A-Za-z_]\w*/g; // M: 标记器
const HARMONY_RE = /<\|(start|end|channel|message|call|return)\|>/g; // 直接 Harmony 令牌
const CHANNEL_WORD_RE = /\b(?:analysis|commentary|assistant|user|system|developer|tool)\s+to=functions\./; // C: 通道词
const GLITCH_RE = /\b(?:changedFiles|RTL|Jsii(?:_commentary)?|\x4aapgolly)\b/; // G: 幽灵 token
const BODY_CASCADE_RE = /to=functions\.\w+\s+code\b[\s\S]{0,200}?to=functions\./; // B: 体级联
const FAKE_RESULT_RE = /to=functions\.\w+[\s\S]{0,80}?code_output\s*\nCell\s+\d+:/; // R: 假结果
```

在模型论中，Harmony 泄漏是一个语言层次混淆的深刻案例。LLM 的内部推理协议泄露到了输出语言中——模型的内部语言 L_{internal} （用于规划工具调用）污染了对外语言 L_{external} （对用户可见的输出文本）。两个不同语言层次的符号在同一个论域中出现了混淆。

`recoverHarmonyToolCall()` 函数实现了恢复逻辑：

```
export function recoverHarmonyToolCall(
  message: AssistantMessage,
  detection: HarmonyDetection,
): HarmonyRecoveredToolCall | undefined {
  // 截断至污染前最后一个合法字符边界
  // 插入哨兵标记要求模型重新提交
}
```

恢复机制是语用性的而非语义性的：它在污染点截断输入，而非解析被污染输出来提取有效部分。这是**安全退化（graceful degradation）**——承认某些偏差无法在语义层修复，必须在语用层处理。

2.4.3 两种修正策略的模型论分析

维度	收缩理论 (DS-TUI capacity_controller)	扩充语言 (omp harmony-leak)
修正对象	模型的注意力广度	模型的输出格式
操作	$T' = T \cup \{\varphi\}$ (加约束, 语言不变)	$L' = L + \{\text{detect}, \text{recover}\}$ (加符号, 表达能力扩展)
模型集关系	$\text{Mod}(T') \subset \text{Mod}(T)$	$\text{Mod}(\Sigma') \subset \text{Mod}(\Sigma)$ (通过新符号检测违规)
一致性影响	无 (纯粹化)	有 (新符号需纳入理论)

维度	收缩理论 (DS-TUI capacity_controller)	扩充语言 (omp harmony-leak)
失败模式	过度约束排除合法行为	漏检 / 误恢复导致状态混乱

收缩理论是**被动-预防型**的——在偏差发生前增加约束。扩充语言是**主动-检测型**的——在偏差发生后识别并恢复。两个策略对应了不同层的反馈：系统层 vs 模型层。

2.5 保守扩充 vs 非保守扩充

在模型论中，给定理论 T 和其扩充 $T' = T \cup S$ ， T' 是 T 的**保守扩充 (conservative extension)** 当且仅当对 T 语言中的任意语句 φ ： $T' \models \varphi \Rightarrow T \models \varphi$ 。新公理不改变原理论中已可证明的命题。

在 Agent 系统中，保守扩充是扩展机制的理想目标：添加新工具、技能或插件后，系统应能执行所有原有操作，且原有行为不被改变。

2.5.1 DS-TUI 的分层叠加：编译时的保守扩充

DS-TUI 的提示词组合在确定性层叠顺序 (prompts.rs:439–447) 中执行：

1. base.md – 核心身份、工具箱、执行契约
2. personality – 声音与语调覆盖层
3. mode_delta – 模式特定权限与工作流
4. approval_policy – 工具批准行为

其组合规则保证下层的扩充是上层的保守扩充：

$$T = T_{\text{base}} \cup T_{\text{personality}} \cup T_{\text{mode}} \cup T_{\text{approval}}$$

其中各子理论按优先级排序。模式定界可以**限制**基础公理（如 plan 模式限制了执行权限），但不可**否定**基础公理——这是保守扩充的核心保证。

L_{base} 是语义核心。 $L_{\text{personality}}$ 添加语气约束但不改变工具语义。 L_{mode} 可能**移除**某些函数符号 ($\Sigma_{\text{Plan}} \subset \Sigma_{\text{Agent}}$)，但这是**符号的隐去而非否定**——不改变保留符号的解释。

实现层，DS-TUI 的技能系统也维持了保守性：

- 技能只追加信息到理论 T ——它们指导模型如何思考特定领域问题，但不修改任何现有工具的解釋
- 技能不注册新的函数符号——如果需要执行操作，必须使用已有工具
- 名称冲突通过 HashMap 去重（先加载的优先）

- 搜索深度限制（MAX_DISCOVERY_DEPTH：8）防止循环和路径爆炸

2.5.2 omp 的运行时注入：受控的非保守扩充

omp 的扩展机制更为复杂，因为插件和自定义工具可以直接在运行时添加新的函数符号。保守性的维持需要通过**设计约束**来保证。

插件清单（Plugin Manifest） 定义了插件的入口点：

```
export interface PluginManifest {
  tools?: string | string[];
  hooks?: string | string[];
  commands?: string | string[];
  extensions?: string | string[];
  settings?: PluginSettingSchema[];
  features?: PluginFeature[];
}
```

关键设计：插件只能通过**声明**添加功能，不能动态修改已有功能的实现。插件清单中的每个条目指向一个独立的模块文件，以工厂函数形式导出实现：

- 新增工具不覆盖现有工具（注册表使用名称去重）
- 钩子作为事件处理程序链运行，不阻止其他处理程序
- 扩展通过事件总线接收生命周期事件，不能篡改核心循环

权限的层次限制是维持保守性的关键原则。hooks/types.ts 的注释直白地说明了这一点：

```
// Parallel to ExtensionUIContext: hooks expose a deliberately narrower UI
// surface – no terminal-input listener, no editor component override, no
// theme management – because hooks are invoked from inside the agent loop
// and must not be able to seize ownership of the editor.
```

钩子被调用的上下文在 agent 循环内部，因此它们的 UI 权限被**故意收窄**——不能获取终端输入、不能覆盖编辑器组件、不能管理主题。这种最小权限设计确保钩子始终是保守的：它们观察到系统状态但不能劫持核心功能。

CustomTool 接口的 onSession 回调允许工具监听会话生命周期事件（session_start、session_shutdown、session_switch 等），但不能拦截或修改事件传播。这维持了保守性：工具可以**响应**系统状态变化但不能改变传播路径。

CustomToolAPI 的 persist 接口是工具获取持久化状态的唯一途径：


```
export interface CustomToolAPI {
  persist: {
    get(key: string): Promise<unknown>;
    set(key: string, value: unknown): Promise<void>;
    delete(key: string): Promise<void>;
  };
}
```

它不能访问文件系统、不能执行 shell 命令、不能修改会话状态。这种沙盒化确保了即使工具包含 bug 或恶意逻辑，也不能破坏系统的保守性。

2.5.3 两类扩充与非保守性

DS-TUI 的扩充是编译时确定的、分层的、**必然保守的**：模式层不能否定核心层，技能层不能增加新符号。任何违反这一原则的扩展都无法通过编译。

omp 的扩充是运行时注入的、分权的、**偶发非保守的**：理论上，一个足够权限的插件可以注册与现有工具同名的 handler 来"劫持"工具调用（名称不冲突的注册表设计只阻止了同名注册，但确实有重构可能）。实际上，工程上的分层限制（钩子权限窄化、事件分发自省、持久化沙盒）将非保守行为的风险降到可接受水平。

两者的差异根植于不同哲学：

维度	DS-TUI	omp
扩充时间	编译时	运行时
扩充方式	分层叠加	插件 / 技能 / 扩展注入
一致性保证	Rust 类型系统 + 编译检查	运行时注册表 + 权限约束
非保守风险	几乎为 0	低但存在（通过设计约束控制）
用户灵活性	低（需重编译）	高（即装即用）

2.6 小结

本章建立了从模型论到 harness 工程的核心映射：

1. **系统提示词 = 理论 T** ：DS-TUI 的 264 行单体公理系 vs omp 的模块化理论分片（skills + rules + goals + memories）

2. **工具集 = 签名 Σ** : DS-TUI 的 ToolSpec trait (编译时契约) vs omp 的 AgentTool 接口 (多协议运行时契约)
3. **工具执行 = 解释函数**: DS-TUI 的静态分发 (trait monomorphization) vs omp 的动态派遣 (多供应商归一化 + coercion)
4. **反馈 = 理论修正**: DS-TUI 的 capacity_controller (收缩理论) vs omp 的 harmony-leak (扩充语言)
5. **扩展机制 = 保守扩充**: DS-TUI 的分层叠加 (编译时保守) vs omp 的运行时注入 (受控的非保守)

这些映射揭示了一个根本性事实：Harness 设计是模型论在工程中的隐式实践。提示词工程师选择公理，类型系统编码签名，运行时路由实现解释，验证逻辑执行模型检查，扩展机制提供保守或非保守的扩充。理解这一形式化底座，是分析、比较和设计下一代 Agent 系统的前提。

II

第二篇

系统论架构

以整体性、层次性、开放性与目的性组织分析。

CHAPTER 03

第三章 整体性：理论的相容性与合并

整体性（wholeness）是系统论的第一原理，也是模型论中理论合并问题的工程具象。从钱学森“整体大于部分之和”到数理逻辑“合取公理系统的一致性”——本章将系统论的整体性直觉转化为精确的模型论语言：系统整体性的维护等价于理论 T 的一致性维护，系统整体性的崩溃等价于理论 T 的矛盾推导。DS-TUI 与 omp 分别代表了两种不同的一致性保证策略——编译时证明与运行时检测——而二者共同面对的根本挑战是：当系统以分布式、模块化、开放的方式组织理论时，没有任何全局的“理论检查器”能够保证整体的一致性。

3.1 系统论整体性原理的模型论重述

3.1.1 从“整体大于部分之和”到“理论合并的一致性”

钱学森断言：系统的整体性质并非各子系统性质的简单加和。设系统 S 的子系统集合为 $\{S_1, \dots, S_n\}$ ，每个子系统 S_i 对应一个理论 T_i 。系统的整体行为由合并理论 $T_{\text{union}} = \bigcup T_i$ 描述。整体性条件的三个层次可表述为：

整体性条件（WC-1）： 合并理论 T_{union} 必须是一致的——不存在公式 φ 使得 $T_{\text{union}} \vdash \varphi$ 且 $T_{\text{union}} \vdash \neg\varphi$ 。在经典一阶逻辑中，不一致的理论推出任意公式——这是“系统整体崩溃”的模型论对应。

整体性条件（WC-2）： 如果子系统之间存在共享符号 $\Sigma_{\text{shared}} = \Sigma(T_i) \cap \Sigma(T_j) \neq \emptyset$ ，则 T_i 与 T_j 在共享符号上的解释必须一致。这在软件工程中对应接口契约——两个模块共享的接口类型、函数签名必须语义一致。

整体性条件（WC-3）： 存在在任意单独 T_i 中不可表达的公式 φ ，但在 T_{union} 中 φ 可被证明——即 $T_{\text{union}} \vdash \varphi$ 但对所有 i ， $T_i \not\vdash \varphi$ 。这正是“整体大于部分之和”的模型论解释： φ 不是预先插入系统的，而是在多子系统相互作用中涌现出的行为规律。

3.1.2 时间维度的一致性

经典模型论中的理论闭包是自足的： $\text{Cn}(T)$ 由 T 的公理集完全决定。但对于开放的复杂巨系统，理论 T 并非固定的——新子系统（MCP 服务器、技能、规则）会不断向 T 添加新公理：

$$T_t = T_{\text{core}} \cup \left(\bigcup_{s \in \text{ActiveSkills}(t)} T_s \right) \cup \left(\bigcup_{r \in \text{McpConnected}(t)} T_r \right)$$

每个时间点 t 维护一个不同的合并理论。整体性条件必须对每个 T_t 成立——静态一致性被推向了动态一致性。系统必须有能力检测并响应合并理论的变化。这正是 DS-TUI 与 omp 在架构上的根本分歧所在。

3.2 理论的相容性：DS-TUI 的单体保证

3.2.1 编译时公理固定

DS-TUI 的策略极端而优雅：**将所有公理在编译时固定**。系统提示词 `base.md` (~264 行) 通过 `include_str!()` 在编译时嵌入。工具签名通过 `ToolSpec` trait 在编译时确定。配置模型通过 `ConfigToml` 在编译时类型检查。

在模型论语境中这意味着：合并理论 T_{DS} 的公理集在编译时完全确定。不存在运行时添加公理的路径。Rust 编译器的作用不仅是翻译——它充当了**理论一致性的证明器**：

- 类型检查 $\approx$ 验证项在签名 Σ 中的良构性
- `traitbound` $\approx$ 验证可定义性约束
- 借用检查 $\approx$ 验证状态转换路径的安全性
- 编译错误 $\approx$ 证明某合并理论不一致，不存在合法的结构 M

3.2.2 Engine 结构体的统一所有权

整体性的最具体体现是 Engine 结构体——19 个字段的集中式运行时核心。在系统论语境中，这是集中式控制架构；在模型论语境中，19 个字段对应合并理论 T_{DS} 的论域中 19 个常元符号。

优势在理论一致性层面清晰：任何对 Engine 状态的操作都在同一个可变借用范围内，Rust 的类型系统保证不会出现两段代码同时修改不同字段时破坏共享不变量。这是 WC-1 在代码层面的编译器证明。

但代价同样源于同一性质：19 个字段之间的隐式依赖关系网——`config` 变化需 `deepseek_client` 同步更新——不是通过类型系统显式表达的。这些隐式依赖构成了 T_{DS} 的**非逻辑公理**：编译器无法验证它们的一致性。

3.2.3 ToolSpec trait：单一契约与隐式耦合

工具系统的整体性由 ToolSpec trait 保证。每个工具实现 `name()`、`description()`、`input_schema()`、`execute()` 等方法——这是签名 Σ 的公理化定义。工具调用经过符号解析、类型检查、能力检查、执行四个阶段，前三个阶段发生在执行之前——这是理论核实而非执行。

但在整体性方面，ToolSpec 有一个结构性弱点：工具间的隐含依赖不在契约范围内。`search` 可能调用 `read_file` 读取上下文；`apply_patch` 可能调用 `read` 获取当前状态。这些工具间的调用链形成了新的合并理论——两个函数的复合产生了单个签名中不可见的约束。

更危险的隐式耦合体现在"超级 impl 块"模式。`turn_loop.rs` 中的 `impl Engine` 扩展 (~1910 行) 在 `Engine` 命名空间中直接操作其私有字段，同时处理流式事件路由、工具规划、LSP 钩子、容量检查点和循环终止。任何子系统的变更都可能通过 `Engine` 共享状态传播到其他子系统。在模型论语境中，这意味着 T_{DS} 的公理集被线性化到同一个证明空间中——不存在子理论的隔离边界。

3.3 理论的分片与合并：omp 的模块化挑战

3.3.1 多源理论装配

omp 的策略截然不同：理论 T_{omp} 由多个文件系统源在运行时装配为**理论分片**：

$$T_{omp} = T_{core} \cup T_{skills} \cup T_{rules} \cup T_{goals} \cup T_{memories} \cup T_{mcp}$$

每个分片的公理集在运行时被扫描、解析、装配。Handlebars 模板控制最终的注入：如果当前工作空间没有技能， $T_{skills} = \emptyset$ ——语言 L_{omp} 中甚至不包含技能相关的关系符号。工具 t 可定义当且仅当它在当前配置中存在——签名 Σ_{omp} 随运行环境自适应变化。

3.3.2 四个正交维度的理论交互

Skill/Rule/Goal/Memory 是设计上的正交抽象，分别对应知识公理、约束公理、偏好公理、事实公理。理想情况下 $T_i \cap T_j = \emptyset$ ；但实践中存在大量隐式交叠——一个 `rules/` 文件要求"使用说明性提示词"而一个 `skills/` 文件示例了指令性提示词（共享论域上产生分歧）；`memory://` 中"偏好 Claude"与 `rules/` `must-use-openai.md` 矛盾。这些冲突是语义冲突，而非语法错误——合并理论包含了两组矛盾约束。

3.3.3 一致性维护的分布式问题

omp 通过三个机制缓解而非消除不一致风险：

优先级排序：不同理论源被赋予显式优先级（规则高于技能，核心提示高于一切）。这是"公理排序"的工程实现——当两个子理论冲突时，优先级高的胜出。

冲突检测与警告：名称冲突在运行时被检测并产生警告而非崩溃——对 WC-1 的"软响应"。

分离的装配路径：不同分片通过不同 Handlebars 槽位注入同一个模板——提供文本隔离而非语义隔离。

但这些机制无法从根本上解决分布式一致性问题：当新技能 `.skill` 包被安装时，没有自动流程验证 $T_{\text{skills_new}}$ 与 T_{core} 的语义兼容性；当新 MCP 服务器连接时，没有运行时检查验证符号引用不冲突。整体性退化为"谁最后测试通过谁负责"——工程上的退化，而非原则上的保证。

3.4 涌现行为的模型论解释

3.4.1 工具系统作为涌现载体

WC-3（存在在单独 T_i 中不可表达的公式 φ 但在 T_{union} 中可推导）在 Coding Agent 中最基础的载体是工具系统。单一 `read_file(path)` 是原子公式——单独看没有工程价值。但组合产生了单个签名中不可见的行为：

$$\varphi_{\text{code_edit}} : \text{search}(\text{"TODO"}) \rightarrow \text{read}(\text{files}) \rightarrow \text{edit}(\text{lines}) \rightarrow \text{verify}()$$

$\varphi_{\text{code_edit}}$ 不能在单个工具的公理集中表达。但在 T_{union} 中——当 `search`、`read`、`edit`、`verify` 共存于同一签名 Σ 中—— $\varphi_{\text{code_edit}}$ 成为可推导的行为模式。工具签名 Σ_F 是"涌现的语法条件"：它定义了哪些原子操作可用，但不定义它们如何组合。组合方式由 Turn 循环（`turn_loop.rs` / `agent-loop.ts`）提供。

3.4.2 DS-TUI 的三层涌现

涌现 1——代码编写。`search`、`read`、`edit`、`apply_patch` 的组合构成完整代码编写 workflow。`base.md` 的 `PREVIEW` $\rightarrow$ `CHUNK` $\rightarrow$ `RECURSIVE` $\rightarrow$ `VERIFY` 公理缩小了"合法证明"的空间。

涌现 2——项目管理。`git` 工具、`shell` 工具、`LSP` 工具的组合产生了跨工具、跨文件的涌现能力：项目结构理解、构建依赖分析、重构规划。任何单一工具都无法提供"理解项目架构"的功能。

涌现 3——自我认识。当 LLM 使用 `<thinking>` 组织内部推理、`<diagnostics>` 注入检查结果、`<turn_meta>` 标记元信息时——它在观察自身行为。`LoopGuard` 通过 `IDENTICAL_CALL_BLOCK_THRESHOLD = 3` 检测重复调用，系统识别出"我陷入了循环"并主动打破。这是元认知的工程雏形。设 $T_{\text{loopGuard}}$ 是 `LoopGuard` 的行为约束理论——它不是工具，是系统对自身证明空间施加的元理论约束。

3.4.3 omp 的涌现：Harmony 泄漏检测

Harmony 泄漏检测是由五个正则表达式、三状态机和姿态检查组合而成的涌现行为：

$$\varphi_{\text{harmony_detect}} : (\exists \text{leak_signal} \wedge \neg \text{false_positive_pattern} \wedge \text{state_consistent}) \rightarrow \text{trigger_recovery}$$

每个组件单独无意义：正则模式匹配"泄露信号"但可能误报；状态机追踪 delta 节流模式但可能在复杂交互中触发；工具调用频率姿态检查无法区分正常和异常调用。组合后的系统具有单个组件不具备的特性：**低误报率的近实时泄漏检测**。

恢复机制的嵌套控制回路（abort_retry → truncate_resume → escalated）进一步体现了涌现的层次性——每一级都在更大时间尺度和更高决策权限上操作。这是整体性原理在工程中的正面表达：功能不是被"实现"的，而是在多组件相互作用中**涌现**的。

3.5 上下文压缩作为整体性维护

3.5.1 压缩即理论约简

当 T_t 的大小超过上下文窗口容量时，系统面临选择：截断（丢失公理，破坏整体性）或压缩（重写信息）。压缩是寻找一个约简理论 T_{reduced} ，使得：

$$T_{\text{reduced}} \subseteq \text{Cn}(T_t) \wedge |T_{\text{reduced}}| < \text{capacity}$$

T_{reduced} 是原理论演绎闭包的子集，其"大小"在容量限制内。压缩操作保持近期交互和关键决策路径的演绎强度，丢弃早期对话和成功细节的弱推论。

3.5.2 DS-TUI：前缀缓存对齐压缩

压缩策略由缓存命中率驱动。should_use_cache_aligned_summary() 决定是否采用"回放原始消息 + 附加总结指令"路径——让 V4 前缀缓存在压缩后继续命中。缓存命中率充当了理论一致性的**间接度量**：

- 高缓存命中率 → 理论 T_t 的结构完整性得以保持
- 低缓存命中率 → 前缀已漂移 → 整体性已被破坏

注释中的"前缀漂移"是整体性退化的早期信号。MINIMUM_AUTO_COMPACTION_TOKENS = 500000 阈值的核心理由是"缓存已有压力，前缀已经漂移"——压缩是必要的理论重建操作。

3.5.3 omp：多策略压缩

omp 采用多策略组合：LLM 摘要（调用 LLM 找出 T_{reduced} 覆盖 T_t 大部分结论）、OpenAI 远程压缩（将约简委派给外部系统）、本地修剪（选择性地放弃信息熵最低的公理）。每种策略的信息保持率不同——摘要保持语义但可能丢失精确事实，修剪反之。

在系统论视角中，omp 的压缩是**分布式整体性维护**——不依赖全局一致性检查器，通过多个独立机制共同维护系统工作记忆边界。DS-TUI 相信单点的全局最优（缓存命中率）；omp 相信多点协商的局部自适应（供应商能力 + 上下文状态）。两条路径各有代价。

3.6 MCP 作为跨系统理论的桥接

3.6.1 两个理论的合并

MCP 连接的本质是**两个理论的合并**：

$$T_{\text{merged}} = T_{\text{harness}} \cup T_{\text{mcp}}$$

T_{mcp} 的函数符号通过标准协议注入 T_{harness} 的签名域。DS-TUI 将外部工具内化为 ToolSpec 实现（经 McpToolAdapter 适配，注册路径与内置工具一致），omp 在运行时通过边界守卫（如 `coerceToolResult()`）防御性规范化。

3.6.2 合并一致性的保证责任

核心问题是：合并后的一致性由谁保证？DS-TUI 和 omp 都缺乏形式化验证。假设 T_{harness} 定义 `read_file(path)` 返回纯文本， T_{mcp} 定义同名工具返回 JSON——合并后 LLM 无法确定符号的预期语义。

在经典一阶逻辑中，跨理论一致性验证只需检查 $\text{Cn}(T_{\text{harness}}) \cap \text{Cn}(T_{\text{mcp}})$ 中是否存在矛盾公式对。但 MCP 中 T_{mcp} 以 JSON + 自然语言编码，不存在形式化的理论语言。一致性验证退化为人程序员审查。

3.6.3 动态合并与运行时漂移

MCP 引入时间维度的一致性挑战： T_{mcp} 不是静态的——服务器升级可能改变签名。

$$T_{\text{merged}}(t_1) = T_{\text{harness}} \cup T_{\text{mcp_v1}}$$

(重启后)

$$T_{\text{merged}}(t_2) = T_{\text{harness}} \cup T_{\text{mcp_v2}}$$

两个系统无完美方案。DS-TUI 在会话启动时一次性发现工具后冻结，牺牲灵活性保全跨时间一致性。omp 的工具桥接层在每次调用时重新获取签名，提高灵活性但无法通知 LLM 工具集已变化。

这是整体性原理在开放系统中的最终约束：**没有什么可以保证 T_{merged} 在任意时刻的一致性，除非限制开放的程度。**

3.7 小结

1. **整体性 = 合并理论的一致性。**系统整体性的维护等价于 T_{union} 的相容性维护；整体性崩溃等价于 T_{union} 的不一致——ex contradictione quodlibet 对应系统行为的不可预测性。
2. **DS-TUI 选择编译时证明：**通过 Rust 类型系统、ToolSpec trait、Engine 统一所有权和编译时固定的 `base.md` 在编译时证明了大量整体性条件。代价是运行时扩展性的丧失和超级 `impl` 块中的隐式耦合。
3. **omp 选择运行时检测：**通过理论分片、动态装配、优先级排序和冲突警告在运行时维持整体性。代价是缺乏全局验证器——一致性维护退化为工程审查。
4. **涌现是整体性的正面表达。**工具组合的三层涌现（DS-TUI）和 Harmony 泄漏检测（omp）都是在合并理论中才可表达的性质——WC-3 的工程实现。
5. **上下文压缩是理论约简。**DS-TUI 的前缀缓存对齐（集中式、缓存命中率驱动）与 omp 的多策略组合（分布式、供应商自适应）以不同方式实现了"在有限容量内保持最大演绎闭包"的目标。
6. **MCP 桥接是跨理论合并。** T_{harness} 与 T_{mcp} 的合并引入签名交叠检测、运行时漂移、跨时间一致性等新问题——系统从封闭到开放时整体性条件的质变。

下一章——**层次性**——将探讨 Coding Agent 中"理论塔"的结构与涌现的层次化组织。

CHAPTER 04

第四章 层次性：理论塔

系统论层次性原理断言：复杂系统必然呈现分层结构。模型论将这一原理重述为理论塔（theory tower）——每层是一个一阶理论，上层是下层的保守扩充。本章检验 DS-TUI 和 omp 的工程层次结构，证明它们形式上等价于 $T_0 \subset T_1 \subset T_2 \subset \dots$ 的嵌套理论序列，并揭示时间尺度约束如何驱动层次收敛。最后诊断层次过载问题——当单层理论承载了不应属于该层的公理时，系统的非保守性导致工程退化。

系统论层次性原理及其模型论重述

层次性：复杂系统的必然结构

钱学森在《创建系统学》中提出，复杂系统必定呈现分层结构。这一论断并非经验归纳——它有深刻的动力学根源。**时间尺度的分离**是所有分层结构的物理基础：一个系统中的不同过程如果特征时间差异超过一个数量级，它们之间就不可能存在直接的因果耦合。快速过程在慢速过程的"时间窗口"中表现为瞬态（近似瞬时响应），慢速过程在快速过程的"时间窗口"中表现为常数或边界条件。强行将它们放在同一层，要么导致快速决策被慢速控制拖慢，要么导致慢速积累被快速噪声淹没。

工程系统的层次性正是对物理时间尺度分离的回应。DS-TUI 和 omp 的开发者并没有"决定"构建五层结构——他们从无层结构出发，经过数十次重构迭代，逐渐"发现"了这五层。层次性不是人为设计的产物，而是**解决问题时自然出现的结构**，正如生物进化中并非有某个智慧体决定了脊椎动物的身体分节。

模型论重述：理论塔

将系统论层次性转换为模型论语言。给定一个复杂系统，定义其行为空间为：

- **语言 L** ：系统所涉及的全部概念符号——工具名称、参数类型、事件类型、配置字段。
- **论域 $|M|$** ：系统运行时触及的所有实体——消息、文件内容、工具结果、会话状态。
- **理论 T** ：约束系统行为的全部公理——系统提示词、类型约束、运行时代码逻辑。

层次性原理断言： T 可以（并且事实上必须）分解为一个嵌套的理论塔：

$$T_0 \subset T_1 \subset T_2 \subset \cdots \subset T_n$$

其中每个 T_k 包含 T_{k-1} 的全部公理（作为子理论），并添加了 k 层特有的新公理。且关键条件是： T_k 是 T_{k-1} 在 L_{k-1} 语言上的保守扩充——在 k 层添加的新公理，不会改变在原语言 L_{k-1} 中可推导的句子。

这个保守性条件的工程含义是深刻的：它意味着从 T_0 出发，每一层只扩展系统能“谈论”的新能力，而不回溯修改下层已经建立的事实。具体到 Harness 系统：

- T_0 （工具层）的公理声明了哪些工具可用。 T_1 不能否定 T_0 中已经声明的工具——“read_file 是可用工具”一旦在 T_0 中建立， T_1 中的任何公理都不能撤销这一事实。
- T_1 可以在 T_0 基础上添加“单一工具调用如何组合成 Turn”的公理。但 T_1 不能修改 T_0 关于工具签名的定义——“read_file 接受一个 path 参数”是 T_0 语言中的句子， T_1 无法改变它。
- T_2 在 T_1 基础上添加“何时压缩上下文”的公理。但 T_2 不能撤销 T_1 中已声明的 Turn 周期语义——每个 Turn 仍然是一个“消息→响应→工具→结果”的完整单元。

理论的保守扩充形成了一个单调增长的推理能力： T_0 能处理单次工具调用； T_1 能处理一个交互单元的编排； T_2 能处理跨单元的状态管理。更高层不仅有更丰富的语言，还有更高阶的推理模式——从“执行”到“编排”到“管理”。

理论塔与层次控制的对应

系统论中的控制关系与模型论中的理论塔具有精确对应：

系统论概念	模型论对应
下层支撑上层	T_k 的语言 L_k 包含 L_{k-1} 的全部符号
上层控制下层	T_k 的公理约束 L_{k-1} 中符号的合格使用方式
层间接口	保守扩充条件——下层事实在上层保持不变
跨层交互	非保守扩充（向下的因果影响）——应尽量避免

这一对应将层次性从“架构原则”升级为可验证的形式性质。一个工程系统是否符合层次性原理，可以用理论塔的保守性来判定：如果对任意 T_k 中的公理，存在某个在 L_{k-1} 语言中的句子 φ 使得 $T_k \models \varphi$ 但 $T_{k-1} \not\models \varphi$ ，则 T_k 不是 T_{k-1} 的保守扩充——系统存在“层间泄漏”。

DS-TUI 的理论塔

将 DS-TUI 的五层物理结构（第三章 3.2 节）重述为嵌套理论：

L_0 ：工具调用理论 T_0

语言 L_0 ：工具名称（`read_file`, `search`, `edit`, `exec_shell`, `bash` 等）、参数类型（`InputSchema` 中的字段）、返回值类型。

论域 $|M_0|$ ：文件系统路径、文件内容字符串、命令输出、搜索结果列表。

公理集 T_0 ：

公理 1：工具执行公理

$$\forall \text{tool} \in \text{ToolRegistry} : \text{tool.execute}(\text{arg}) \rightarrow \text{output}$$

公理 2：自描述公理

$$\forall \text{tool} : \text{ToolSpec} :: \text{spec}() \rightarrow \text{tool.definition}$$

公理 3：特定工具行为公理

$$\text{execute}(\text{read_file}, \text{path}) = \text{file_content}$$

公理 4：搜索公理

$$\text{execute}(\text{search}, \text{pattern}, \text{paths}) = \text{match_results}$$

公理 5：编辑公理

$$\text{execute}(\text{edit}, \text{path}, \text{old}, \text{new}) = \text{success_flag}$$

公理 6：Shell 执行公理

$$\text{execute}(\text{exec_shell}, \text{cmd}) = \text{cmd_output}$$

T_0 的时间尺度是毫秒到秒级。它不关心“多个工具调用怎么组合”“上下文满了怎么办”“用户中断了怎么办”。它只回答一个问题：给定一个结构化的工具调用请求，返回结构化的执行结果。这是系统的最底层——所有上层推理最终归结到这里。

T_0 的保守性条件： T_0 中的公理仅在 L_0 语言中定义。上层不能改变 T_0 关于"read_file 接受一个 path 参数"的定义——这是不可撤销的。

L_1 : Turn 理论 T_1

语言 L_1 : $L_0 \cup \{\text{Turn}, \text{Message}, \text{AI_Response}, \text{ToolCallEvent}, \text{Failure}, \text{Retry}, \text{Checkpoint}\}$ 。

公理集 T_1 : 在 T_0 上加入以下公理。

公理 7

$$\text{Turn} = \text{Message_in} \rightarrow \text{AI_Response} \rightarrow \text{ToolCalls} \rightarrow \text{ToolResults} \rightarrow (\text{Loop} | \text{Halt})$$

公理 8: 失败重试公理

$$\forall \text{tool_call} : \text{execute}(\text{tool}) = \text{Failure} \rightarrow \text{Retry}(\text{tool})$$

公理 9: 成功延续公理

$$\forall \text{tool_call} : \text{execute}(\text{tool}) = \text{Success} \rightarrow \text{NextTool}$$

公理 10: 连续失败阻止公理

$$\text{count_failures}(\text{tool}) \geq 3 \rightarrow \text{BLOCK}(\text{tool})$$

公理 11: 输出预算约束

$$\text{total_output_tokens} \leq \text{TURN_MAX_OUTPUT_TOKENS}$$

公理 12: 重试限制

$$\text{total_context_recovery_attempts} \leq \text{MAX_CONTEXT_RECOVERY_ATTEMPTS}$$

T_1 的时间尺度是秒到分钟级。它管理的是一个完整的交互单元——从用户发出一条消息，到 LLM 响应、工具调用、结果返回、可能的下一次 LLM 调用，直到最终回复用户。每个 Turn 是一个"微观会话"——有开始,有结束,有内部状态。

关键的公理(7)定义了 Turn 的标准结构。公理(8)-(9)定义了错误处理策略——失败后的重试与成功后的继续。公理(10)-(12)是保护性约束——防止 LLM 在一个 Turn 中无限运行。

从模型论角度, T_1 的核心贡献是引入了**时间顺序公理**——工具调用的发生不再是孤立的, 而是按照"用户消息→LLM 响应→(可选) 工具→结果→继续或结束"的序列性组织。序列性公理将 T_0 中无时序的"所有工具可用"转化为 T_1 中时序化的"特定工具在当前上下文中使用"。

保守性检查：对于任意 L_0 语言中的句子 φ （例如 "read_file(path) 返回文件内容"）， $T_1 \models \varphi$ 当且仅当 $T_0 \models \varphi$ 。 T_1 没有修改 T_0 中任何工具的行为定义——它只是添加了工具调用的时序编排公理。这满足保守扩充条件。

L_2 ：会话理论 T_2

语言 L_2 ： $L_1 \cup \{\text{Session, History, ContextBudget, Compaction, Cycle, Archive}\}$ 。

公理集 T_2 ：在 T_1 上加入以下公理。

公理 13：压缩触发

$$\text{total_token_count}(\text{history}) \geq \text{PRESSURE_THRESHOLD} \rightarrow \text{trigger_compaction}()$$

公理 14：80% 阈值

$$\text{compaction_threshold} = 0.8 \times \text{context_window}$$

公理 15：压缩执行

$$\text{compaction}() = \text{rewrite}(\text{history}, \text{summary_strategy})$$

公理 16：周期归档

$$\text{total_token_count}(\text{history}) \geq \text{TURN_MAX_OUTPUT_TOKENS} \rightarrow \text{archive_cycle}()$$

公理 17：归档→移除旧公式

$$\text{archived_cycle} \rightarrow \text{deactivate}(\text{old_formula_set})$$

公理 18：分层接缝

$$\text{seam_manager}(\text{cycle}) = \{\text{seam}_1, \text{seam}_2, \dots, \text{seam}_n\}$$

公理 19：接缝强度随位置变化

$$\forall \text{seam} \in \text{seams} : \text{intensiveness}(\text{seam}) \propto \text{position}(\text{seam})$$

T_2 的时间尺度是分钟到小时级。它管理的不是单个交互单元，而是交互单元的序列——会话。会话是 Turn 的容器， T_2 的公理定义了会话的生命周期：何时压缩、何时归档、归档后如何处理旧数据。

公理(13)-(14)定义了压缩触发条件。当消息历史的总 token 数超过上下文窗口的 80%

(`PRESSURE_THRESHOLD` 在代码中等效于 $0.8 \times \text{context_window}$)，压缩被触发。这是一个纯数据的决策——不依赖 LLM 的判断，完全由运行时逻辑决定。

公理(15)定义了压缩的执行：将历史消息重写为摘要。从模型论来看，这是用更弱的公式替换原有公式——摘要的语义强度低于原文，但它保留了原文的关键推理支持关系。一个 2000 词的对话被压缩为一句话的摘要——丢失了细节，但保留了因果链。

公理(16)-(17)定义了周期归档。当历史超过单个 Turn 的预算 (`TURN_MAX_OUTPUT_TOKENS = 262144`)，整个旧周期被"封存"——重新加载时不再参与推理。从模型论来看，这是从理论 T 中移除公理——不是删除（因为存档在磁盘上），而是暂时从活跃推理闭包中移除。这等价于将 $\text{Cn}(T \cup \{\text{archived_formulas}\})$ 缩小为 $\text{Cn}(T)$ ——推理能力减少，但推理效率提升。

公理(18)-(19)定义了分层接缝——在上下文的多个中间点执行不同强度的压缩，而非仅在压缩阈值处一次性压缩。这是对 V4 模型退化曲线的工程回应——越靠后压缩强度越大。

保守性检查： T_2 在 L_1 语言上的行为没有引入新句子。 T_2 添加的全部是"如何管理 Turn 序列"的公理——压缩什么、何时归档、归档后有多少历史不可用。这些公理不改变 T_1 中定义的 Turn 结构。 $T_1 \models$ "每个 Turn 有开始和结束"； $T_2 \models$ 同样句子。保守性成立。

L_3 ：跨会话理论 T_3

语言 L_3 ： $L_2 \cup \{\text{Skill, Memory, Context_Transfer, Cross_Session_ID, Version_Migration}\}$ 。

公理集 T_3 ：在 T_2 上加入以下公理。

公理 20：技能安装 $\rightarrow$ 会话添加公理

$$\text{skill_installed}(s) \rightarrow \forall \text{session} : (\text{session}(T_3) \rightarrow \cup\{T_3, \text{axioms}(s)\})$$

公理 21：记忆持久化

$$\text{memory_persisted}(m) \rightarrow \forall \text{session} : (\text{session}(T_3) \rightarrow \cup\{T_3, m\})$$

公理 22：工作空间信任

$$\text{workspace_trusted}(w) \rightarrow \text{session}(w) \in \text{authorized_sessions}$$

公理 23：模式迁移

$$\text{schema_version_changed}(v_1, v_2) \rightarrow \text{migrate}(v_1 \rightarrow v_2)$$

T_3 的时间尺度是天到月级。它超越了单个会话的边界，管理跨会话的持久状态和知识积累。技能安装、记忆持久化、工作空间信任决策都在这一层。

公理(20)是 T_3 的最重要公理——技能安装公理。当一个技能被安装到系统中，它向当前会话和所有未来会话的 T_2 中添加一组新的公理。这不是一个局部操作——它是一个**全局公理注入**。在模型论中，这等价于理论的索引化扩张：有一个索引集 I （已安装技能列表），对于每个 $i \in I$ ， $T_2 + \text{axioms}(i)$ 是一个新增理论。会话启动时， T_2 被所有已安装技能的公理扩充。

公理(21)定义了记忆持久化——跨会话的知识传递。`memory_persisted(m)` 将一条记忆 m 存储到持久化存储中，并在后续会话启动时注入到 T_2 中。

公理(23)是模式迁移公理。当持久化数据的版本变化时，系统必须执行迁移——将旧版本的公式重写为新版本的公式。这是一个理论的重写：公式的符号集变化了，但公式在模型中的解释应保持等价。

保守性检查： T_3 对 T_2 的保守性不再是"在所有公式上成立"，而是在"不涉及新安装技能的新公理"的原 T_2 语言上成立。即：在 L_2 语言（不包括 skill 相关的符号）中， T_3 没有引入新可推导句子。但注意：技能安装的公理(20)实际上可以改变 T_2 的行为——当技能添加了新工具（ L_0 层面的扩展）时， T_2 中关于"所有可用工具"的公理被扩充。这是**语言扩展导致的非保守性**，但不是层次间泄漏——它是下层理论的自然增长途径。

L_4 ：基础设施理论 T_4

语言 L_4 ： $L_3 \cup \{\text{Config, Thread, MCP_Client, Sandbox, Network_Policy}\}$ 。

公理集 T_4 ：在 T_3 上加入以下公理。

公理 24：配置加载

$$\text{config_loaded}(c) \rightarrow \text{apply_config}(c, T_4)$$

公理 25：线程管理

$$\text{thread_spawned}(t) \rightarrow t \in \text{active_threads}$$

公理 26：MCP 传输

$$\text{mcp_client}(\text{host}, \text{port}) \rightarrow \text{transport_established}$$

公理 27：沙箱执行

$$\text{sandbox}(\text{input}) \rightarrow (\text{permitted} \rightarrow \text{execute}(\text{input}) | \text{reject})$$

T_4 的时间尺度是系统生命周期——从启动到关闭。它支撑所有上层运行，提供配置加载、线程管理、MCP 客户端管理、沙箱执行等基础服务。 T_4 中的公理定义了系统的"物理边界"——配置从哪里加载、线程如何管理、网络策略如何执行。

T_4 对 T_3 是语言扩展的保守扩充。配置加载和线程管理与上层理论（跨会话、会话、Turn、工具）没有逻辑冲突——它们定义了基础设施的可用条件，但不改变上层理论的推理闭包。

omp 的理论塔

L_0 : 原生层理论 T_0

语言 L_0 : N-API 函数名称 (grep, tokenize, read_width, ast_search, pty_session, key_parse)、输入输出类型。

论域 $|M_0|$: 文件字节流、文本宽度值、AST 节点、终端的按键事件、进程状态。

公理集 T_0 :

公理 1: 原生操作公理

$$\forall \text{native} \in N - \text{API} : \text{native}(\text{input}) \rightarrow \text{output}$$

公理 2: 高性能搜索

$$\text{grep}(\text{pattern}, \text{paths}) = \text{matched_lines}$$

公理 3: 分词

$$\text{tokenize}(\text{text}) = \text{token_count} + \text{token_list}$$

公理 4: 文本宽度

$$\text{read_width}(\text{text}) = \text{pixel_width}$$

公理 5: AST 搜索

$$\text{ast_search}(\text{pattern}, \text{code}) = \text{AST_match_nodes}$$

公理 6: PTY 会话

$$\text{pty_session}(\text{cmd}) = \text{pty_stream}$$

公理 7：按键事件解析

$$\text{keys}(\text{input}) \rightarrow \text{key_events}$$

T_0 的时间尺度是微秒到毫秒级。与 DS-TUI 不同，omp 的 L_0 不是"工具调用抽象"，而是"原生操作注入"。omp 的核心思想是：那些对响应时间敏感的、需要绕过 JS 引擎 Garbage Collector 的操作，全部下沉到 Rust N-API 层。grep.rs (51KB) 的搜索不是通过 JavaScript 正则引擎进行的——它是用 Rust 的 regex crate 在文件系统上直接操作。fs_cache.rs (25KB) 的缓存查找避免了 JS 层序列化/反序列化的开销。

T_0 在 omp 中比 DS-TUI 的 T_0 具有更窄的语义范围——它不包括"工具"的概念。grep 不是一个工具——它是一个原生操作，可以被上层组合为工具。

L_1 ：AI 层理论 T_1

语言 L_1 : $L_0 \cup \{\text{LLM_Provider}, \text{Stream}, \text{Delta}, \text{Model}, \text{Message}, \text{Completion}\}$ 。

公理集 T_1 ：在 T_0 上加入以下公理。

公理 8：流式调用公理

$$\text{streamSimple}(\text{prompt}, \text{model}) \rightarrow \text{token_stream}$$

公理 9：供应商归一化

$$\forall \text{provider} \in \text{ProviderSet} : \text{normalize}(\text{provider.output}) \rightarrow \text{uniform_stream}$$

公理 10：节流公理

$$\text{delta_throttle}(\text{window} = 50\text{ms}) : \text{merge_rapid_deltas}()$$

公理 11：模型元数据

$$\text{model_metadata}(m) \rightarrow \{\text{resolution}(m), \text{pricing}(m), \text{capabilities}(m)\}$$

T_1 的时间尺度是毫秒到秒级。这是 omp 相较于 DS-TUI 最突出的架构特征：AI 层是独立的理论层。在 DS-TUI 中，AI 调用逻辑嵌入在 Turn 循环中（ L_1 的一部分）。在 omp 中，AI 调用被提升为独立的理论 T_1 ——它可以在不修改 Agent 层的情况下独立扩展。

公理(8)定义了核心流式接口——从 prompt 到 token 流的映射。这是 T_1 中最基础的公理：它声明了"LLM 调用"这个操作的存在性。

公理(9)是供应商归一化公理。omp 支持约 30 个 LLM 供应商（OpenAI、Anthropic、Google、AWS Bedrock 等），每个供应商的 API 格式不同（某些用 SSE、某些用 WebSocket、某些用 gRPC）。归一化公理要求：不论底层供应商的协议差异，上层看到的都是一个统一的 `AssistantMessageEventStream` 类型。这是一个**理论化简公理**——它将外部的多样性压缩为内部语言 L_1 中的单一概念。

公理(10)是节流公理。50ms 的时间窗口内，多个 delta 事件被合并为一个。这不是网络延迟或计算资源的约束——它是为了确保上层（UI 层）不会因过于频繁的事件刷新而卡顿。节流是一个**跨层约束**： T_1 的公理考虑了上层渲染的能力边界。

保守性检查： T_1 对 T_0 是扩展语言的保守扩充。 T_1 没有改变 T_0 中任何原生操作的行为——"grep 返回匹配行"在 T_1 中仍然成立。 T_1 添加的仅是"如何调用 LLM""如何归一化供应商""如何节流"的新公理，这些公理涉及的语言符号（`LLM_Provider`, `Stream`, `Delta`）不在 L_0 中。

L_2 : Agent 层理论 T_2

语言 L_2 : $L_1 \cup \{\text{AgentEvent}, \text{Tool}, \text{AgentLoop}, \text{RunResult}, \text{Execute}, \text{ContextBuilder}\}$ 。

公理集 T_2 : 在 T_1 上加入以下公理。

公理 12: Agent 循环公理

$$\text{agentLoop}(\text{messages}) \rightarrow \text{agent_events_stream}$$

公理 13: 事件类型

$$\text{AgentEvent} \in \{\text{agent_start}, \text{turn_start}, \text{message_start}, \text{message_update}, \text{message_end}, \text{tool_execution_start}, \text{tool_execution_update}, \text{tool_execution_end}, \text{agent_end}\}$$

公理 14: 工具执行公理

$$\text{tool_execute}(\text{tool}, \text{input}) = \text{tool_output}$$

公理 15: 上下文构建

$$\text{context_build}(\text{messages} + \text{system_prompt}) = \text{llm_context}$$

T_2 的时间尺度是秒到分钟级。它实现了标准的 Agent 循环：构建上下文→LLM 调用→工具执行→结果追加→循环。这是 omp 的核心编排层。

公理(13)定义了 AgentEvent 类型——九个事件构成了 Agent 循环的完整状态机。每个事件是层间信息传递的载体：上层（会话层）通过监听事件做出响应，而无需关心下层 AI 层和原生层的具体实现。

从模型论角度， T_2 对 T_1 的重要扩充是引入了**执行顺序与事件流**的对应关系。 T_1 只定义了"如何调用 LLM"， T_2 定义了"如何在 LLM 调用之间插入工具执行"：

```
AgentLoop = loop {
  context ← buildContext(messages)    // T2 公理(15)
  stream  ← streamSimple(context)     // 调用 T1 公理(8)
  events  ← parseEvents(stream)       // T2 公理(13)
  for event in events {
    if event.type = tool_call {
      result ← executeTool(event.tool) // T2 公理(14)
      appendResult(result)             // 追加到消息历史
    }
  }
}
```

这个循环不是 T_1 的流式调用的简单包装——它在 T_1 的上层添加了一个新的控制结构：**工具执行的穿插与消息历史的累积**。

保守性检查： T_2 在 L_1 语言上的行为没有改变。 T_1 中的 `streamSimple` 在 T_2 中仍然返回相同的 token 流—— T_2 只是在这个流之上添加了解析、工具执行、结果循环。保守性成立。

L_3 ：会话层理论 T_3

语言 L_3 ：

$L_2 \cup \{\text{Session, Storage, Compaction, RetryFallback, Handoff, IRC, AuthGate, Approval}\}$ 。

公理集 T_3 ：在 T_2 上加入以下公理。

公理 16：会话持久化

$$\text{session_persist}(\text{messages}) \rightarrow \text{disk_storage}$$

公理 17：自动压缩触发

$$\text{compact_token_count} \geq \text{AUTO_COMPACTION_THRESHOLD} \rightarrow \text{compact}$$

公理 18：重试回退链

$$\text{execute}(\text{tool}) = \text{Failure} \times N \rightarrow \text{RetryFallbackSelector}(N)$$

公理 19：手递手

`handoff_request(agent_id, context) → spawn_subagent`

公理 20: IRC 路由

`irc_message(from, to, content) → message_queue`

公理 21: 认证门控

`auth_gate(action) → (authenticated → proceed|reject)`

公理 22: 审批门控

`approval_required(action) → pause(turn_until_approved)`

T_3 的时间尺度是分钟到小时级。它对应 `agent-session.ts` (8599 行) —— 整个 omp 中最厚重的单文件。

公理(16)定义了会话的持久化——消息历史在磁盘上的存储格式和策略。每次消息添加都会触发持久化写入（通过 `session-storage.ts` 的 `session_state_channel`），确保崩溃后可以恢复。

公理(17)定义了压缩触发——当消息总 token 数超过阈值时，自动压缩被触发。与 DS-TUI 类似，这是一个纯数据的决策。但 omp 的压缩触发是通过事件 (`auto_compaction_start`) 通知上层（UI 层）的——上层可以在压缩开始前暂停用户输入、显示进度条。

公理(18)定义了重试回退链 (`RetryFallbackSelector`)。当工具执行连续失败 N 次时，重试策略不再是简单的"再试一次"——而是按预设的回退链选择替代策略。例如：`try_primary → try_secondary → try_simplified → try_minimal → report_failure`。这是一个**有限状态机公理**——它定义了失败状态下的状态转换规则。

公理(19)定义了手递手机制——将上下文从一个 Agent 转移到另一个。这是 T_3 中最复杂的公理之一，因为它涉及跨 Agent 的"会话连续性"——子 Agent 不知道父 Agent 的完整历史，但需要以正确的上下文继续执行。

关于 **handoff.rs** (793 字节在 DS-TUI 中 vs `agent-session.ts` 中的完整实现)。这种差异本身就是理论塔结构的证据：DS-TUI 将手递手实现为独立文件 (793 字节)，因为它只是一个轻量的上下文传递操作——属于 T_2 层。而 omp 将手递手嵌入到 `agent-session.ts` 中，因为它需要与持久化、压缩、重试、审批等 T_3 机制交互——手递手在 omp 中是 T_3 层概念。

保守性检查： T_3 对 T_2 的保守性取决于事件传递是否改变事件含义。 T_2 产生了 `AgentEvent` (`tool_execution_start` 等)， T_3 监听了这些事件并在其上执行持久化。 T_3 没有改变 T_2 的事件产生逻辑——它只是订阅了事件流。在 L_2 语言中， T_3 没有引入新的可推导句子。保守性成立。

L_4 : UI 层理论 T_4

语言 L_4 : $L_3 \cup \{\text{TUI}, \text{Diff}, \text{Render}, \text{Focus}, \text{Input}, \text{InteractiveMode}\}$ 。

公理集 T_4 : 在 T_3 上加入以下公理。

公理 23: 差分渲染

$$\text{diff_render}(\text{old_state}, \text{new_state}) = \text{render_diff}$$

公理 24: 焦点路由

$$\text{focus_route}(\text{input}) = \text{target_component}$$

公理 25: 交互模式

$$\text{interactive_mode}(\text{session}) \rightarrow \text{user_input_stream}$$

公理 26: 按键到动作的映射

$$\text{key_event}(\text{key}) \rightarrow \text{action}(\text{map}(\text{key}))$$

T_4 的时间尺度是用户交互的实时响应——毫秒级。它是理论塔的顶层（最"外部"的层），直接面对用户。

公理(23)定义了差分渲染。TUI 引擎维护一个状态树，每次更新时计算新旧状态的差异，只渲染变化部分。`tui.ts:1397` 行的差分引擎确保一个 1000 行的终端界面在单行更新时只发送一行内容的 ANSI 转义码。这是一个**计算优化**公理——它不改变语义（新旧状态在语义上等价），只改变表示成本。

公理(24)定义了焦点路由。当用户输入一个键盘事件时，焦点路由决定哪个组件处理该事件：输入框获得文本输入、消息列表获得滚动事件、工具调用卡片获得展开/折叠事件。这个公理确保了"一个事件只有一个消费者"的语义——避免了事件争用和不确定的行为。

公理(25)定义了交互模式——`interactive-mode.ts`（2775 行）的抽象。交互模式是 T_4 的核心控制结构：它将底层的 Agent 循环流式事件转换为终端界面上的实时渲染。当 Agent 层的 `tool_execution_start` 事件到达时，交互模式创建一个工具调用卡片；当 `message_update` 到达时，逐字增量渲染。

T_4 对 T_3 的保守性是显而易见的——UI 层的渲染决策不会改变会话层的持久化逻辑或 Agent 层的工具执行逻辑。但要注意的是： T_4 中的"用户中断"事件（用户按 Ctrl+C 中断当前的 Agent 执行）在 T_3 中对应"终止当前会话"。这是一个**跨层控制**——不是通过公理推导，而是通过事件总线的直接通信。这种非公理化的层间交互是"理论塔"模型需要补充的边界——经典模型论中没有"中断"的概念。

层次收敛的模型论解释

理论塔的结构同构

DS-TUI 和 omp 独立演化出的五层结构在模型论框架下同构。使用"理论塔"的记法：

层次 / 时间尺度	DS-TUI	omp
L_0 (微秒)	$T_0 = \{\text{工具执行}\}$	$T_0 = \{\text{原生操作}\}$
L_1 (秒)	$T_1 = T_0 \cup \{\text{Turn编排}\}$	$T_1 = T_0 \cup \{\text{AI流式调用}\}$
L_2 (分钟)	$T_2 = T_1 \cup \{\text{会话管理}\}$	$T_2 = T_1 \cup \{\text{Agent循环}\}$
L_3 (小时)	$T_3 = T_2 \cup \{\text{跨会话状态}\}$	$T_3 = T_2 \cup \{\text{会话持久化}\}$
L_4 (系统)	$T_4 = T_3 \cup \{\text{基础设施}\}$	$T_4 = T_3 \cup \{\text{UI渲染}\}$

两张塔的结构同态映射为：

最底层：可执行操作

$$f(\text{DS-TUI}.L_0) = \text{omp}.L_0$$

LLM 交互编排

$$f(\text{DS-TUI}.L_1) = \text{omp}.L_1 + \text{omp}.L_2$$

会话生命周期

$$f(\text{DS-TUI}.L_2) = \text{omp}.L_3$$

(子集) 跨会话知识

$$f(\text{DS-TUI}.L_3) = \text{omp}.L_3$$

系统边界

$$f(\text{DS-TUI}.L_4) = \text{omp}.L_4$$

注意映射不是逐层一一对应的——DS-TUI 将 LLM 交互逻辑内聚在单层 (L_1 : turn_loop.rs)，而 omp 将其拆分为两层 (L_1 : AI 层 + L_2 : Agent 层)。但原则相同：至少有一个层专门承载 LLM 交互的公理集。

时间尺度：层次收敛的驱动力

为什么两个独立项目演化出了几乎相同的层次？答案在时间尺度中。

给定一个系统，从其操作的**特征时间常数** τ 出发。系统中有多个操作，每个操作有自己的 τ ：

操作	τ	对上层的影响
文件读取	~1ms	瞬态——上层"看到"结果而非过程
AI 推理	~1s	响应——上层"等待"完成或流式接收
会话管理	~60s	持续——上层"监督"趋势而非瞬时事件
记忆积累	~1day	缓慢——上层"适应"而非直接控制
系统启动	~1min	离散——上层"依赖"而非调度

层次分离的判据：当两个操作的时间尺度相差超过 10 倍时，它们必须分属不同层次。如果文件读取

（1ms）和 AI 推理（1s）放在同一层，则文件读取会被 AI 推理的等待阻塞——文件系统可能在 AI 推理完成前已经返回，但线程被 AI 调用占用，无法消费文件结果。如果会话管理（60s）和 AI 推理（1s）放在同一层，则会话层的压缩决策（每 60s 检查一次）与 AI 推理循环（每 1s 迭代一次）之间存在 60 倍的时间差——要么压缩检查过于频繁（浪费 CPU），要么 AI 推理在压缩期间停滞（浪费令牌）。

自然选择过程（重构迭代）消去了那些层数不够的中间态。回到最初——两个系统都是从单文件开始的。DS-TUI 最初在一个 main.rs 中实现了"读取用户输入→调用 AI→显示响应→等待输入"的单循环。随着工具数量从 3 个增长到 30+ 个，会话从单轮变为多轮，存储从内存改为持久化——每次重构都新发现一个"时间边界"，并在该边界处创建新层。这不是设计——这是涌现。

理论塔的普适性定理

我们可以将上述观察提升为系统论的一个形式断言：**给定任意要求与外部世界交互并具备内部状态的编码智能体系统 S ， S 在持续演化过程中必然稳定收敛到包含至少四个具有不同时间尺度的层次结构。** 证明思路：

1. S 必须与操作系统交互（文件、进程、网络）——这是微秒到毫秒级的操作，对应 L_0 。
2. S 必须与 LLM 交互——这是秒级操作，对应 L_1 。
3. S 必须管理消息历史——这是分钟到小时级的操作，对应 L_2 。
4. S 必须持久化跨会话状态——这是天级的操作，对应 L_3 。
5. 前四层需要一个物理容器——操作系统服务和配置，对应 L_4 。

这不是偶然。它是一个**维度定理**：编码智能体系统必须同时管理的五个时间维度（文件 IO、AI 推理、会话状态、记忆积累、系统配置）提供了层次分离的物理基础。如果某个维度被忽略（例如：系统不持久化跨会话记忆），则对应层次可以收缩或消失。但任何"完整"的 Harness 系统都必然包含至少四层——这是与 LLM 交互的固有性质。

层次过载问题

理论塔给出了上层是下层的保守扩充这一理想模型。但工程现实是：理论塔的各层并非严格保守——**层次过载**是常见的工程退化。

模式一：单层承接过多的时间尺度（层内非保守）

DS-TUI 的 `turn_loop.rs` (95KB) 是层次过载的典型示例。按照理论塔的分配，`turn_loop.rs` 应承载 T_1 (Turn 编排) 的公理。但真实的代码包含了跨越 L_1 - L_3 的职责：

- L_1 职责：Turn 循环、流式事件路由、工具调用编排。这是预期的。
- L_2 职责：工具目录的延迟加载策略（应与 `tool_catalog.rs` 配合的 L_2 级逻辑）。`turn_loop.rs` 直接管理哪些工具在哪些上下文中可用——这是会话级别的状态管理。
- L_2 职责：MCP 工具水合 (hydration)。MCP (Model Context Protocol) 管理属于 L_2 的跨 Turn 状态初始化——但它被嵌入了 Turn 循环中。
- L_3 职责：子代理完成处理。子代理的完成涉及跨会话状态（子代理有自己的会话上下文），这是 L_3 职责——但处理逻辑在 `turn_loop.rs` 中。

从模型论角度，这对应**理论的非保守性**。当 `turn_loop.rs` 同时管理 Turn 编排和工具目录延迟加载时， T_1 和 T_2 的语言 L_1 和 L_2 之间的边界被模糊。具体表现为：

- T_2 中的工具延迟加载逻辑使用了 L_1 的语言 (Turn 事件流) 来描述 L_2 的概念 (工具可用性策略)
- 反之， T_1 中的 Turn 循环使用了 L_2 的语言 (工具目录状态) 来决定 L_1 的行为 (当前 Turn 可以执行哪些工具)

这种语言混淆意味着某些 L_2 语言中的句子 φ 可以在 T_1 中推导——例如，"在当前 Turn 中，工具 X 不可用来处理调用请求 Y "。在保守的理论塔中，这个句子应该只在 T_2 中可推导——因为它涉及"工具可用性策略"（一个 L_2 概念）。但在 `turn_loop.rs` 的非保守实现中， T_1 就可以推导它——因为 T_1 中包含了原本应该属于 T_2 的公理。

这种非保守性的工程后果是：

1. **测试困难**： T_1 的单元测试无法建立，因为 T_1 依赖于 T_2 层状态。要测试一个 Turn 循环的简单行为，需要模拟工具目录状态——这是 T_2 层的设置。
2. **重构风险**：修改工具目录延迟加载策略（一个 "按理应该影响 L_2 的行为" 的变更）必须修改 `turn_loop.rs`。由于 T_1 在 95KB 的代码空间中与 L_2 状态交织，改动的影响范围难以评估。
3. **性能不确定性**： T_1 的每次执行都隐含了 T_2 层操作（检查工具目录状态）。导致 Turn 的时间消耗不稳定——有时快（工具已缓存）、有时慢（需要延迟加载）。 T_1 的时间常数不再是一个确定的量。

DS-TUI 的开发者选择这种设计有其合理性：性能。将所有 Turn 级逻辑集中在一个编译单元中（95KB 的 Rust 文件）便于 LLVM 的内联优化。这是**性能优先于清晰度**的设计选择。但从理论塔的角度看，这是一个可以辨识的"架构债务"——它等价于将理论 T_1 非保守地扩展到了 T_2 的领域。

模式二：单层理论过大

omp 的 `agent-session.ts` (8599 行) 是另一种过载模式——不是跨层次混淆，而是**单层理论承载了过多的内部公理**。`agent-session.ts` 对应 T_3 （会话层），但它在公理数量上远超其他各层之和。

T_3 的"内部公理数量"可以大致通过导出的方法和类型数量估算（保守估计）：

T_3 内部子理论：

- 持久化子理论：~20 个公理（读写、快照、恢复、崩溃保护）
- 压缩子理论：~15 个公理（触发、执行、策略选择、结果验证）
- 重试子理论：~10 个公理（重试链定义、状态迁移、超时处理）
- 手递手子理论：~25 个公理（上下文传递、生命周期、子代理创建/销毁）
- IRC 子理论：~10 个公理（消息路由、发送、接收、去重）
- 认证子理论：~8 个公理（身份验证、权限检查、会话绑定）
- 审批子理论：~6 个公理（请求创建、等待审批、审批后执行）

总计：~94 个公理（估算）

相比之下， T_2 （Agent 层）的核心公理约 15 个——`agentLoop` 定义、事件类型、工具执行。 T_1 （AI 层）约 10 个——流式调用、供应商归一化、节流。 T_4 （UI 层）约 8 个——差分渲染、焦点路由、交互模式。

T_3 是其他各层公理数量总和的 2-3 倍。这不是单纯的文件大小问题——8599 行代码可以因为包含大量短小的辅助函数而较长。核心问题是： T_3 涵盖了过多逻辑上独立的子理论。

从模型论分析， T_3 中的子理论之间缺乏**正交性**：

- 持久化子理论与压缩子理论交互：压缩触发需要检查持久化记录中的 token 计数，压缩结果需要持久化。
- 重试子理论与手递手子理论交互：重试失败后可能需要将上下文传递给子代理——这是 T_3 内部的两种控制流交织。
- IRC 子理论与认证子理论交互：IRC 消息的路由需要认证信息（发送者身份、接收者权限）。
- 审批子理论与持久化子理论交互：审批状态需要持久化——会话恢复时必须恢复审批状态。

这些交互本身是合理的—— T_3 中的子理论本来就应该相互配合。但问题在于，当所有交互都在同一个 8599 行的文件中实现时，子理论之间的关系不再是公理化的“逻辑推导”关系，而是程序化的“紧耦合”关系。紧耦合意味着：

1. 修改一个子理论可能影响其他所有子理论——因为共享可变状态（`session.state` 在 `agent-session.ts` 中是全局的）。
2. 子理论无法独立测试——测试重试逻辑需要设置持久化状态和 IRC 路由。
3. 新功能添加困难——一个新的 T_3 子理论（例如：工具调用模板缓存）必须嵌入到现有的大文件中，因为所有 T_3 状态都集中在这里。

这种过载是难以避免的阶段性特征。当一个系统从单文件（如 `omp` 早期版本）演化到多层结构时，中间态总是某些层被过度加载。代理人在每次重构中“发现”一个新层，将一个子理论从过载的父层分离出去。`agent-session.ts` 可能在下一次重构中分裂为 `persistence.ts`、`compaction.ts`、`retry-strategy.ts`、`handoff.ts`、`irc-router.ts`、`auth-gate.ts`、`approval-flow.ts`——每个文件对应一个独立的子理论。

过载的修复：回到保守扩充

层次过载的修复遵循模型论的一条原则：**恢复保守性**。对于 DS-TUI 的 `turn_loop.rs`（模式一），修复策略是将 L_2 和 L_3 的职责提取到独立的层中：

`turn_loop.rs`（当前）： $T_1 + T_2$ （子集） + T_3 （子集）

→ `turn_loop.rs`（修复后）： T_1 （纯 Turn 编排）

`tool_catalog.rs`（独立）： T_2 （工具目录管理）

`completion_handler.rs`（独立）： T_3 （子代理完成处理）

这种提取恢复了理论塔的保守性： T_1 不再依赖 T_2 或 T_3 的语言符号。Turn 循环的语言 L_1 中不再包含“工具目录策略”或“子代理生命周期”的概念。

对于 `omp` 的 `agent-session.ts`（模式二），修复策略是将具有独立“理论身份”的子理论分解为单独文件：

`agent-session.ts` (当前): 所有 T_3 子理论的并集

→ `session-persistence.ts`: $T_{3,persist}$ (持久化子理论)

`session-compaction.ts`: $T_{3,compact}$ (压缩子理论)

`session-retry.ts`: $T_{3,retry}$ (重试子理论)

`session-handoff.ts`: $T_{3,handoff}$ (手递手子理论)

`session-irc.ts`: $T_{3,irc}$ (IRC 子理论)

`session-auth.ts`: $T_{3,auth}$ (认证子理论)

`session-approval.ts`: $T_{3,approval}$ (审批子理论)

`session-orchestrator.ts`: $T_{3,orch}$ (协调器——负责组合所有子理论)

每个子理论在分裂后形成一个独立的**微型理论塔**——它们共享 L_3 语言的符号集 (`Session`, `Message`, `AgentEvent`)，但各自的公理集是正交的。协调器 `session-orchestrator.ts` 是 L_3 语言之上的"元层"——它组合子理论，但不包含子理论的实现细节。

这里的关键是**分裂后的保守性**：每个子理论在自己的语言范围内是完整的、可独立测试的——它们之间的交互通过协调器显式编排，而不是通过共享可变状态隐式耦合。协调器本身可能包含一些"调度公理"（先压缩再存档、重试失败后发送 IRC 通知等），但这些公理是 L_3 语言的"组合公理"——它们定义了子理论的调用顺序，而不是子理论内部的实现。

过载的必然性

需指出：层次过载是任何处于演化中的系统都可能出现的现象。它不是设计失误——它是自然的、不可避免的。当一个新能力被添加到系统中时，开发者倾向于将其附着在最近似的层上，而不是创建一个专门的新层。只有在重复添加导致单层膨胀到难以理解时，"提纯提层"的重构才会发生。

分层重构的触发条件通常是**认知成本**超过某个阈值：

$$\text{CognitionCost}(\text{layer}) = \text{AxiomCount}(\text{layer}) \times \text{InterAxiomCoupling}(\text{layer}) \times \text{FileSize}(\text{layer})$$

当认知成本显著超过其他层时，开发者会不自觉地感受到"这个文件太乱了"——这是重构的信号。DS-TUI 将 `turn_loop.rs` 保持为 95KB 而非进一步分裂，说明它的认知成本仍在可接受范围内（或性能收益超过了分解收益）。omp 的 `agent-session.ts` 从 3000 行增长到 8599 行尚未被分裂，说明累积的复杂性还未突破团队的认知阈值。

但从理论塔的角度看，这种等待是低效的。理论塔提供了一条形式判据：**如果层 k 的语言中包含本应属于层 $k+1$ 的符号，则层 k 已经过载。**按此判据：

- `turn_loop.rs` 包含"工具目录延迟加载"——这是一个 L_2 符号 ("工具管理策略")，不在 L_1 中 $\rightarrow$ 过载成立。
- `agent-session.ts` 同时管理持久化、压缩、重试、手递手、IRC、认证、审批——这些应各自为独立的子理论 $\rightarrow$ 过载成立。

理论塔不仅诊断过载的存在性，还指导过载的修复方向——**恢复保守性**。

小结

层次性在系统论中是一个宏观原则——复杂系统必定分层。在模型论中，它获得了精确的形式：嵌套理论塔 $T_0 \subset T_1 \subset T_2 \subset \dots$ ，每层是下层的保守扩充。保守性条件具有直接的工程意义——它确保下层事实在上层推理中保持不变，层间职责边界清晰。

DS-TUI 和 omp 独立演化出的理论塔从两个方向验证了层次收敛定理：

- **时间尺度分离**是分层的第一驱动力。微秒（文件 IO）、秒（AI 推理）、分钟（会话管理）、天（记忆积累）、系统生命周期（配置）——五个量级的时间差异迫使系统天然地裂变为五层。
- **层间接口的显式化**是层次性的第二标志。DS-TUI 通过 Rust 类型系统 (ToolSpec trait)、omp 通过 npm 包导出 (`pi-ai` $\rightarrow$ `pi-agent-core` $\rightarrow$ `pi-coding-agent`) ——两种方式都实现了语言的显式边界。

层次过载是理论塔的逆向模式。当单层包含本应属于不同层的公理（如 `turn_loop.rs` 的 L_1+L_2 混合）或单层承载了过多的内部子理论（如 `agent-session.ts` 的多个 T_3 子理论）时，理论的保守性被破坏。修复方向统一为：**提纯子理论并恢复层间接口的形式化边界**。

理论塔的核心贡献不在于预测"系统应该有五层"——那是一个偶然的事实而非必然的结论。真正的贡献是：它提供了一个精确定位**工程策略理论层次**的工具。当开发者说"这个功能应该放在哪一层"时，他们在模型论意义上问的是："这个公理属于 T_k 还是 T_{k+1} ？"——一个可以回答的问题。

CHAPTER 05

第五章 开放性：语言扩展

系统与环境之间的信息交换决定了系统的适应能力。在模型论语境中，开放性表现为对语言 L 的运行扩展——添加新的函数符号、关系符号、常元符号，从而扩展理论的表达能力和结构的操作空间。本章从模型论和系统论的双重视角，比较 DS-TUI 与 omp 的语言扩展策略，揭示"受控开放性"与"激进开放性"的根本性差异。

系统论开放性原理 + 模型论重述

钱学森在开放的复杂巨系统理论中指出，封闭系统与开放系统的本质区别在于：开放系统与环境之间存在持续的物质、能量或信息交换，其行为不能仅通过内部状态预测。在 Coding Agent harness 的语境中，"开放"意味着系统可以在运行时接入新的能力——新的工具、新的知识、新的交互模式。

在模型论语境中，开放性对应一个精确的形式化操作：**语言 L 的扩展**。

语言扩展的三种基本形式

给定语言 $L = (C, F, R)$ ，其中 C 是常元符号集、 F 是函数符号集（每符号附有无数量数 n ）、 R 是关系符号集（每符号附有无数量数 n ）。向 L 中添加新符号产生一个扩展语言 L' ：

添加常元符号 c_{new} ： $C' = C \cup \{c_{\text{new}}\}$ 。新常元对应于一个可在运行时确定的固定值——例如当前工作目录路径、会话 ID、用户名称。常元符号在理论 T 中可以被公理约束：`equals(workspace_path, "/workspace/oh-my-pi")` 在结构 M 中是一个原子公式，其真值取决于运行时环境。

添加函数符号 f_{new} ： $F' = F \cup \{f_{\text{new}}\}$ ，其中 f_{new} 附有无数量数 $n = k$ 。新函数符号对应于一个在运行时执行的计算——例如 `read(path: string) → string`、`search(query: string) → string[]`。函数符号是 harness 语言扩展的最常见形式，因为工具本质上就是 n 元函数从论域的 k 次幂到论域的映射。

添加关系符号 R_{new} ： $R' = R \cup \{R_{\text{new}}\}$ ，其中 R_{new} 附有无数量数 n 。新关系符号对应于一个约束条件——例如 `path_is_safe(p)`、`tool_available(t)`。关系符号定义结构 M 中元素之间的定性关系，典型的用途是权限规则和验证约束。

理论扩展的三种层次

语言的扩展必然带来理论的扩展。给定原始理论 T （语言 L 中的句子集），向 T 中添加新公理形成扩展理论 T' ：

保守扩展（conservative extension）： T' 是 T 的保守扩展，当且仅当 T' 中所有用原始语言 L 可表达的句子都在 T 中可证。换句话说，扩展引入了新符号和新公理，但不改变原始语言 L 中任何句子的可证性。这是最安全的扩展方式——它不破坏 T 的逻辑一致性。

定义扩展（definitional extension）：一种特殊的保守扩展，其中新符号通过显式定义引入。例如，引进一个新函数符号 `list_files` 并用 `read("ls")` 来定义它。定义扩展总是保守的，因为新符号可以被显式消去——将每次出现替换为其定义。

实质扩展（proper extension）： T' 不是 T 的保守扩展——存在一个 L -句子 φ 使得 $T \not\models \varphi$ 但 $T' \models \varphi$ 。实质扩展改变了原始理论的能力——新公理蕴含了原始语言中不可推导的结论。在 harness 工程中，添加一个新的系统级约束（如“所有文件操作必须记录日志”）可能构成实质扩展。

满足关系在扩展下的行为

语言扩展 $L \rightarrow L'$ 和理论扩展 $T \rightarrow T'$ 对满足关系 $M \models \varphi$ 的影响是微妙的。给定一个原始 L -结构 M 和一个扩展 L' -结构 M' （ M' 限制到语言 L 时与 M 一致）：

- 对于 $\varphi \in L$ （原始语言句子）： $M' \models \varphi$ 当且仅当 $M \models \varphi$ 。因为扩展结构的 L -片段解释不变。
- 对于 $\psi \in L' \setminus L$ （新语言句子）： $M' \models \psi$ 的真值取决于新符号在 M' 中的解释。

这意味着：语言扩展不破坏已建立的事实，但引入了新的事实空间。这是 harness 工程中“向后兼容”的形式化表述。

DS-TUI 的受控语言扩展

DS-TUI 采用**保守的语言扩展策略**：新符号必须适配现有签名 Σ ，通过预定义的接口类型和编译时检查来约束扩展范围。这与其“性能优先”的目标函数一致——扩展越受控，系统的可优化路径越清晰。

MCP：运行时函数符号注入

MCP（Model Context Protocol）是 DS-TUI 中最主要的语言扩展机制。MCP 服务器在运行时通过 `tools/list` 发现其可用工具，然后通过 `McpToolAdapter`（`registry.rs:898-899`）适配为内部 `ToolSpec trait`。

在模型论中，这一过程对应于：

DS-TUI 的基础语言

$$L = (C, F_{\text{DS-TUI}}, R)$$

加入 MCP 工具后的扩展语言

$$L' = (C, F_{\text{DS-TUI}} \cup F_{\text{mcp}}, R)$$

$$T' = T \cup \{\text{公理} : \text{MCP 工具使用规则}\}$$

其中 $F_{\text{mcp}} = \{f_{\text{mcp_tool},1}, f_{\text{mcp_tool},2}, \dots, f_{\text{mcp_tool},n}\}$ 是自动发现的 MCP 函数符号集合。每个符号的签名（名称、参数类型、返回类型）通过工具发现协议从 MCP 服务器获取。

关键在于，DS-TUI 的 MCP 扩展是**签名的适配而非定义的扩展**。ToolSpec trait 提供了统一的接口：

```
// crates/tui/src/tools/spec.rs
trait ToolSpec {
    fn spec() -> ToolDefinition;           // 签名声明
    async fn execute(args: Value) -> Result<Value>; // 解释函数
}
```

ToolDefinition 是语言 L 中的签名声明——它声明了函数符号的名称、参数 schema 和行为描述。
execute 是结构 M 中的解释实现。新符号 $f \in F_{\text{mcp}}$ 被包装为 ToolSpec 后，在语言 L 中完全等同于内置函数符号。这个适配策略在模型论中对应**定义扩展**：每个新符号 f 通过其 execute 方法被显式定义。

DS-TUI 的 MCP 集成包含四种传输层协议——StdioTransport、SseTransport、HttpTransport、StreamableHttpTransport——每种实现 McpTransport trait (mcp.rs:379–388)。这是一个分层的**语言实现**：传输层是底层的通信语言（字节流），MCP 协议层是上层的工具语言（抽象符号），McpToolAdapter 是连接两层的解释函数。这种分层确保了传输层对语言 L 的不可见性——LLM 看到的只有函数符号 $f \in F_{\text{mcp}}$ ，不需要理解底层的 SSE 或 HTTP 协议。

Hooks：事件响应而非语言扩展

DS-TUI 的 Hooks 系统 (hooks.rs:25–49) 提供 6 种生命周期事件的 Shell 进程挂钩：session_start、session_end、tool_call（前后）、mode_change、message_submitted、error。每个 Hook 接收环境变量形式的上下文 (HookContext, hooks.rs:224–254)，执行一个独立的 Shell 进程。

从模型论视角，Hooks 不是语言扩展——它不向 L 中添加新的函数符号或关系符号。Hooks 是一种**观测机制**（observation mechanism）：系统在特定时刻发送观测信号（事件通知），但观测者的行为不在语言 L 的描述范围内。

更精确地说，Hooks 对应模型论中的**外延结构**（external structure）：Hook 的运行环境是一个独立的进程空间，其输出（stdout、文件系统变更、网络请求）是系统自发的环境副作用，而非被 T 所约束的结构 M 中的元素。Hook 不参与满足关系 $M \models T$ ——它既不是 T 中的公式，也不是 M 中的解释。

这种设计选择在系统论中体现了**受控开放**：系统为外部脚本开放了有限的事件钩子，但 Hook 的执行是完全无状态的（每次调用独立的 Shell 进程），不维护跨 Hook 的累积状态，不能发起到系统的主动通信。从信息论角度看，这是一个信息的**单向流**：系统→Hook（事件通知），Hook→系统（副作用），但没有 Hook→系统（有结构的反馈）。后一种双向反馈机制在 omp 中才得以实现。

Skills：公理注入而非符号添加

DS-TUI 的 Skills 系统（skills/mod.rs:48KB）安装和管理技能包。技能不是可执行代码，而是 Markdown 格式的系统提示词扩展——它们向 LLM 注入专业知识片段。

在模型论中，Skills 对应的是**公理注入**而非语言扩展：

T_{base} = 基础理论（base.md，约 264 行公理）

$$T_{\text{skill}} = T_{\text{base}} \cup \{\varphi_1, \varphi_2, \dots, \varphi_n\}$$

其中每个 φ_i 是**理论 T 中的一个新句子**，而非语言 L 中的一个新符号。Skills 不改变签名 Σ ，不引入新函数符号——它们只添加新的公理来约束已有符号的行为。例如，一个"Python 最佳实践"技能可能添加公理：

$$\forall p : \text{is_python_file}(p) \rightarrow \text{use_ruff_for_formatting}(p)$$

$$\forall i : \text{is_import_statement}(i) \rightarrow \text{group_imports}(i)$$

这些公理只引用语言 L 中已经存在的函数符号（use_ruff_for_formatting、group_imports、is_python_file），但为这些符号的使用模式施加了更严格的约束。

这与模型论中的**公理化扩充**（axiomatic extension）概念对应：理论 T 被扩展为 $T' = T \cup A$ ，其中 A 是新公理集，但语言 L 没有变化。公理化扩充比定义扩展更弱——它不增加新符号，只增加对已有符号的约束。

Skills 的设计选择体现了 DS-TUI 的核心工程权衡：**知识可以独立于代码演进，但不允许代码逻辑因知识扩展而被绕过**。系统提示词可以随 Skills 而改变，但可执行代码不可以。这保证了系统的安全性（恶意技能无法注入可执行代码），但也限制了技能的表达能力。

前缀缓存对语言扩展的约束

DS-TUI 语言扩展面临的最强约束来自**前缀缓存**。DeepSeek V4 的前缀缓存以 128-token 粒度工作，提供约 90% 的缓存命中折扣。语言 L 的每次扩展（添加 MCP 工具、安装新 Skills）都改变了提示词的前缀字节，导致缓存穿透。

在模型论中，前缀缓存是对**理论 T 的物理实现层约束**。理论 T 中的任何变化——添加一个句子 φ 、引入一个新符号 f ——在物理实现层表现为提示词字符串的变化。字符串的变化导致 KV 缓存（一个存储了前缀键值对的优化结构）完全失效。

这产生了 DS-TUI 中的 `OnceLock` 缓存模式。工具目录（`tool_catalog.rs`）和 MCP 配置在会话启动后被冻结——语言 L 的签名 Σ 在整个会话生命周期内不可变。在模型论中，这对应一个**冻结的语言层**：

会话期间不可扩展

$$L_{\text{session}} = L_{\text{frozen}}$$

语言扩展只能在会话边界处发生（切换工作空间、重新加载配置）。这种设计将语言修改的代价从运行时推迟到会话初始化阶段，与 DS-TUI "一旦前缀缓存建立，就不要打破它"的核心目标完全一致。

omp 的激进语言扩展

Oh My Pi 采用**激进的扩展策略**：任何新符号只要实现 `AgentTool` 接口即可加入签名 Σ ，语言 L 可以在运行时不间断地扩展。这与其"最大化供应商多样性"的目标函数一致——系统对单一工具、单一提供方的依赖越低，适应性越强。

MCP：动态发现与 Smithery 生态

omp 的 MCP 集成（`packages/coding-agent/src/mcp/`）在 DS-TUI 的 MCP 支持上增加了两层开放性：

第一层：动态工具发现。Smithery 注册表（`smithery-connect.ts`、`smithery-registry.ts`:14KB）提供了从注册表自动发现和安装 MCP 服务器的能力。这与 DS-TUI 的静态 MCP 配置（手动编辑 `mcp.json`）有本质区别。在模型论中，Smithery 对应一个**语言发现协议**：系统不仅可以添加新符号，还可以通过查询注册表得知有哪些符号可能被添加。注册表可以形式化为一个元级结构：

$$\Omega = \{(f_1, \text{desc}_1, \text{schema}_1), (f_2, \text{desc}_2, \text{schema}_2), \dots, (f_n, \text{desc}_n, \text{schema}_n)\}$$

其中 Ω 是"可发现工具宇宙"，每个元素描述一个潜在的函数符号。系统从 Ω 中选择激活 $\Sigma_{\text{actual}} \subseteq \Omega$ 子集作为当前语言 L 的函数符号集。

第二层：可发现工具元数据。 `discoverable-tool-metadata.ts` 允许工具被标记为"可发现但默认隐藏"。这与 DS-TUI 的"激活即用"不同——工具可以在不被注入语言 L 的情况下被注册。LLM 可以通过 `search_tool_bm25` 工具主动搜索并选择性地激活它们。在模型论中，这对应**惰性加载的签名**：

$$\Sigma_{\text{hidden}} = \{f \in \Omega : f.\text{discoverable} \wedge f \notin \Sigma_{\text{active}}\}$$

Σ_{hidden} 中的符号存在于语言 L 的可能扩展空间中，但不消耗当前理论的注意力预算（不占用上下文窗口）。只有当 LLM 显式查询某个 hidden 工具时，它才被提升到 Σ_{active} 中。这种设计使得 omp 可以应对数百个 MCP 工具的空间而不致上下文膨胀。

Extensions：元级语言扩展

omp 的 Extensions 系统是它超越 DS-TUI 的最显著特征。Extensions 是完整的 TypeScript 模块，通过 `ExtensionFactory` 工厂函数加载。与 DS-TUI 的 Skills（公理注入）不同，Extensions 可以**修改系统本身**——修改提示词、注册新工具、变更模型注册表。

在模型论中，Extensions 对应**元级语言扩展**（meta-level language extension）。要理解这个概念，需要区分"对象语言 L "和"元语言 M "：

- 对象语言 L ：LLM 推理时使用的语言，包含函数符号（工具）、关系符号（规则）、常元符号（配置）。
- 元语言 M ：harness 代码本身使用的语言，它的论域是语言 L 的组成要素——元语言中的"常元"可以是一个对象语言中的函数符号。

Extensions 处于元语言 M 的层次。一个 Extension 的函数签名近似于：

```
// Extension 是元语言 M 中的操作
type Extension = (harness: HarnessAPI) => void;

// 它可以：
harness.registerTool(new MyTool()); // 向对象语言 L 添加新函数符号 f
harness.modifyPrompt("...");        // 向理论 T 添加新公理 φ
harness.wrapTool("read", wrapper);   // 修改已有函数符号的解释
```

`sdk.ts:2167` 中的 `createAgent()` 函数接受一个 `Extension[]` 数组——扩展在系统初始化时作为一等公民传入，而非作为外部附属品。这意味着语言 L 的构成不再是编译时固定或配置驱动的，而是由**代码驱动的运行时构造**。

`wrapRegisteredTools()` 函数（`extensions/` 目录）允许 `Extensions` 拦截和修改已有工具的行为。在模型论中，这对应**重定义函数符号的解释**：给定一个原始解释 $I(f)$ 和包装函数 w ，新解释变为 $I'(f) = w \circ I(f)$ 。这是比添加新符号更强的操作——它不仅扩展了语言 L ，还改变了已有符号在结构 M 中的行为。

Custom Tools：用户定义的新函数符号

`custom-tools/` 目录允许用户通过声明式配置文件注册自定义工具。这比 `Extensions` 更靠近对象语言层——用户不需要编写 TypeScript 代码来扩展 L 。

在模型论中，Custom Tool 对应用户定义的新函数符号：

```
// 用户在配置中声明：
{
  name: "my_custom_tool",
  description: "执行自定义任务",
  schema: { type: "object", properties: { "param": { type: "string" } } },
  execute: async (args) => { /* 用户提供的脚本 */ }
}
```

这个过程的形式化本质是：用户在运行时向签名 Σ 添加了一个新的函数符号 f_{custom} ，并为其提供了一个在结构 M 中的解释函数。这在 DS-TUI 中需要编写完整的 Rust 工具实现和重新编译，在 omp 中只需一个配置文件。

Custom Tools 的运行时验证远比 DS-TUI 松散。DS-TUI 在编译时验证 `ToolSpec` trait 的实现——类型系统确保每个工具的方法签名与 `ToolDefinition` 一致。omp 在运行时验证——Custom Tool 的 `execute` 函数可以是任意返回 `Promise<string>` 的代码，错误的类型在运行时暴露而非编译时。这体现了一个根本性的权衡：**编译时验证的信息保存率更高（所有失败在发布前被捕获），但运行时验证更灵活（未知的扩展路径在运行时被发现和适配）。**

Slash Commands：命令模式扩展

/ 命令是通过文件系统发现的（`slash-commands.ts`）。在模型论中，Slash 命令是语言 L 的语法糖——它们不引入新的函数符号，而是定义从字符串模式到现有符号组合的语法转换。

例如，用户输入 `/search BM25` 可能被映射为：

```
run_tool("search_tool_bm25", { query: "BM25" })
```

这是一个**缩写定义**（abbreviation definition）——在模型论中通过在语言 L 中添加新的语法构造来实现。缩写定义不改变理论的模型类——每个带缩写符号的公式可以在不带缩写的语言中被等价地写出。但从工程角度看，缩写定义降低了用户的认知负担。

Hooks：观测 + 注入的双向开放

omp 的 Hooks 系统（`hooks/types.ts:599` 行）在事件覆盖范围和能力上远超 DS-TUI。它覆盖 14 种事件类型（`types.ts:378-393`），是 DS-TUI 的两倍多。但关键差异不在事件数量，而在 Hooks 的执行模型。

DS-TUI 的 Hook 是 Shell 进程——事件触发 → 进程执行 → 进程结束（单向观测）。omp 的 Hook 是 TypeScript 工厂函数——`HookFactory = (pi: HookAPI) => void`（`types.ts:586`），接收 HookAPI 对象（`types.ts:470-580`）进行事件订阅（`pi.on()`）和消息注入（`pi.sendMessage()`）。

在模型论中，omp 的 Hooks 是**参与结构构造的观测者**：

$$M_{\text{step},n} \rightarrow \text{Hook 观测事件} \rightarrow \text{Hook 注入新消息} \rightarrow M_{\text{step},n+1}$$

Hook 的输出可以改变结构 M 的下一状态——它注入新消息，这些消息成为 LLM 下一轮推理的上下文。这与 DS-TUI 的“观测不参与”有本质区别。Hook 从语言 L 的**外部观测者**变成了**参与理论 T 的推理链的主动实体**。

更精确的形式化：Hook 的 `pi.sendMessage()` 在结构 M 中添加了一个新的论域元素——一条由 Hook 系统而非用户或 LLM 生成的消息。理论 T 中的公理需要同时约束三种消息源（用户、LLM、Hook）的行为，这增加了理论设计的复杂度，但也扩展了系统的表达能力。

通过 `RegisteredCommand`（`types.ts:459-464`），Hooks 还可以注册 / 命令。这意味着 Hook 不仅观测和注入消息，还可以扩展语言 L 本身的语法——它向语言 L 添加了新的语法构造（缩写定义）。从系统论视角，这是一种**正反馈**：Hook 扩展语言后，LLM 和用户都可以使用新扩展的语言构造，进而产生更多的事件，被更多的 Hook 观测。

两种语言扩展策略的模型论解释

DS-TUI 和 omp 代表了语言扩展的两种极端。将其比较放在模型论框架中，可以揭示其根本差异。

签名适配 vs 接口实现

DS-TUI 的扩展基于**签名适配**（signature adaptation）：外部符号必须通过一个适配层

（`McpToolAdapter`、`ToolSpec`）转换为语言 L 中存在的格式。这对应模型论中的**同构嵌入**：给定外部语言 L_{ext} 和内部语言 L_{int} ，存在一个翻译函数 $\tau : L_{\text{ext}} \rightarrow L_{\text{int}}$ ，将每个外部符号映射到内部表示。

omp 的扩展基于**接口实现**（interface implementation）：外部符号只要实现了 `AgentTool` 接口即可被直接纳入语言 L 。这对应模型论中的**定义扩展**：新符号 f 通过接口方法 `execute` 被显式定义。

关键差异在于**接口的定义位置**：

维度	DS-TUI	omp
接口定义	在系统代码中（ <code>ToolSpec</code> trait）	在系统代码中（ <code>AgentTool</code> 接口）
适配位置	在系统代码中（适配器层）	在扩展代码中（实现接口时）
扩展的签名验证	编译时（Rust trait 检查）	运行时（TypeScript duck typing）
扩展的表达能力	限于适配层能表达的	完全由扩展实现自定义

DS-TUI 选择将适配逻辑放在系统中，意味着系统控制着扩展的表达范围。omp 将适配逻辑交给扩展，意味着系统对扩展几乎没有表达能力控制。

保守扩展 vs 实质扩展

从理论扩展的类型来看：

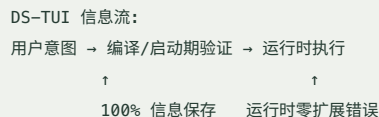
DS-TUI 的扩展天然倾向于**保守扩展**。MCP 工具通过 `McpToolAdapter` 适配，其签名被约束为 `ToolSpec` 可表达的格式——添加的函数符号不改变原始语言 L 中已有句子的可证性。`Skills` 添加新公理但不添加新符号，是更纯粹的保守扩展。核心保证是：**DS-TUI 的任何扩展都不影响基础语言 $L_{\text{DS-TUI}}$ 的模型类 $\text{Mod}(T_{\text{DS-TUI}})$ 在 $L_{\text{DS-TUI}}$ 上的限制。**

omp 的扩展可以构成**实质扩展**。`Extensions` 可以重写系统提示词、修改工具行为、注册新工具——这些操作可能引入新的公理或改变已有公理的解释，从而改变原始语言 $L_{\text{omp_base}}$ 中句子的可证性。omp 不禁止实质扩展，因为它的目标函数不是“保守性”而是“适应能力”。

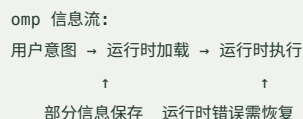
信息保存率：编译时验证 vs 运行时验证

“信息保存率”衡量的是语言扩展的信息完整性——当用户传递意图时，多少信息被系统结构保留，多少在传递过程中丢失。

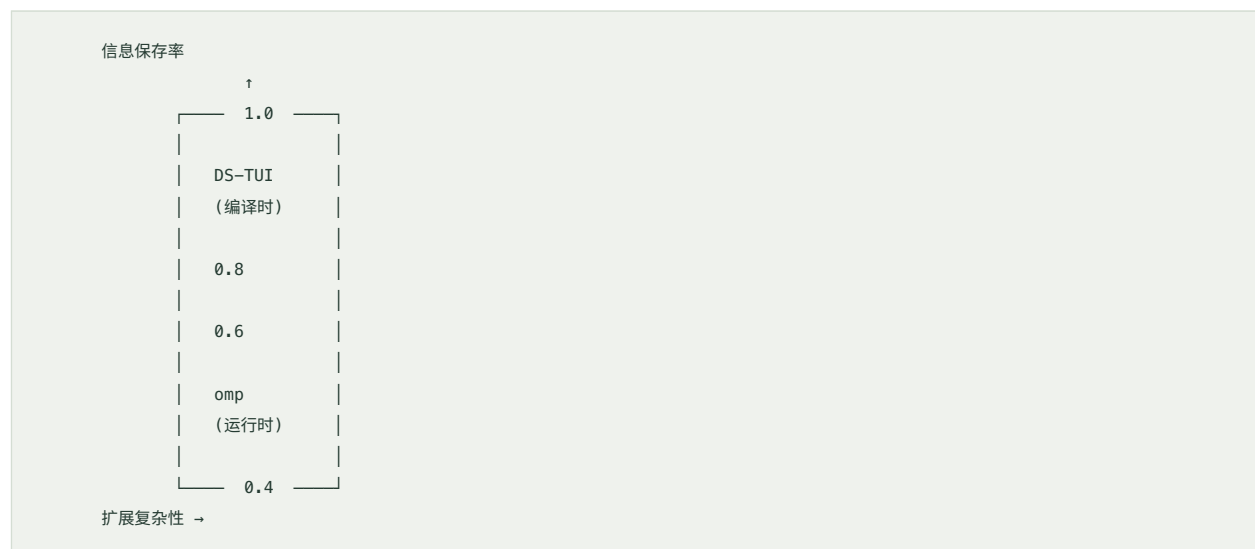
DS-TUI 采用**编译时验证策略**。MCP 工具在启动时被调度、适配并注册。如果适配失败（例如工具 schema 与 ToolSpec 不兼容），系统在启动阶段报告错误，不会进入运行阶段。这意味着扩展错误的信息损失率为 0——错误被完全捕获在编译/启动期，不会进入运行时状态空间。



omp 采用**运行时验证**策略。Custom Tools 和 Extensions 在会话过程中被加载和执行，错误在运行时暴露。类型错误、API 不匹配、权限问题都在使用时刻被发现。这意味着扩展错误的信息损失率取决于运行时错误处理的完善程度。



两种策略的差异可以用下图表示:



当扩展复杂性低时（简单工具、标准签名），两种策略的信息保存率接近 1.0。当扩展复杂性升高时（复杂交互、可变参数、动态行为），DS-TUI 的编译时验证被迫拒绝更多有效扩展（因为静态分析无法表达动态约束），信息保存率下降更快。omp 的运行时验证在复杂度高的区域信息保存率更高——因为运行时可以访问更多上下文信息来验证行为的正确性。

这不是数量上的差异，而是质的差异：DS-TUI 只在“正确”和“编译不通过”之间做选择——这是经典模型论中的二值满足关系。omp 在正确性谱系中做权衡——这是偏离一（软满足）在扩展验证上的自然延伸。

心智模型：语言 L 的认知层面

语言扩展不仅是形式系统的操作，也是认知结构的约束。LLM 通过语言 L 理解世界——语言 L 的边界就是 LLM 推理能力的边界。本节从认知科学的角度分析语言 L 对 LLM 的约束。

分布认知：认知不是只在模型里

赫伯特·西蒙（Herbert Simon）的“有限理性”（bounded rationality）理论指出，人类的认知能力受到信息处理容量的限制。Edwin Hutchins 的“分布认知”（distributed cognition）理论延伸了这一观点：认知不是只局限在个体的大脑中，而是分布在工具、环境、文档和社会的相互作用中。

LLM 的“认知”同样不是只在模型参数里。在 Coding Agent 系统中，LLM 的认知能力依赖于：

- 语言 L 的表达力**：语言 L 中可使用的符号集决定了 LLM 能执行哪些操作。如果一个函数符号 $f \notin \Sigma$ ，LLM 就无法使用 f ——即使它“知道” f 的功能。系统提示词中不包含某个工具，LLM 就无法调用它。
- 结构 M 的论域**：当前会话历史中的消息、文件内容、工具结果构成了 LLM 可引用的论域。论域之外的实体不可引用。
- 理论 T 的约束**：系统提示词中的公理约束了 LLM 的推理路径。公理不完整时，LLM 的回退行为由训练数据决定。

语言扩展——向 L 中添加新符号——在认知层面扩展了 LLM 的可操作范围。LLM 不能使用语言 L 中没有的符号，正如一位只有锤子的工人只能看到钉子。这对应维果茨基（Vygotsky）的“工具中介理论”：认知工具的集合决定了认知活动可能的形式。

DS-TUI 文件系统存储 vs omp 结构化数据库

两个系统的数据存储策略反映了不同的认知架构。

DS-TUI 的文件系统存储直接对应其对语言 L 的态度：知识（Skills）是文件系统中的 `.md` 文件，配置（MCP）是 `mcp.json`，会话历史是固定路径下的 `.sqlite` 数据库。文件系统是一个**无模式的存储系统**——目录结构和文件命名约定承担了模式管理功能。

在认知层面，DS-TUI 的文件系统存储对应**扁平知识结构**：知识以文件为单位组织，跨知识单元的显式关系需要由 LLM 在推理时重建。这类似于人类早期文明的账本——每个交易记录在独立的竹简上，跨竹简的查询需要人工关联。

omp 的结构化数据库（综合的配置系统、模型注册表、等价性判断）对应**关系化知识结构**：知识被编码为类型化的记录，系统自动维护跨条目的关系。模型注册表（`model-registry.ts`:78KB）维护了模型之间的等价关系（`model-equivalence.ts`:26KB），使得 LLM 不需要记住哪些模型可以互相替换。

在认知层面，omp 的结构化存储对应**关联知识网络**：知识被系统化地编码，显式关系由系统而非 LLM 维护。这类似于图书馆的目录系统——图书之间的交叉引用被编入索引，用户不需要记住每本书与其他书的关系。

两种策略的认知效果差异：

维度	DS-TUI（文件系统）	omp（结构化数据库）
信息检索路径	文件路径 + 字符串匹配	类型化查询 + 索引
跨知识关联	LLM 推理时重建	系统预计算并维护
规模扩展瓶颈	目录层级深度（人类可维护性）	查询性能（数据库优化）
认知负担位置	LLM 端（LLM 需理解文件组织）	系统端（系统负责组织关系）
扩展方式	新建文件 + 保持目录约定	注册新类型 + 插入数据库

提示词作为心智模型构造蓝图

语言 L 的根本认知角色是**心智模型构造蓝图**。LLM 每次会话的行为结构 M 都是基于语言 L 和理论 T 构造的"心智模型"——一个关于当前任务、可用工具、环境状态和约束条件的心智图式。

提示词——即理论 T 的文本编码——是构造心智模型的蓝图。蓝图中的每个公理、每个约束、每个描述都影响着 LLM 如何构建其内部表征：

- "You are THE staff engineer..." → 身份公理构造了一个"高级工程师"的心智模型模板
- "Use apply_patch for structural edits" → 偏好规则构造了工具选择的心智分支
- "Verify before proceeding" → 验证义务构造了认知循环中的检查点

当语言 L 扩展时——添加新工具、注入新技能、注册新命令——心智模型蓝图也随之改变。LLM 的心智模型必须"扩容"以容纳新的符号。这种扩容不是无代价的：

注意力稀释。理论 T 中的每个句子都参与约束结构 M 。句子越多，每个句子的边际约束力越弱——类似于混合模型中的注意力摊薄。DS-TUI 的策略：保持 T 短而精确（264 行基础公理），扩展仅限于关键的符号添加。omp 的策略：接受 T 的动态增长，但通过模块化分片（Skills、Rules、Goals、Memories）提供选择性激活。

首因效应与近因效应。 在长会话中，语言 L 的早期符号获得更大的认知权重（首因效应：LLM 更遵循系统提示词开头的指令），而最近添加的符号可能未被充分整合。DS-TUI 的 `base.md` 通过固定的物理位置保证首因效应。omp 的模块化系统需要显式地组织分片顺序来控制认知权重分配。

认知负担与前瞻缓存约束。 语言 L 的每次扩展不仅改变 LLM 的心智模型，还改变提示词的物理字节。对 DS-TUI 来说，这直接决定了前缀缓存的经济性（见第 3.4 节）。对 omp 来说，扩展的认知代价由模块化激活平衡——每个会话只激活与其任务相关的 T 分片，避免无效的心智模型扩容。

语言 L 的扩展策略选择，最终是对**认知经济性**的权衡：DS-TUI 追求极致的认知效率（通过冻结语言 L ），omp 追求极致的认知范围（通过运行时扩展）。两者都是合理的策略，但它们塑造了截然不同的 LLM 行为模式和使用体验。

结语

开放性——语言 L 在运行时的扩展能力——定义了 Coding Agent 的适应边界。DS-TUI 的受控扩展策略（签名适配、编译时验证、冻结的语言层）将其优化目标放在了“扩展的经济性”上——每次扩展尽可能少地影响系统的可优化路径和前缀缓存效率。omp 的激进扩展策略（接口实现、运行时验证、动态加载的语言层）将其优化目标放在了“扩展的可能性”上——最大化系统能接纳的新符号和新行为。

从模型论看，DS-TUI 是保守扩展的典范，omp 是实质扩展的实践者。从系统论看，DS-TUI 是受控开放的谨慎设计，omp 是激进开放的适应性设计。从认知科学看，DS-TUI 将认知负担主要放在系统端（设计时确定语言 L 的边界），omp 将认知负担更多交给运行时（语言 L 随需求动态演变）。

这两种策略没有孰优孰劣——它们服务于不同的目标函数，适用于不同的使用场景。但将它们放在统一的模型论框架下分析，使我们可以**精确地**描述它们的差异，**有预见地**评价各自的长处和局限，并有**根据地**选择策略应对未来的需求。

CHAPTER 06

第六章 目的性：目标函数驱动理论选择

"一切有目的的行为都可以用反馈机制来解释。一个系统如果持续趋向某个特定状态并抵抗偏离该状态的力量，那么该状态就是系统的目标。系统的目标不需要意识——只需要一个隐含的参考状态和一个比较器。"

—— 诺伯特·维纳，《控制论》（1948）

"从模型论看，目标函数就是理论选择的判据：在所有可能的结构类 $\mathbb{K} = \{M_i\}$ 中，使目标函数 $J(M)$ 达到最大的那个结构类 $K_{\text{selected}} \subseteq \mathbb{K}$ ，就是被系统实现的理论 T 。因此，不同的目标函数导致不同的理论选择——即使初始公理集完全相同。"

—— 本章的核心命题

系统论目的性原理

目的性在四大原理中的位置

在钱学森的系统论中，目的性（purposefulness）是四大原理之一，与整体性、层次性、开放性并列。但它在四大原理中占据一个特殊位置：整体性、层次性和开放性回答的是"系统如何构成"——系统由哪些部分以什么方式组织、分解为哪些子结构、与环境的边界在哪里。而目的性回答的是完全不同的问题：**系统为什么以它当前的方式行为？什么引导着系统的结构分化和演化方向？**

这个区别在模型论的框架中尤为清晰。整体性对应理论的一致性（consistency）—— T 的所有公理在同一个结构 M 中相容。层次性对应理论塔的层叠结构—— $T_0 \subset T_1 \subset T_2$ ，高层理论在低层理论的基础上延伸。开放性对应语言 L 的可扩展性——LLM 系统的形式语言不是封闭的，而是在运行时持续接纳新符号（工具签名、MCP 协议、用户自定义命令）。

而目的性对应的是：**在所有可能的理论扩张 T' 中，为什么系统选择了这一个而不是另一个？**

在模型论中，一个理论 T 通常有无穷多个模型。一阶算术有标准模型 N ，也有非标准模型——包含无穷大元素。群论有无穷多互不同构的群。理论本身不唯一确定它的实现结构。但在工程系统中，我们只允许一种实现——系统运行时处于且仅处于一个状态。目的性原理给出了从可能空间到实际选择的映射：**目标函数 $J: \mathbb{K} \rightarrow \mathbb{R}$ 从结构类中挑选出实际被实现的那一个。**

这一视角将"目的"从一个哲学概念转化为工程可操作的分析工具：要理解一个系统的架构，不需要追问它的设计者的意图，只需要找出**系统行为可观察的一致性方向**——它在持续优化什么、抵抗偏离什么、以什么标准做决策。

目标函数的结构分化功能

目标函数不仅是评价准则——它是**结构分化**（structural differentiation）的驱动力。系统论中的一个基本实证发现是：具有不同目标函数的系统，即使在解决形式上完全相同的问题时（比如"管理上下文窗口"），也会演化出根本不同的内部结构。

这一现象的深层原因在于：工程结构是对目标函数的物理编码。一个模块内部的每个 `if` 分支、每个 `match` 语句、每个配置参数，都是对目标函数的一个维度的实现。目标函数决定了：

1. **什么信息需要被测量。** 如果优化目标是缓存命中率，系统需要测量前缀的字节序列一致性（SHA-256 指纹）；如果优化目标是供应商多样性，系统需要测量每个供应商的可用性和质量评分。
2. **什么信息可以被丢弃。** 如果优化缓存命中率，供应商的响应时间波动是次要的噪声；如果优化供应商多样性，缓存命中率在各供应商之间不可移植，因此缓存优化是次要的。
3. **什么控制结构是合理的。** 单一供应商下，编译时固定的系统提示词是合理的（因为前缀不变性是最优策略）；多供应商下，运行时组合的模块化提示词是必要的（因为每个供应商需要不同的适配层）。
4. **什么扩展机制是必需的。** 固定前缀需要编译时嵌入的静态系统提示词和工具注册表；供应商多样性需要运行时发现、模型等价性判断、动态认证解耦。
5. **什么错误处理是有意义的。** 缓存优化环境下，溢出是罕见且昂贵的错误（应全力避免）；供应商多样性环境下，溢出是预期的异常之一（有多个供应商可回退）。

本章的核心论题是：DS-TUI 和 Oh My Pi 的根本架构差异——包括它们在压缩策略、工具组织、错误处理、扩展机制上的所有分歧——都可以追溯到它们不同的目标函数。不是设计者的"选择偏好"导致了差异，而是目标函数作为"第一推力"在每一个决策点上强制了特定的方向。

反馈控制的模型论解释

目的是抽象的——它存在于系统的设计文档和架构师脑海中。要让目的变为可操作的行为，需要一个实现机制。这就是控制论。控制论在系统论目的性原理中的位置不是一门独立的学科，而是**目的性的运作机制**：系统论定义了"系统有什么目的"，控制论解释了"目的如何通过反馈、前馈和层级结构在运行时得以维护和实现"。

在模型论框架中，控制论的运作可以被精确表述为**维持或偏离** $M \models T$ 的动态过程。

负反馈：维持 $M \models T$ 的不变性

负反馈（negative feedback）是维纳控制论的核心。它的模型论表述是：

给定一个理论 T （系统提示词的公理集合），系统在每一时刻 t 有一个当前结构 M_t （当前会话状态——包括已发生的工具调用、已接收的消息、已做出的决策）。负反馈维持的操作是：

当 M_t 偏离 T 时（ $M_t \not\models \varphi$ 对于某个 $\varphi \in T$ ），控制器施加修正动作 u ，使 M_{t+1} 重新满足 T （ $M_{t+1} \models T$ ）。

这可以形式化为一个循环：

感知： $M_t \not\models \varphi$? 其中 $\varphi \in T$

计算： $e_t = \text{偏差}$ —— φ 中哪些约束被违反？

决策： $u_t = C(e_t)$ —— 选择修正动作

执行： $M_{t+1} = \text{apply}(M_t, u_t)$

验证： $M_{t+1} \models \varphi$?

在 LLM 编码助手的上下文中，最常见的负反馈回路是上下文压缩。考虑理论 T 中隐含的约束

φ_{context} ：“会话中的总 token 数不超过模型上下文窗口的容量”。当 M_t 违反 φ_{context} （token 超限），压缩动作 u_{compact} 将 M_t 映射为 M_{t+1} （删除或摘要化历史消息），使 M_{t+1} 重新满足 φ_{context} 。

这个负反馈回路有两个关键性质：

第一，**回路增益决定稳定性**。如果压缩动作 u_{compact} 过于激进（压缩后 token 数量远远低于阈值），系统可能丢失了关键信息的结构 M 的重要部分，使得后续轮次中 M_{t+2} 在满足 φ_{context} 的同时不再满足其他公理 $\varphi' \in T$ （如“保持工作记忆中的文件上下文”）。这是过度调节导致的性能下降。

第二，**反馈延迟决定响应速度**。如果系统只在 API 返回溢出错误后才触发压缩（反馈延迟 = 一次 API 调用的往返时间）， M_t 已经越过了 T 的允许边界。压缩动作将 M_t 拉回安全区域，但已经产生了成本和延迟代价。

在 DS-TUI 中，负反馈的经典实例是 compaction。compaction.rs 中的 `should_compact()` 函数定义了一个阈值检查：`active_input_tokens >= config.token_threshold`，其中默认 `token_threshold = 800,000`。这个 80% 阈值不是随意选择的——它是维持 `stable_prefix` 的动态平衡点：既足够早地触发以预留响应头空间，又不至于早到频繁破坏前缀缓存。

在 omp 中，负反馈通过 `shouldCompact()` (compaction.ts) 实现，其默认策略 `resolveThresholdTokens()` 中的 `reserveTokens = 16384` 体现了一个不同的平衡点——保留约 16K token 作为安全边际。两者都是在维持 $M \models \varphi_{\text{context}}$ ，但阈值不同，因为它们的风险偏好不同（DS-TUI 更重视缓存保护，因此倾向于更晚触发；omp 更重视避免溢出，因此倾向于更早触发）。

正反馈： M 偏离 T 的加速过程

正反馈（positive feedback）在工程系统中几乎总是有害的。它的模型论表述是：

当 M_t 偏离 T 时，修正动作 u 不仅没有减少偏差，反而使 M_{t+1} 的偏离更大。

形式化：

如果存在 $\varphi \in T$ 使得 $M_t \not\models \varphi$,

施加修正 u ,

结果 M_{t+1} 在更多公式上偏离 T （即 $|\{\varphi \in T : M_{t+1} \not\models \varphi\}| > |\{\varphi \in T : M_t \not\models \varphi\}|$ ），

则称该反馈回路为正反馈。

在 LLM 编码助手中，正反馈的经典场景是**错误扩散循环**：

1. M_t 中包含一个错误的工具输出（如错误地执行了删除文件的命令）
2. LLM 在后续推理中看到了这个错误输出，生成一个更复杂的修正方案
3. 修正方案执行后引入了更多的新消息和工具输出
4. M_{t+1} 的上下文更大、错误更多，模型在庞大错乱的上下文中更难做出正确判断
5. 循环继续

这就是为什么两个系统都实现了**循环守卫**（loop guard）：DS-TUI 的 `MAX_STEPS_PER_TURN`、`TURN_TIMEOUT` 和 `MAX_BYTES_PER_TURN`；omp 的 `maxSteps` 和 `timeout`。模型论上，这些守卫执行的是**强制回路断开**——切断从 M_t 到 M_{t+1} 的反馈路径，将控制权从自动反馈回路转移到更高层监督者（通常是用户）。

前馈：在 M 偏离 T 之前修正 T

前馈控制（feedforward control）在目的性框架中占据一个独特位置。与反馈在偏差发生后才行动不同，前馈在偏差发生之前就施加补偿。它的模型论表述是：

基于扰动预测模型 M_{pred} ，在输入 x 进入系统之前判断它是否会导致未来的 $M \not\models \varphi$ 。如果预测到未来的偏离，在偏离发生之前修改理论 T （临时增加约束 $\varphi_{\text{feedforward}}$ ）或修改输入 x 。

形式化：

输入 x （即将发送的请求）

预测： $M_{\text{pred}}(x) \rightarrow$ 如果原样发送， M_{t+1} 是否可能 $\not\models \varphi$ （如"上下文不溢出"）？

如果预测为"是"：

修正：修改 x （如压缩后再发送）或临时扩充 T （如注入一条"请更简洁"的约束）

发送修正后的请求 x'

期待： $M_{t+1} \models \varphi$ （因为前馈补偿了潜在偏差）

前馈的独特之处在于它修正的是**输入或理论**，而非输出。DS-TUI 的 `capacity_controller` 是这一模式的典范。它的 `observe()` 方法在每次 API 请求前计算故障概率 p_{fail} ，如果 p_{fail} 较高，系统在请求发送之前就做出干预——或要求模型刷新上下文（`TargetedContextRefresh`），或要求验证并重放（`VerifyWithToolReplay`）。干预的效果是修正即将发送的输入 x ，而不是等待输出错误后再修正。

从 v0.8.11 开始，`capacity controller` 默认关闭。这一决策在模型论框架中有一个深刻的解释：前馈控制依赖于预测模型 M_{pred} 的准确性。当 M_{pred} 的误报率（假阳性：预测溢出但实际未溢出）触发不必要的干预，而这些干预本身又会**破坏系统运行的条件**（干预改写消息日志，破坏了 `prefix cache`）时，前馈控制就变成了自我挫败的——预测和干预的联合效应反而使系统更远离目标。因此关闭 `capacity controller` 是在默认情况下（V4 模型的 1M 窗口足够大）做出的合理决策：当预测质量不足以超过反馈的简单性时，选择更简单的反馈控制比复杂但不可靠的前馈更好。

DS-TUI 的目标函数：最大化缓存命中率

目标函数的数学形式

DS-TUI 的根本目标函数可以形式化地写为：

$$J_{\text{DS-TUI}} = \max \frac{\text{cache_hit_tokens}}{\text{total_input_tokens}}$$

在 DeepSeek V4 的经济模型中，cache hit 的定价约为 cache miss 的 1/120。这意味着每 1% 的缓存命中率提升，在大规模使用下对应约 1% 的总成本降低。但这还不是全部：缓存命中不仅是成本问题——它还意味着响应时间的降低（命中时前缀 KV 计算被跳过）。因此，缓存命中率的提升同时优化了经济目标和性能目标。

这个目标函数的特性值得深入分析：

- **域敏感性**：目标函数只在单一供应商（DeepSeek V4）的单一缓存架构下有定义。跨供应商的缓存命中率没有意义——不同供应商的缓存系统不共享。
- **单调性**：在任何其他条件不变的情况下，更高的缓存命中率总是更好的。没有“最优缓存命中率”——系统的目标就是尽可能地接近 100%。
- **结构性约束**：目标函数不直接控制缓存命中率（系统无法控制 DeepSeek 内部的缓存调度），它只能控制自己这一侧的可变因素——提示词前缀的稳定性。

这种特性意味着 DS-TUI 的所有架构决策都受到一个隐性约束的支配：**不要改变前缀，除非代价超过收益**。这一约束渗透到了系统的每一个模块。

PrefixStabilityManager：SHA-256 指纹的守卫

PrefixStabilityManager (`crates/tui/src/prefix_cache.rs:140-306`) 是 DS-TUI 目标函数最直接的编码。它的职责是监控系统提示词和工具集的前缀稳定性——用 SHA-256 哈希计算前缀的指纹，在每次请求前检查指纹是否发生变化。

它的状态机由四个字段定义：

```

/// 系统提示词最近的 SHA-256 哈希值
cached_system_prompt_hash: Option<[u8; 32]>,
/// 工具注册表签名的最近哈希值
cached_tool_spec_hash: Option<[u8; 32]>,
/// 连续缓存命中计数（用于稳定性比率计算）
stability_counter: usize,
/// 自上次变化以来的稳定性比率
/// 定义：稳定性比率 = stability_counter / (stability_counter + change_counter)
stability_ratio: f64,

```

check_and_update() 方法执行核心逻辑：

```

pub fn check_and_update(&mut self, current_system_prompt: &str, current_tool_specs: &str) -> Option<PrefixCacheChange> {
    let new_sys_hash = sha256(current_system_prompt);
    let new_tool_hash = sha256(current_tool_specs);

    let sys_changed = Some(&new_sys_hash) != self.cached_system_prompt_hash.as_ref();
    let tool_changed = Some(&new_tool_hash) != self.cached_tool_specs_hash.as_ref();

    let changed = sys_changed || tool_changed;

    if changed {
        self.stability_counter = 0;
        self.change_counter += 1;
    } else {
        self.stability_counter += 1;
    }

    self.stability_ratio = self.stability_counter as f64
        / (self.stability_counter + self.change_counter).max(1) as f64;

    self.cached_system_prompt_hash = Some(new_sys_hash);
    self.cached_tool_specs_hash = Some(new_tool_hash);

    if changed {
        Some(PrefixCacheChange {
            system_prompt_changed: sys_changed,
            tool_spec_changed: tool_changed,
            new_stability_ratio: self.stability_ratio,
        })
    } else {
        None
    }
}

```

从模型论的角度看，PrefixStabilityManager 维护的是一个关于理论 T 的元理论信息：它不关心 T 的语义内容（提示词中写了什么规则），只关心 T 的语法形式（提示词的字节序列是否与前一轮相同）。SHA-256 哈希是一个元观察函数——它将一个理论 T 映射到一个 32 字节的指纹 $f(T)$ ，然后通过比较 $f(T_t)$ 和 $f(T_{t-1})$ 来检测 T 的变化。这是一个纯粹的形式化操作：它不需要理解 T 的含义，只需要逐字节比较。

前缀布局：最静态内容最前面

目标函数为最大化缓存命中率，直接决定了提示词的物理布局策略。DS-TUI 的系统提示词遵守一个严格的不变量：**内容按稳定性从高到低排列**。最静态的内容放在最前面，因为 DeepSeek V4 的 prefix cache 以 128-token 粒度工作，前缀的字节序列必须在请求间完全一致才能命中缓存。

提示词的七层结构验证了这一原则（`crates/tui/src/prompts.rs`）：

层号	内容	稳定性	说明
L_1	Agent 身份定义 (base.md)	完全静态	编译时嵌入，从不变化
L_2	Personality 层	完全静态	同一用户配置下不变
L_3	Mode 层 (normal/plan)	会话级静态	同一会话内不变
L_4	Approval 层 (权限门控)	会话级静态	同一会话内不变
L_5	技能层 (skills)	分支级静态	同项目内不变
L_6	工作集 (working_set)	动态	每轮可能变化
L_7	当前轮次用户消息	完全动态	每轮都不同

关键的工程决策是：**volatile-content-last**—— L_6 和 L_7 被放在提示词的末尾（通过在最后添加 **Volatile-content boundary** 注释标记）。这意味着在 DeepSeek V4 的 128-token 缓存粒度下， L_1-L_5 的静态内容被完全缓存命中，每次请求只需计算 L_6-L_7 的 KV。如果反过来（动态内容在前），每次请求的前缀在第一个动态 token 处就发生偏移，缓存完全失效。

在这个布局中，`working_set.rs` (57.1KB) 扮演了一个有趣的矛盾角色：它的内容在每轮用户输入后更新（被当前操作的文件路径替换），但它的更新位置被严格限制在缓存边界**之后**。工作集的内容被注入到用户消息（块）而非系统提示词中，使得它的变化不会影响提示词前缀的字节流。这是“静态外壳包裹动态内核”策略的一个执行细节。

工具排序字母序固定

工具在系统提示词中的注册顺序同样是缓存优化的一部分。如果工具集在不同的请求间输出不同的顺序，即使所有工具的定义内容完全不变，变更的顺序也足以使中间某个工具的定义之后的全部字节偏移——从而破坏缓存命中。

DS-TUI 的解决方案是**字母序固定排序**（`crates/tui/src/tools/registry.rs`）。ToolRegistry 在构建时对所有工具的描述进行字母序排序，并存储在 OnceLock 缓存中：

```
pub fn get_tool_descriptions(&self) -> &Vec<ToolDefinition> {
    self.tool_descriptions.get_or_init(|| {
        let mut descriptions: Vec<_> = self.tools.iter().map(|t| t.definition()).collect();
        descriptions.sort_by(|a, b| a.name.cmp(&b.name));
        descriptions
    })
}
```

注意 `OnceLock` 的语义：工具描述在第一次访问后就被永久缓存，之后在同一进程中永不变化。这意味着所有工具在提示词中的出现顺序在进程生命周期内完全固定——即使运行时注册了新工具（通过 MCP），已缓存的部分也不会变化。

从模型论的角度看，这相当于为理论 T 的工具签名部分固定了一个**规范序**（canonical order）。规范序消除了理论表示中的等价类内部变异——两个在逻辑上等价的理论 T 和 T' （工具集相同、定义相同、仅排序不同）在 DS-TUI 中被强制表示为相同的字节序列。

这种策略的代价是：**运行时动态工具无法进入固定前缀**。如果通过 MCP 协议注册了一个新工具，它的定义被追加到固定排序的工具列表之后——这个新位置打破了前缀的连续性，导致缓存从新工具插入点开始全部失效。DS-TUI 对此的工程取舍是：编译时注册的静态工具（32+ 个）覆盖了核心功能需求，运行时注册的 MCP 工具作为扩展集被放在缓存边界之后。这是“最大化缓存命中率”目标函数的直接体现：静态工具进入缓存前缀是优先级最高的，动态工具被接受但付出缓存代价。

Compaction 80% 阈值

`compaction.rs` 中的 `token_threshold: 800,000`（在 1M 窗口下相当于 80%）的触发值，也是目标函数驱动的工程选择。

选择 80% 而非 90% 或 70% 的原因需要在模型经济学和缓存保护的张力中理解：

- **更高的阈值（如 90%）**：更少的压缩次数（缓存破坏次数更少），但每次触发时上下文可用空间更小（10% 的响应头空间），模型输出被截断的概率更高。
- **更低的阈值（如 70%）**：压缩更频繁，缓存破坏更频繁，但每次压缩后上下文更宽松。

80% 是 DS-TUI 设计者的经验判断：它提供了 200K token 的响应头空间（足够 V4 模型在一次推理中完成复杂的编码任务），同时将压缩频率控制在合理范围内（大多数会话每天触发 2–5 次压缩，在常见使用模式下）。

但更重要的是 500K 的 `auto_floor_tokens`（`MINIMUM_AUTO_COMPACTIION_TOKENS`）。这个硬地板的存在从侧面证实了“最大化缓存命中率”是比“最大化信息保留”更优先的目标。注释明确解释了：

compaction rewrites the stable prefix, which destroys the KV cache. At low token counts the prefix cache is healthy and compaction's cost (full re-prefill at miss prices) dwarfs its benefit (a tiny budget reclaim).

在模型论框架中，500K 地板定义了一个**局部最优陷阱**：如果 token 数在 500K–800K 之间，系统不允许自动压缩，即使信息压缩能提高后续推理效率——因为压缩破坏缓存的代价超过了信息压缩的收益。只有当 token 数超过 800K 时，缓存破坏的代价才被“容许可接受”——因为不压缩的系统崩溃风险（上下文溢出）更大。

CapacityController 前馈的停用

capacity_controller 从 v0.8.11 起默认关闭的事实，是目标函数支配架构决策的最强证据。

capacity_controller 的设计（前馈预测 + 干预）在抽象控制论角度是优秀的——它计算故障概率、分级响应、有冷却期抑制震荡。但在 DS-TUI 的目标函数框架下，它的干预是**有害的**：

capacity_controller 的 VerifyWithToolReplay 会改写消息日志——重放最近的工具调用并替换旧结果。这改变了消息列表结构，从而破坏了 DeepSeek V4 的 prefix cache。如果 cache hit 和 cache miss 之间差 120 倍的价格，一次 VerifyWithToolReplay 干预可能在后续 50–100 轮请求中破坏缓存，总代价远超它避免的那次溢出。

因此，DS-TUI 做出的决策是：关闭 capacity_controller，信任模型处理完整 1M-token 上下文的能力。这是目标函数驱动下做出的反直觉但合理的决策——“更好的控制”（更精细的压力管理）如果损害了根本目标（缓存命中率），就不是更好的。

omp 的目标函数：最大化供应商多样性

目标函数的数学形式

Oh My Pi 的根本目标函数可以形式化地写为：

$$J_{\text{omp}} = \sum_{p \in \text{Providers}} w_p \cdot \text{availability}(p) \cdot \text{quality}(p)$$

其中 w_p 是供应商权重（由用户配置、成本效率和地理接近性决定）， $\text{availability}(p)$ 是供应商在当前时间点的可用性（0 或 1，由健康检查和错误率决定）， $\text{quality}(p)$ 是供应商对当前任务的适用度（由模型等价性判断和性能历史决定）。

这个目标函数的特性与 DS-TUI 的目标函数截然不同：

- **域无关性**：目标函数不绑定到特定供应商的缓存架构。它通过多供应商覆盖对冲单一供应商的失效风险。
- **可替换性**：在任何其他条件不变的情况下，支持更多供应商总是更好的——即使每个供应商的使用频率都很低。多样性本身就是价值。
- **约束型**：目标函数不是单峰值（不是追求单个最优值），而是在约束条件下（成本、延迟、质量）最大化覆盖空间。

这种特性驱动了 omp 的架构决策向完全不同的方向演化：它不是集中式优化单一目标的少数变量，而是建立一套分布式协调框架，使 40+ 供应商能够以统一的方式被访问、比较和替代。

独立 AI 层和 30+ 供应商的抽象统一

omp 维护了一个**独立的 AI 抽象层**（`packages/ai/`），将 40+ 个 LLM 供应商的 API 差异封装在一个统一的接口后面。这个层的核心设计问题不是“如何优化一个固定供应商的性能”，而是“如何在尽可能多的供应商之间建立可替换性”。

`model-registry.ts` 实现了模型注册表，核心数据结构是模型等价性索引：

```
interface ModelEquivalenceIndex {
  provider: string;      // 供应商 ID
  id: string;            // 模型 ID
  canonicalIndex: number; // 等价类索引—可互换的模型有相同索引
  capabilities: ModelCapabilities;
  cost: ModelCost;
}
```

`canonicalIndex` 的存在是实现目标函数的关键。当供应商 *A* 的模型不可用时，系统查找具有相同 `canonicalIndex` 的其他模型——它们在语义上被视为等价的可替换选项。这直接对应于模型论中的**同构**概念：两个结构 M_1 和 M_2 如果在语言 L 的所有公式下不可区分，则它们是同构的。omp 的 `canonicalIndex` 定义了一个**可接受的近似同构**——供应商 *A* 和供应商 *B* 的两个模型可能 API 协议不同、定价不同、推理行为有微妙差异，但只要它们被标记为相同的 `canonicalIndex`，系统就假定它们在目标函数中是等价的。

这种等价性判断的精确度直接影响目标函数的质量。过于宽松的等价性（将不可互换的模型标记为等价）会导致用户获得的推理质量下降；过于严格的等价性（只标记完全同构的模型）会减少可替换选项，降低供应商多样性的实际效用。omp 的模型注册表通过逐模型的手动标注 + 自动定期的等价性验证来管理这种张力。

RetryFallbackSelector：供应商感知回退

在没有目标函数驱动的情况下，一个简单系统遇到 LLM 调用失败时只会重试同一模型。在"最大化供应商多样性"的目标函数驱动下，omp 实现了一个更复杂的机制——RetryFallbackSelector（agent-session.ts:407-444）。

回退链的关键设计是：

```
interface RetryFallbackSelector {
  raw: string;          // "anthropic/claude-opus-4-20250514"
  provider: string;     // "anthropic"
  id: string;           // "claude-opus-4-20250514"
}
```

回退链的解析和遍历逻辑：

```
function getRetryFallbackEffectiveChain(role: string): RetryFallbackSelector[] {
  const primary = getRetryFallbackPrimarySelector(role);
  if (!primary) return [];
  const chain = [primary];
  const seen = new Set([primary.raw]);
  for (const selector of getRetryFallbackChains()[role] ?? []) {
    const parsed = parseRetryFallbackSelector(selector);
    if (!parsed || seen.has(parsed.raw)) continue;
    seen.add(parsed.raw);
    chain.push(parsed);
  }
  return chain;
}
```

回退链的执行逻辑（简化）：

1. 首次调用使用主选模型（如 anthropic/claude-opus-4）
2. 如果主选模型调用失败（网络错误、API 错误、速率限制）：
 - a. 查找回退链中的下一个候选
 - b. 通过模型注册表验证该候选是否可用（find/provider/id）
 - c. 检查该候选是否被抑制（suppressed）
 - d. 获取该候选的 API 密钥
 - e. 使用候选重新发送请求
3. 如果候选仍然失败，继续到下一个候选
4. 如果所有候选都不成功：短路（`fallbackShortCircuit`）

从模型论的角度看，回退链定义了一个模型序列 $\{M_1, M_2, M_3, \dots, M_n\}$ ，其中每个 M_i 是同构类中的一个候选项。当 $M_i \not\models \varphi$ （调用失败，即 M_i 未能实现用户请求所对应的理论 T_{request} 的子集），系统不是终止或报错，而是切换到 M_{i+1} ，尝试用同构类中的下一个候选来满足 T_{request} 。

这个回退链的工程含义深刻：omp 在设计时假设了**单一供应商不可靠**。这不是一个悲观假设，而是"最大化供应商多样性"目标函数的直接推论——如果你相信多样性是好的，你必然会为每个候选独立失败（而非系统性地同时失败）建立模型。这与 DS-TUI 的设计假设形成鲜明对比：DS-TUI 假设 DeepSeek 作为单一供应商是可靠的，因此不需要跨供应商的回退路径。

思考级别映射的统一抽象

omp 维护了五个统一的思考级别（Effort 枚举：Minimal / Low / Medium / High / XHigh），但 40+ 个供应商对思考级别的支持各不相同。model-thinking.ts（683 行）实现了从统一级别到供应商特定参数的映射。

核心映射函数：

```
function mapEffortToAnthropicAdaptiveEffort(model, effort): string {
  switch (requireSupportedEffort(model, effort)) {
    case Effort.Minimal: case Effort.Low: return "low";
    case Effort.Medium:           return "medium";
    case Effort.High:             return "high";
    case Effort.XHigh:
      return anthropicModelHasRealXHighEffort(model) ? "xhigh" : "max";
  }
}
```

各供应商的支持集差异：

- Anthropic Opus 4.7+: Minimal, Low, Medium, High, XHigh（支持真实 XHigh）
- Gemini 3 Pro: Low, High（仅两级）
- Gemini 3 Flash: Minimal, Low, Medium, High（四级，无 XHigh）
- GPT-5.2+: Low, Medium, High, XHigh（无 Minimal）
- GPT-5.1 Codex Mini: Medium, High（仅两级）

当请求的级别不被支持时，执行夹紧（clamping）：

```
function clampThinkingLevelForModel(model, requested): Effort | undefined {
  const levels = getSupportedEfforts(model);
  if (levels.includes(requested)) return requested;
  let clamped: Effort | undefined;
  for (const effort of levels) {
    if (THINKING Efforts.indexOf(effort) > THINKING Efforts.indexOf(requested)) break;
    clamped = effort;
  }
  return clamped ?? levels[0];
}
```

从模型论看，思考级别映射定义了一个从**统一理论** T_{effort} ("用户请求的推理强度") 到**供应商特定语言** L_p 的**翻译函数** $\tau_p : T_{\text{effort}} \rightarrow \Sigma_p$ ，其中 Σ_p 是供应商 p 的 API 参数签名。夹紧处理的是翻译函数不能保持语义精度的情况——当 T_{effort} 中的符号（如 "XHigh"）在供应商 p 的语言 L_p 中没有对应物时， τ_p 选择最接近的下级符号（"max" 或 "high"）。这是一个**近似解释**（approximate interpretation）——翻译后的签名 Σ'_p 满足 T_{effort} 的程度低于原请求，但在系统中是可接受的。

认证存储解耦

在"最大化供应商多样性"的目标函数下，认证管理（API 密钥、访问令牌）必须与核心控制逻辑解耦。40+ 供应商的认证机制各不相同——Anthropic 使用 API key Header，OpenAI 支持 API key 和 OAuth，Google 使用 API key 或服务账号，Ollama 和 llama.cpp 等本地模型不需要认证。

omp 的认证管理系统通过 `modelRegistry.getApiKey(model, sessionId)` 封装了这种多样性。调用者不需要知道具体供应商的认证协议——认证选择是 `modelRegistry` 的内部实现细节。这种解耦设计使系统能以统一的方式管理 40+ 供应商的异构认证需求。

从模型论的角度看，认证存储解耦实现了一个**元语言/对象语言的分离**：核心逻辑在对象语言 L_{obj} （模型选择、思考级别映射、回退链遍历）中运行，而认证细节在元语言 L_{meta} （认证存储、密钥查找、令牌刷新）中处理。 L_{obj} 中的每个"模型选择"决策都可以独立于 L_{meta} 中的"如何认证"细节。这种分离使得目标函数（最大化供应商多样性）可以在不增加核心逻辑复杂度的前提下，通过元语言层支持任意数量的供应商。

目标函数差异的形式化分析

同一问题的不同最优解

两个系统面对的本质问题是同一个：在有限上下文窗口约束下，使编码智能体能够高效地完成长时间运行的编码任务。但"高效"的标准由不同的目标函数定义，导致系统选择了不同的"最优"架构。

这可以通过形式化的决策理论框架来分析。设系统面临一个决策空间 D （可能的状态-动作对组合），目标函数 $J : D \rightarrow \mathbb{R}$ 定义每个决策的质量。不同的 J 导致不同的最优决策 $d^* = \arg \max_d J(d)$ 。

决策一：提示词组装策略

- DS-TUI 的 J 偏好 d1：编译时嵌入的静态提示词 + 工作集在缓存边界后注入
- 理由：前缀字节流不变 $\rightarrow$ 缓存命中率最大 $\rightarrow J$ 最大
- omp 的 J 偏好 d2：运行时组装的模块化提示词（skills $\times$ rules $\times$ goals $\times$ memories）
- 理由：需要为不同供应商的不同能力注入不同的适配层 $\rightarrow$ 供应商多样性最大 $\rightarrow J$ 最大

决策二：上下文压缩策略

- DS-TUI 的 J 偏好 d1: 追加式摘要 (soft seam) 优先于替换式压缩
- 理由: 追加不破坏前缀缓存, 替换破坏 $\rightarrow$ 缓存命中率最大 $\rightarrow J$ 最大
- omp 的 J 偏好 d2: 可配置策略 (handoff / context-full / OpenAI 远程压缩)
- 理由: 不同供应商支持不同的压缩模式, 需要适配多种策略 $\rightarrow$ 供应商多样性最大 $\rightarrow J$ 最大

决策三：工具组织方式

- DS-TUI 的 J 偏好 d1: 编译时注册 + 字母序固定
- 理由: 工具描述前缀字节流不变 $\rightarrow$ 缓存命中率最大 $\rightarrow J$ 最大
- omp 的 J 偏好 d2: 运行时注册 + MCP 协议 + 用户自定义工具
- 理由: 需要为不同供应商提供不同工具集, 运行时扩展是多样性的前提 $\rightarrow$ 供应商多样性最大 $\rightarrow J$ 最大

决策四：错误处理策略

- DS-TUI 的 J 偏好 d1: 透明重试 (同供应商重试最多 3 次) + 硬阈值安全网
- 理由: 在信任单一供应商的前提下, 错误是偶发的、可重试的 $\rightarrow$ 维持原定路径
- omp 的 J 偏好 d2: 供应商感知回退 (RetryFallbackSelector) + Harmony 泄漏检测
- 理由: 错误是预期的, 应从候选集切换而非重试供应商 $\rightarrow$ 多样性覆盖

目标函数偏好的结构性映射

以上的形式化分析揭示了一个结构性规律: DS-TUI 的 J 偏好"稳定与持久"——任何破坏前缀不变性的操作都被惩罚; omp 的 J 偏好"灵活与覆盖"——任何限制供应商可替换性的策略都被惩罚。

这个规律不是偶然的——它源于目标函数本身的数学性质:

性质	DS-TUI 的 J	omp 的 J
定义域	DeepSeek V4 单一实例	所有支持供应商的并集
变量类型	连续 (缓存命中率 0-1)	离散 (供应商数量整数)
最优解形状	单峰值 (100% 缓存)	帕累托前沿 (多目标折中)
约束敏感性	对前缀变化极度敏感	对供应商不可用敏感
时间维度	所有轮次共享同一前缀	每轮可换供应商

单峰值的 J 自然引导系统走向"精炼单一解"的架构——把所有工程资源投入到优化一个核心路径上。帕累托前沿的 J 自然引导系统走向"维护多种折中方案"的架构——把工程资源投入到使多种路径可替换和可协调上。

为什么不能同时最大化两个目标

一个自然的问题是：为什么不能同时"最大化缓存命中率"和"最大化供应商多样性"？

答案是：这两个目标在工程上冲突。

- **缓存命中率要求前缀稳定。**这意味着提示词不能在不同供应商间变化、不能频繁修改、不能为不同供应商的适配层改变结构。
- **供应商多样性要求适配变化。**不同供应商需要不同的提示词格式（Anthropic 的系统提示词在 Messages API 中有特殊的 XML 格式要求，OpenAI 的 responses API 使用不同的结构，Gemini 的 API 有内容安全约束）、不同的思考级别映射、不同的工具描述格式。

如果同时最大化两个目标，系统必须维护多个提示词前缀（每个供应商一个），每个前缀单独优化缓存。这在理论上是可行的（为每个供应商维护独立的 PrefixStabilityManager），但在实践中，多个前缀意味着失去跨供应商共享缓存的经济效益——每个供应商的前缀缓存从零开始积累。更根本的是，供应商多样性的核心价值在于**在运行时切换供应商**，而缓存命中率的核心价值在于**保持前缀不变**——这两个价值在运行时是无法同时实现的。

因此，两个系统做出了不同的选择。DS-TUI 选择了"一个供应商 + 极致缓存"，omp 选择了"多供应商 + 灵活切换"。这不是对错之分，而是目标函数不同造成的自然分岔。

目标函数驱动架构分化对比表

以下表格系统性地对比了 DS-TUI 和 omp 在 12 个架构维度上的差异，以及每个差异如何追溯到目标函数的不同选择。

维度	DS-TUI（目标：max cache_hit/total）	omp（目标： $\sum_p w_p \cdot \text{avail}(p) \cdot \text{quality}(p)$ ）
提示词策略	编译时嵌入单体（include_str!("base.md")），264 行公理系，缓存边界分隔	运行时组装模块化提示词（skills × rules × goals × memories），Handlebars 模板控制

维度	DS-TUI（目标：max cache_hit/total）	omp（目标： $\sum_p w_p \cdot \text{avail}(p) \cdot \text{quality}(p)$ ）
前缀稳定性	PrefixStabilityManager（SHA-256 指纹），检测变化即发出 PrefixCacheChange 事件	无对应机制——不在单一供应商上做缓存优化
工具组织	编译时注册 + 字母序固定排序 + OnceLock 缓存	运行时注册 + MCP 协议 + 插件/扩展 + 用户自定义工具
上下文压缩	追加式缝合（soft seam L_1-L_3 ），80% token 阈值触发压缩，500K 硬地板	可配置策略（handoff / context-full / OpenAI 远程），按触发原因分三类（overflow / threshold / idle）
压缩决策函数	seam_level_for_active_input() 三阶阈值（192K/384K/576K），单调递增	shouldCompact() 阈值触发 + resolveThresholdTokens() 三态配置
前馈控制	capacity_controller（失效概率模型 + 四级响应），v0.8.11 后默认关闭	isContextOverflow()（模式匹配 + 精确 token 计数），始终开启
错误处理	透明流重试（同供应商最多 3 次）+ 循环守卫（超时/步数/字节）	RetryFallbackSelector（供应商感知回退链）+ HarmonyLeakInterruption（七信号泄漏检测）
思考级别	固定 3 级（select() 关键词匹配 + 子代理判断）	统一 5 级（夹紧映射到 Nu supplier-specific params）+ promotion 链
模型注册表	一个模型（DeepSeek V4），无等价性判断需求	40+ 供应商的 canonicalIndex 等价类 + 版本比较 + 能力表
认证管理	内置在单一代理解析器中	解耦合 modelRegistry.getApiKey()，每种供应商独立认证流
扩展机制	编译时安全的有限扩展（HookSink trait + MCP 协议 + static tools）	运行时动态的丰富扩展（hooks/plugins/extensions/custom-tools，8 个子系统）
缓存经济学	cachehit : cachemiss $\approx 1 : 120$ ，前缀稳定性是最优策略	缓存因供应商而异，不统一优化——成本由模型选择（思考级别 $\rightarrow$ 费用）间接管理
跨会话记忆	cycles（检查点-重启）+ /anchor（锚点持久化）+ memory.md	session persist/restore + handoff 文档 + 技能/记忆系统
目标函数的编码位置	prefix_cache.rs（PrefixStabilityManager）、prompts.rs（七层 volatile-content-last）、compaction.rs（75%/80% 阈值 + 500K 地板）	model-registry.ts（等价性索引）、model-thinking.ts（夹紧映射）、agent-session.ts（RetryFallbackSelector + 三因压缩）

表中的每一行都对应着一个工程决策——而这个决策的方向由系统的根本目标函数决定。如果将两个系统的目标函数交换——即让 DS-TUI 追求供应商多样性，或让 omp 追求缓存命中率——两个系统的架构将与当前状态完全不一致。

这揭示了目的性原理在系统论中的核心角色：**目标函数不仅定义了"什么是对的"，还通过系统化的决策选择过程，将每个工程决策向同一方向收敛。**最终，两个最初看起来相似的系统（都是编码智能体 CLI）会演化出完全不同的内部结构——不是因为设计者的"品味不同"，而是因为目标函数作为"第一推力"在每一个决策点上强制了特定的方向。这是系统论目的性原理最直接的工程验证。

III

第三篇

综合

比较系统结构，走向统一定义与未来展望。

CHAPTER 07

第七章 比较分析与统一定义

一切理论都是模型。一切模型都是被构造出来的。Harness 工程 = 应用模型论。

在本章中，我们将说明这两句话不是修辞，而是观测到的结构不变性的陈述。

五维度系统对比矩阵

在第二章至第六章中，我们从五个独立的理论视角逐章解构了 DS-TUI 与 omp 的架构。本章不再重复前文的逐子系统对比，而是从这五个理论视角各取一个核心概念，形成一张 5×2 的**概念张量**——每个视角回答同一个问题：两个系统在该视角下的核心差异是什么？

理论视角	核心概念	DS-TUI 的关键结构	omp 的关键结构
模型论	结构 $M \models$ 理论 T 的满足关系	单体公理系（264 行 base.md 编译时嵌入），理论 T 与其唯一解释 M 之间的偏差通过容量控制器和前缀缓存稳定性监视器检测	分片公理系（skills $\times$ rules $\times$ goals $\times$ memories），理论 $T_{\text{core}} \cup T_{\text{skills}} \cup T_{\text{rules}} \cup T_{\text{goals}}$ 运行时组装， $M \models T$ 的验证分布在压缩检测、Harmony 泄漏检测、工具结果强制归一化三个独立机制中
系统论	层次性：理论塔 $T_0 \subset T_1 \subset T_2 \subset \dots$	三层塔：工具执行层（ToolRegistry） $\rightarrow$ Turn 循环层（handle_deepseek_turn()） $\rightarrow$ 元控制层（PrefixStabilityManager + CapacityController），层间通过 Engine 结构体紧耦合	五层塔：MCP 协议/工具适配层 $\rightarrow$ provider 适配层（providers/） $\rightarrow$ agent 循环层（agent-loop.ts） $\rightarrow$ 会话管理层（AgentSession） $\rightarrow$ 扩展系统层（ExtensionRunner），层间通过事件总线 and 类型化事件流解耦
工程控制论	前馈 vs 反馈控制	前馈主导：capacity_controller 在请求发送前预测失败概率，PrefixStabilityManager 在缓存失效前检测变化，should_compact() 在 token 超限前触发压缩	反馈主导：错误触发 RetryFallbackSelector 的回退链，泄漏触发 HarmonyLeakInterruption 的恢复序列，压缩触发 shouldCompact() 的阈值检测

理论视角	核心概念	DS-TUI 的关键结构	omp 的关键结构
控制论	负反馈维持稳态	多级保护回路（8 个 Guard）叠加在单层 LLM ↔ 工具循环之上，每个 Guard 有自己的决策阈值和执行策略，类似工业控制中的 override controller	双层决策循环（战术层处理工具调用，战略层处理 follow-up 消息），错误通过异常类型 + graceful degradation 分层恢复
心智模型	外部认知存储的分布策略	文件系统作为主要的外部存储媒介（~/deepseek/sessions/ + ~/.deepseek/snapshots/ + side-git），agent 通过 read_file/write_file 与自己记忆交互	结构化数据库和会话条目作为主要媒介（AgentSession 持久化 + hindsight REST API + compaction 摘要），session manager 决定什么进上下文、什么被压缩

观察这张矩阵的每一列可以发现：**五种理论视角不是平行独立的分析工具，而是从不同投影角度看到的同一结构**。模型论揭示了" T 和 M 的关系"，系统论揭示了" T 如何分层组织"，工程控制论揭示了"控制如何跨越层间边界"，控制论揭示了"什么维持了塔的稳定"，心智模型揭示了"塔的外部支撑结构是什么"。五个视角描画的是同一个物体在五个不同光照下的投影。

投影的不变性

从投影几何的角度看，五个视角对同一系统给出的描述——虽然表象不同——包含了**不变量**。在 DS-TUI 的五个视角分析中，都指向了同一个核心结构：**一个高度集中的、单一解释的、前馈优化的单体系统**，其所有子系统围绕"保护前缀缓存"这一经济约束旋转。在 omp 的五个视角中，都指向了一个**分布式的、多解释共存的、反馈优化的模块化系统**，其所有子系统围绕"支持供应商多样性"这一生态约束旋转。

这种跨视角的一致性不是偶然的。它说明：**五个理论视角不是构造性的分析框架（我们发明了它们然后套在数据上），而是在观测中自然浮现的分析维度**。每种理论视角都独立地指向了同一个组织不变量——这是跨理论收敛的证据，也指向了一个更根本的结构统一性：两个系统共享相同的抽象结构，只是在不同的约束条件下产生了不同的具体表现。

模型论视角的统一定义

一个初步定义是：Harness 工程是对分层组织的形式理论塔的设计、实现、运行和演化；各层理论通过 agent 行为得到执行，负反馈维持满足关系，正反馈推动理论演化。本章在双框架（模型论 + 系统论）的语境下将其精确化。

定义 7.1 (Harness 系统)。一个 Harness 系统 H 是一个六元组 (T, M, L, F, E, J) ，其中：

1. $T = \{T_0, T_1, \dots, T_k\}$ 是一个**分层理论塔** (layered theory tower)，每一层 T_i 是一阶语言 L_0 中的一个有限公理集合（即系统提示词的文本段落），满足 $T_0 \subseteq T_1 \subseteq \dots \subseteq T_k$ （每层包含下层所有公理并增加新公理）。 T_0 是工具执行层的理论（函数符号等于工具签名）； T_k 是元控制层的理论（包含关于其他层的自观察公理）。
2. $M = \{M_0, M_1, \dots, M_k\}$ 是 T 的**结构实现** (structural realization)，每个 M_i 是 L_0 的一个解释，使得 $M_i \models T_i$ （至少提供地）。 M_i 由 agent 的运行时行为——已执行的工具调用、已生成的文本输出——完全确定。
3. L 是**语言** (language)，即系统可用的所有符号（工具签名 + 命令名 + 事件类型 + 配置键）构成的签名集合。 L 在编译时是 L_0 （内置工具），在运行时可通过扩展机制扩充到 $L_t = L_0 \cup L_{\text{ext}}$ 。
4. $F = \{f_1, f_2, \dots, f_m\}$ 是**反馈回路集合** (feedback loop set)，每个 f_i 是一个形如 $(\text{detect} : M \not\models \varphi, \text{correct} : M \rightarrow M')$ 或 $(\text{predict} : P(\perp) > \theta, \text{prevent} : M \rightarrow M^*)$ 的修正算子。 F 包含两个子集：- $F_{\text{neg}} \subseteq F$ ：负反馈回路，维持 $M \models T$ 的不变性（压缩、容量控制、泄漏检测/恢复）- $F_{\text{pos}} \subseteq F$ ：正反馈回路，驱动 T 的演化（扩展安装、技能学习、记忆整合）
5. $E \subseteq L \times \Sigma^*$ 是**扩展机制** (extension mechanism)，一组从符号到实现的语言扩展规则。在 DS-TUI 中， E 通过 HookSink trait + ToolRegistry + mcp/McpManager 实现；在 omp 中，通过 ExtensionAPI + AgentTool 接口 + HookRunner 实现。
6. $J: \mathbb{K} \rightarrow \mathbb{R}$ 是**目标函数** (objective function)，在所有可能的结构实现类 $\mathbb{K}$ 上定义一个偏序。 J 的梯度在每一层驱动结构分化：不同 J 值导致在相同的约束集上产生不同的实现结构（第六章的核心命题）。

这个定义与上述初步定义的区别在于三个关键点：

第一，引入了明确的系统论层次性 $T_0 \subset T_1 \subset \dots \subset T_k$ 。上述初步定义只谈到了“分层组织的理论塔”，但没有将层次绑定到具体的子系统。这里，每一层 T_i 都有对应的结构 M_i ，且 M_i 的“域”完全由该层子系统的运行时行为定义。工具执行层 M_0 的论域是文件路径、文件内容、shell 输出；元控制层 M_k 的论域是缓存命中率、失败概率、上下文使用率。

第二，反馈回路 F 的上下二分。将 F 分为 F_{neg} （维持）和 F_{pos} （演化）两个子集，直接对应维纳控制论中的负反馈（稳态维持）和正反馈（结构演化）。DS-TUI 的 F_{neg} 集中在压缩和容量控制（保护前缀缓存这一经济稳态）， F_{pos} 集中在技能安装和 MCP 注册（扩大语言）。omp 的 F_{neg} 集中在泄漏检测和工具结果强制归一化（保护类型契约这一运行时稳态）， F_{pos} 集中在 hindsight 记忆整合和扩展安装（扩大认知边界）。

第三，**目标函数 J 的显式引入**。目的性原理（第六章）揭示了：架构的根本差异不是设计者的"品味"决定的，而是由 J 的不同值决定的。将 J 纳入定义，使得定义 7.1 不仅描述了一个系统是什么，还描述了它为什么是现在这样。

三个结构同构的证明

有了定义 7.1，我们可以正式地证明 DS-TUI 和 omp 在抽象结构上的同构性。"同构"在这里不是在图论或范畴论的严格意义上（两个系统的边和节点一一对应并保持关系），而是在一个更实际的工程意义上：**两个系统的六元组 (T, M, L, F, E, J) 共享相同的"模板"——差异仅在于每个分量在具体编程语言和技术栈中的实例化方式。**

证明 1：理论塔结构的同构

两个系统的理论塔 T_{DS} 和 T_{omp} 都服从同一个层次模式：

T_n : 元控制层（自观察公理）

↑ 包含

T_{n-1} : 会话稳态层（上下文管理公理）

↑ 包含

...

↑ 包含

T_1 : Turn 循环层（工具序列公理）

↑ 包含

T_0 : 工具执行层（函数符号签名公理）

在模型论中，这是**理论的保守扩张链**（chain of conservative extensions）。每一层在保持下层所有真值不变的前提下，增加新的公理约束。

DS-TUI 的实例化：

层	公理来源	结构位置	关键约束
T_0	ToolRegistry 中每个 ToolSpec 的描述文本	turn_loop.rs 的 ToolUse → tool execution 分支	工具签名的 JSON schema

层	公理来源	结构位置	关键约束
T_1	base.md 的推理公理 (PREVIEW/CHUNK/ RECURSIVE)	handle_deepseek_turn() 的 loop 体	验证义务、分解模式、路径安全 检查
T_2	CompactionConfig + CycleConfig + SeamConfig	compaction.rs 的 should_compact() + cycle_manager	token_threshold=800K, auto_floor=500K
T_3	PrefixStabilityManager 的指 纹比较 + CapacityController 的失败概率模型	turn_loop.rs 的保护回路 (Guards 5-8)	前缀 SHA-256 一致性、失败概 率阈值

omp 的实例化：

层	公理来源	结构位置	关键约束
T_0	AgentTool 接口 + MCP 协议的工 具注册	agent-loop.ts 的 executeToolCalls()	工具签名的 zod schema 验证
T_1	模式特定系统提示词 + CONTRACT / SYSTEM 规则块	runLoopBody() 的内层循环	工具使用约束、模式切换规则
T_2	compaction.ts 的 shouldCompact() + pruning.ts + 摘要模板	agent-loop.ts 的 refreshContext()	reserveTokens=16384、压缩触 发条件、摘要策略
T_3	harmony-leak.ts 的七信号融合 + coerceToolResult() 的类型 归一化	agent-loop.ts 的异常捕获 + streamAssistantResponse() 的 泄漏检测	泄漏检测的 MCSGBRT 七信号、 恢复的最大重试次数

同构映射 $\varphi : T_{DS} \rightarrow T_{omp}$ 定义为：

- $\varphi(T_{0,DS}) = T_{0,omp}$ ：工具签名描述 $\rightarrow$ AgentTool schema
- $\varphi(T_{1,DS}) = T_{1,omp}$ ：推理公理 $\rightarrow$ 模式约束 + 规则块
- $\varphi(T_{2,DS}) = T_{2,omp}$ ：压缩决策 $\rightarrow$ 压缩决策（阈值、条件、策略不同，但函数签名的结构相同：都是 $\text{token_count} \times \text{config} \rightarrow \text{decision} \times \text{action}$ ）
- $\varphi(T_{3,DS}) = T_{3,omp}$ ：元控制 $\rightarrow$ 泄漏检测 + 类型归一化

这个映射是保序的：如果 $T_{i,DS} \subseteq T_{j,DS}$ （在 DS-TUI 中 $i \leq j$ ），那么 $\varphi(T_{i,DS}) \subseteq \varphi(T_{j,DS})$ 在 omp 中也成立。保守扩张结构在映射下保持不变。

证明 2：反馈拓扑的同构

两个系统都维护了一个嵌套的四层反馈循环。我用"四重循环"来刻画这种拓扑：

外循环 4（跨会话—演化）：

DS-TUI：会话结束 → 写入 cycles/ → 下次会话加载 cycle → 行为差异 → 用户手动调整系统提示/注册新技能
omp：会话结束 → hindsight retain() → 下次会话 recall() → 提取<memories>/<mental_models> → 行为差异 → 自动或手动调整配置

模型论含义： T_{k+1} （下一次会话的理论）由 T_k （本次会话）和 M_k 的观测结果共同决定。这是**理论演化**—— T 通过跨会话反馈改变自身。

外循环 3（会话内—稳态）：

DS-TUI：上下文增长 → should_compact() 检查 → 超过 800K? → 触发 LLM 摘要压缩 → 上下文重置到安全水平
omp：上下文增长 → shouldCompact() 检查 → 超过阈值? → 触发摘要/手递/修剪 → 上下文重置

模型论含义：当 $M \not\models \varphi_{\text{context}}$ （token 超限约束）时，压缩动作 u 将 M 映射为 M' ，使得 $M' \models \varphi_{\text{context}}$ 。这是标准的**负反馈维持**。

外循环 2（Turn 内—协调）：

DS-TUI：LLM 响应 → 解析工具调用 → 执行工具 → 结果注入 → 回到循环顶部
omp：LLM 响应 → 解析工具调用 → 执行工具 → 结果注入 → 检查 follow-up/steering → 继续或退出

模型论含义：每个 Turn 是 M 对 T 的**单次赋值**（一次 assignment）。工具调用是函数符号的应用；工具结果是对谓词的赋值；后续推理是在新赋值下的演绎。

内循环 1（字符级—流式稳定）：

DS-TUI：流式事件 → tokio::select! → 超时/Cancel/StreamEnd → 累加 ContentBlock(Delta) → 最终解析工具调用
omp：流式事件 → streamAssistantResponse() → 中途泄漏检测 → 触发恢复机制 → 继续或重新请求

模型论含义：内循环 1 维持了**赋值过程的稳定性**——在赋值完成之前（工具调用 JSON 还没解析完），控制流不能中断。这相当于保护“正在构造的 M ”不被外部干扰破坏。

四重循环的嵌套关系在两个系统中是相同的：

循环 4（跨会话） $\supset$ 循环 3（会话） $\supset$ 循环 2（Turn） $\supset$ 循环 1（流式 I/O）

外层的控制决策改变内层运行的**参数**（压缩阈值、模型选择、思考级别），但内层运行产生的结果**约束**外层决策（失败率数据、token 消耗统计、缓存命中率）。这种双向约束在系统论中称为**递阶控制的双向耦合**——上层设定边界条件，下层产生反馈数据。

两个系统的反馈拓扑在抽象层面上是完全同构的。差异在于具体的实现机制：- DS-TUI 将四重循环实现在单个函数 `handle_deepseek_turn()` 中，通过 8 个 Guard 和嵌套 loop 显式编码。- omp 将四重循环分布在 `agent-loop.ts`（循环 2）、`compaction.ts`（循环 3）、`hindsight/backend.ts`（循环 4）和 `streamAssistantResponse()`（循环 1）中，通过事件流和异步回调隐式连接。

这种差异不是同构的破坏——**函数式同构不要求实现方式的同构**。递归函数可以编码为递归（DS-TUI 的方式），也可以用迭代+显式栈编码（omp 的方式）；两种编码在计算函数映射上是等价的。

证明 3：目标函数驱动分化的同构

第六章证明了目的性原理：目标函数 J 的差异驱动了两个系统的架构分化。这里我要进一步证明：**两个系统在"目标函数 $\rightarrow$ 结构分化"的映射函数上是同构的。**

定义映射 $\psi: J_{DS} \times \text{struct_DS} \rightarrow \text{struct}'_{DS}$ 。在 DS-TUI 中， J_{DS} = "最大化缓存命中率 / 总请求数"。这个 J 驱动了以下结构分化：

$$J_{DS} = \max \frac{\text{hit}}{\text{total}}$$

→ 前缀必须稳定（PrefixStabilityManager）

→ 提示词必须编译时嵌入（`include_str!("base.md")`）

→ 工具排序必须不变（字母序 + OnceLock）

→ 易变内容必须移到用户消息中（volatile-content-last 策略）

→ 压缩必须在缓存损失可接受时才能进行（500K 硬地板 + 容控制器预测偏移）

在 omp 中， J_{omp} = "最大化供应商加权可用性 $\sum_p w_p \cdot \text{avail}(p) \cdot \text{quality}(p)$ "。结构分化路径为：

$$J_{omp} = \sum_p w_p \cdot \text{avail}(p) \cdot \text{quality}(p)$$

→ 提示词必须运行时组装（Handlebars 模板 + skills/rules/goals/memories）

→ 工具注册必须动态（MCP 协议 + 扩展/插件/自定义 tool）

→ 错误处理必须多供应商回退（RetryFallbackSelector 的回退链）

→ 认证必须解耦（`modelRegistry.getApiKey()` 每种供应商独立流）

→ 思想级别映射必须可配置（夹紧映射 + promotion 链）

两种分化的**模式**是相同的：从 J 的解析开始，沿着推理链逐步推出具体工程决策。这个推理链在两种情况下都是**单调的**：给定 J 的解析，推导出的结构子集是确定的——没有设计者的"自由意志"在中间插入替代分支。 J 的差异导致结构集合的差异，但 $J \rightarrow \text{structure}$ 的映射函数 ψ 在两个系统中是相同的。

正式地：存在一个映射函数 $\psi_{\text{meta}} : J \rightarrow P(\text{struct})$ （目标函数到结构集合的幂集映射），使得 $\psi_{\text{meta}}(J_{\text{DS}}) = \{\text{DS-TUI 的所有架构决策}\}$, $\psi_{\text{meta}}(J_{\text{omp}}) = \{\text{omp 的所有架构决策}\}$, 且 ψ_{meta} 在两个实例中是同样的函数——只是输入参数 J 不同。

这三个同构证明共同指向一个结论：**DS-TUI 和 omp 不是两个不同类型的系统，而是同一个抽象结构 (T, M, L, F, E, J) 在两种不同的约束、语言、目标函数下的具体实例。** 这个抽象结构——即"Harness 工程"的定义——在两个项目中以不同的实现材料呈现，但遵循相同的组织法则。

设计空间的四个正交维度

有了统一定义和同构证明，我们可以将四个设计维度从"工程选择"升格为"Harness 系统分类学"。

维度 1：单一模型 vs 多模型

定义： Harness 系统中的"模型数"是指系统中独立维护的理论-结构对 (T_i, M_i) 的数量。单一模型系统只有一个这样的对（即只有一个主 agent 循环、一个主提示词）。多模型系统有多个对，它们通过通信协议（共享文件系统、MCP 协议、事件总线）协调。

在定义 7.1 的术语中：单一模型系统 $|\text{Index}(T)| = 1$ （虽然 T 有层次，但所有的 T_i 被同一个 agent 解释为 M ）。多模型系统 $|\text{Index}(T)| = n$, 每个 (T_i, M_i) 是自主对（自主 agent）。

实例： - DS-TUI 和 omp 的主循环都是单一模型系统。 - omp 的 swarm-extension 是多模型系统。 - DS-TUI 的 subagent.rs 和 task_manager 是受限的多模型（子代理共享主代理的 T 但有自己的 M ）。

核心权衡： - 单一模型： T 的一致性有编译时保证（所有公理在同一空间中），但扩展到异质任务时需要为不同子任务"伪造"公理。 - 多模型： $|\text{Index}(T)| = n$ 时， n 个理论之间需要兼容性条件—— T_i 的输出必须被 T_j 的解释函数理解。这个条件等价于存在一个元语言 L_{meta} ，使得每个 T_i 在该语言中的断言可以被所有其他 agent 的 M_j 解释。在 swarm-extension 中， L_{meta} 是文件系统；在 MCP 中， L_{meta} 是协议规范。

维度 2：静态模型 vs 动态模型

定义： 理论 T 在会话周期内是否会被修改。静态模型中， T 在会话启动后不变——所有公理在编译时固定或在启动时一次性确定。动态模型中， T 在运行时被主动修改——扩展注入新公理，compaction 替换旧公理，handoff 文档添加新公理。

在定义 7.1 的术语中：静态模型 $T(t) = T(0)$ 对所有会话中的 t 成立。动态模型 $T(t+1) = \text{modify}(T(t), \text{obs})$ 依赖于运行时观测。

注意： M （agent 行为）的动态演化不归属于这个维度——即使最静态的系统， M 也在每个 Turn 发生变化。这里的"动态"是 T 层面的动态，不是 M 层面。

实例：- DS-TUI 倾向于静态：系统提示词在编译时固定，PrefixStabilityManager 的存在本身就是对"静态性被高度重视"的证据。- omp 倾向于动态：generateHandoff() 在 /handoff 时实时生成新公理，hindsight 在跨会话中积累扩展公理。

核心权衡：- 静态模型最大化前缀缓存效率（DeepSeek 的 128-token 粒度缓存，90% 成本折扣），但限制了系统对运行时变化的响应能力。- 动态模型最大化运行时自适应能力（每轮 turn 的理论都可调整），但代价是缓存频繁失效。

维度 3：紧耦合 vs 松耦合

定义：理论塔 $T_0 \subset T_1 \subset \dots$ 中相邻层的通信方式。紧耦合系统中，层间通信是同步的、过程调用的（fn handle_deepseek_turn() 直接调用 fn execute_tool()）。松耦合系统中，层间通信是异步的、事件驱动的（agentLoop() push AgentEvent，各子系统通过事件监听接收）。

在定义 7.1 的术语中：紧耦合系统中，层间关系通过函数调用图中的调用边编码；松耦合系统中，层间关系通过事件类型签名中的通信协议编码。

实例：- DS-TUI 的 turn_loop.rs 是紧耦合的典范：8 个 Guard + 1 个 loop 块，所有层在同一函数栈帧中。- omp 的 agent-loop.ts + AgentSession + ExtensionRunner + hindsight 是松耦合的：通过 AgentEvent 流和事件订阅连接。

核心权衡：- 紧耦合：控制精确，状态一致性好（层间没有并发竞争），易于推理和调试。代价是并发受限——一个长时间运行的 shell 命令阻塞整个循环。- 松耦合：可扩展性好（新组件监听事件流即可接入），可并行化（工具执行与状态管理可以分离）。代价是全局状态不确定性增加——事件流的非确定性顺序可能导致偶发的可见性问题。

维度 4：单体 vs 模块化

定义： (T, M, L, F, E, J) 中组件的绑定时间。单体系统中，所有组件在编译时绑定： T 通过 include_str!() 嵌入， L 通过 ToolRegistry 在编译时确定， E 是有限的外部钩子。模块化系统中，组件在运行时绑定： T 通过 Handlebars 模板运行时组装， L 通过 ExtensionRunner 在运行时加载， E 是具有完整生命周期的一等公民。

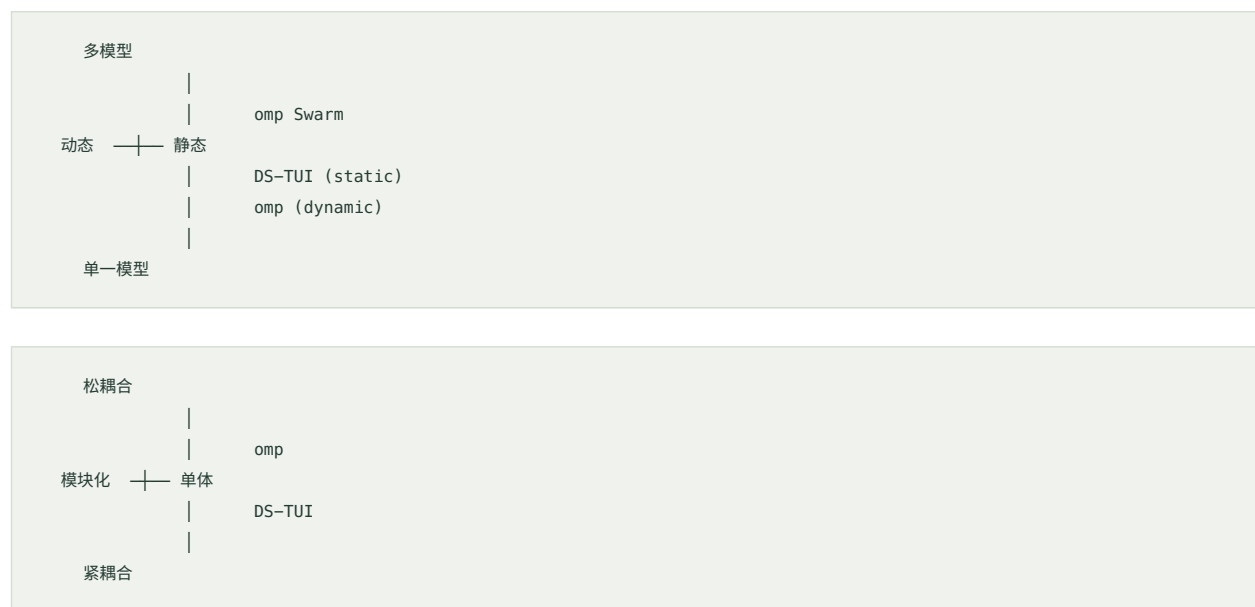
注意：紧耦合/松耦合是"运行时交互的刚性"，单体/模块化是"开发时依赖与发布单元的独立性"。一个系统可以是紧耦合+模块化（每个模块在运行时通过过程调用同步交互，但模块本身可以独立开发和发布），也可以是松耦合+单体（所有组件在同一可执行文件中，但通过事件流异步通信）。

实例：- DS-TUI 是单体（14 个 workspace crate 但链接为单一二进制，工具注册编译时固定），但有模块化萌芽（MCP、Skills 在运行时加载）。- omp 是模块化（packages/ 下的独立包，可独立开发、测试和发布；扩展系统支持运行时加载）。

核心权衡：- 单体：编译器在链接时验证所有接口（trait bound、类型签名、生命周期约束），运行时不存在"版本冲突"。代价是**发布周期受限**——任何修改都需要完整编译。- 模块化：接口在运行时通过类型检查和协议约束验证（`coerceToolResult()` 的存在就是证据），扩展可以独立发布。代价是**版本相容性管理**——扩展 API 的相容性是运行时的常态问题。

四个维度的组合空间

四个正交维度定义了 Harness 设计空间的 16 种可能组合。两个系统占据了不同的区域：



DS-TUI 在 (单一模型, 静态, 紧耦合, 单体) 的象限中；omp 的主循环在 (单一模型, 动态, 松耦合, 模块化) 中；omp 的 swarm-extension 在 (多模型, 动态, 松耦合, 模块化) 中。每个象限对应着一组不同的约束条件和经济原理。

从比较中提取可转移的洞见

本章的全部论证最终归结为一个实用问题：**两个项目能从对方学到什么？**

DS-TUI 可以学 omp 什么

1. 工具执行的解耦与可插拔性。 DS-TUI 的工具执行深度内嵌在 `turn_loop.rs` 的函数调用树中——没有工具 `middleware`、没有工具拦截器、没有工具执行的插件点。omp 的 `beforeToolCall` / `afterToolCall` hook 体系、`tool-wrapper.ts` 中的拦截机制、`coerceToolResult()` 的合约边界守卫，都是不需要修改核心循环就改变工具执行行为的方式。对于 DS-TUI，即使不引入 omp 级别的动态性（那会破坏前缀缓存），为关键工具（`exec_shell`、`write_file`、`edit_file`）添加类型化的 `pre/post hook`——不是全局的 `HookSink` 而是针对具体工具签名的钩子——也能在不牺牲单体安全性的前提下增强可观测性。

2. 供应商多样性作为风险管理。 DS-TUI 目前绑死在 DeepSeek V4 上。即使不追求 omp 的 40+ 供应商广度，在 `ConfigToml` 中添加一个“备用模型”字段——当 DeepSeek V4 的 API 故障率超过阈值（由 `capacity_controller` 提供数据）时自动切换——就能在单体架构中引入基本的容灾能力。这个切换不需要破坏前缀缓存（切换后的提示词前缀当然不同，但故障时的“降级精度”远好过“完全不可用”）。

3. Compaction 的多策略选择。 DS-TUI 的压缩是单路径的：一条提示模板、一个 LLM 摘要调用。omp 的 6 套提示模板、本地修剪 + LLM 摘要 + OpenAI 远程压缩的三重选择提供了**压缩的粒度控制**。DS-TUI 的 `CACHE_ALIGNED_SUMMARY_CONTEXT_BUDGET_PERCENT = 85` 和 `MINIMUM_AUTO_COMPACTIION_TOKENS = 500_000` 已经展示了两策略的萌芽——但可以进一步将“短摘要”（用于轻度压缩）和“完整摘要”（用于重度压缩）分离为不同的提示路径。

omp 可以学 DS-TUI 什么

1. 前馈控制的系统化。 omp 的反馈回路主要是反应式的——压缩阈值到达后才压缩，泄漏发生后才恢复。DS-TUI 的 `capacity_controller` 在请求发送前预测失败概率并选择干预动作，这代表了**从反应式到预测式的范式转换**。omp 已经拥有必要的数据基础（`telemetry.ts` 记录每个工具调用的耗时、成功率、token 消耗）——将这些数据喂入一个简单的概率模型（不需要 DS-TUI 级别的复杂精度），就能在失败发生之前预调度干预。

2. 前缀缓存意识。 omp 是多供应商的，但前缀缓存的经济学在单一供应商会话中是普遍适用的——即使在 omp 的上下文中，当一个用户在一个会话中反复使用同一供应商时，缓存命中率仍然影响成本和质量。omp 可以为每个供应商的会话维护一个 `PrefixFingerprint` 式的简单缓存稳定性指标——不要求 DS-TUI 的 SHA-256 完美度（那会引入对提示词格式的硬依赖），只要在压缩决策中记录“当前提示词相对于上一轮是否发生变化”，就能为压缩时机选择增加一个新的有用信号。

3. 压缩的结算式硬边界。 DS-TUI 的 `MINIMUM_AUTO_COMPACTIION_TOKENS = 500_000` 是一个清晰的**结算式边界**（clearing boundary）——它量化了“什么时候压缩的经济收益（释放的上下文预算）超过了经济成本（重填前缀缓存）”。omp 的 `reserveTokens = 16384` 相对保守，且缺乏对不同供应商缓存经济学的差

异化处理。根据供应商的不同缓存折扣比率（DeepSeek 的 ~90% vs OpenAI 的 ~50%），为每个供应商计算动态的"压缩最低阈值"，可以将这个洞见从 DS-TUI 直接迁移到 omp 的 `resolveThresholdTokens()` 中。

对称的洞见

两个项目可以相互学习的最对称的洞见是：**负反馈应该在正功能之前建立**（反馈机制参见第六章“反馈控制的模型论解释”）。DS-TUI 的前缀缓存优化（一种正功能优化）如果没有 `PrefixStabilityManager` 的负反馈监控，它的经济效果会大折扣——每次前缀变化导致的缓存损失是无声的、不可见的。omp 的多供应商支持（一种正功能扩展）如果没有 `coerceToolResult()` 的合约边界守卫和 `HarmonyLeakInterruption` 的类型保护，供应商接口的异质性会在运行时产生不可预测的故障。两个系统都在"首先建立负反馈，然后增加正面能力"这一法则则是正确的。

本章是第三篇综合的开篇。通过统一定义、三个同构证明和四个正交维度的分类，我们建立了 Harness 工程的形式化基础。第八章将在此基础上展望：从当前的 (T, M, L, F, E, J) 结构出发，Harness 工程的下一个演化阶段是什么。

CHAPTER 08

第八章 展望：模型论视角的 Harness 工程未来

"一切理论都是模型。我们选择模型时，依据的不是真理性，而是对问题的最佳适应。"

—— 本章的基调：模型论不是对 Harness 系统的终极解释，而是我们迄今为止找到的最好的分析工具。

"今天，我们站在理论选择的岔路口。明天，我们可能选择不同的公理集。但只要保持一致的框架，我们就能持续理解和改进我们的系统。"

前七章构建了一个自洽的分析框架：从模型论的形式语言出发（第一章），确立了"系统提示词即理论 T 、LLM 行为即结构 M 、满足关系 $M \models T$ 即行为正确性"的映射（第二章），然后通过系统论的四大原理——整体性、层次性、开放性、目的性——将这个框架应用于 Harness 工程的全方位分析（第三章至第六章），最后通过两个工业级系统的比较验证了框架的解释力（第七章）。

但分析和理解不是终点。Harness 工程是一块活着的、正在快速演进的研究领域。本章从这个框架出发，推测五个关键方向：自修正理论、Agent 生态、预测式控制、高阶自观察、以及对实践者的启示。这些推测不是预言——它们是"如果模型论框架成立，那么合理的延伸方向是什么"。

不同方向的成熟度不同：自修正理论已有明确萌芽（代码级原型），高阶自观察仍处于设计阶段。我们按风险递增的顺序组织——从最接近实践的变革到最远期的愿景。

8.1 自修正理论：从静态提示词到运行时自我修改

当前的系统提示词（理论 T ）是人类编写、编译时固定的。模型（ M ）可以偏离 T ，但 T 本身只有在开发者发布新版本时才会改变。这是典型的静态公理化—— T 像一部民法典，详细规定了行为准则，但修改法典需要立法程序（代码变更 + 重新部署）。

萌芽已经出现于三个代码级原型。

第一个萌芽是 DeepSeek TUI 的 `auto_reasoning.rs`。它根据用户消息中的关键词（如 debug、错误、崩溃）自动选择思考深度（`ReasoningEffort::Max / Low / High`）。这在等价于**每次 turn 前动态调整 T 的一个参数**——思考努力级别是 T 中一个预定义的参数符号的实例化。这个符号的论域是 $\{Low, High, Max\}$ ，自动推理器根据关键词匹配选择当前实例化的论域元素。这不是对 T 的修改——符号集 C 在编译时固定——但它开创了一个先例：**系统可以根据感知到的上下文，从 T 中激活不同的公理子集。**

第二个萌芽是 DeepSeek TUI 的 `capacity_controller`。它的 `select_intervention()` 方法根据失败概率选择一个干预动作——这个干预动作改变了后续 turn 的行为约束。如果 `CapacityController` 检测到 API 调用的失败概率超过了阈值，它会选择 "简化输出" 或 "减少工具调用" 等干预。这等效于运行时对 T 施加临时约束公理。关键区别在于：`auto_reasoning` 是从预定义符号集中选择，而 `capacity_controller` 是**生成新的临时约束**——它往 T 中添加了原本不存在的公理（"在接下来的 3 个 turn 中，每轮最多使用 2 个工具"）。这是自修正的第一步，尽管约束是临时的且可撤销的。

第三个萌芽是 Oh My Pi 的 `harmony-leak.ts`。当 `HarmonyLeakInterruption` 检测到协议泄漏（污染的工具输出），`agent-loop.ts` 捕获该异常并在下一轮 LLM 请求中注入 "上一次工具调用的输出被污染了" 的消息。与 `capacity_controller` 不同，这没有修改 T ，而是在**解释函数 I 的层面改变了 agent 对历史输出的解读方式**。系统在等效地告诉模型："你看到的这个结构 M 的部分内容不可靠，请在推理时将其标记为不可信。" 这接近于修改 T 中关于证据保真度的公理。

8.1.1 自修正的三个等级

从这些萌芽出发，自修正理论可以划分为三个递进等级。

一级：参数化激活。 T 中包含参数符号的占位符——系统在 turn 开始时选择这些参数的实例化值。DS-TUI 的思考深度选择已实现这一级。Oh My Pi 的 `shouldCompact()` 决策也可视为一个布尔参数（"当前上下文是否应压缩"）的运行时判定。这一级的核心特征是：**符号集 C 固定，但每个符号的赋值在运行时可变**。在模型论中，这对应于一阶句法中的带参公式（parameterized formula），其中参数是 T 中预留的自由变量。

二级：公理蒸馏。系统从运行时统计数据中提炼出新公理，并注入 T 。一个合理的扩展是：让 `PrefixStabilityManager` 的检查结果（"缓存命中率从 95% 降到 40%"）不仅发送事件到状态栏，而且自动触发提示词重组——系统分析什么内容导致了前缀变化，自动将已经验证为稳定的部分（如工具注册表）与易变的部分（如项目配置信息）在 T 中分离开。这等效于从 M 的经验中学习 T 的优化结构——是一种**归纳推理**：从 $\{M_1, M_2, \dots, M_n\}$ 的统计模式中提炼出使 $\text{Mod}(T)$ 更稳定的公理重组。Oh My Pi 的 `handoff-generation-pipeline.zh.md` 展现了类似的概念雏形：在会话间创建结构化交接文档，本质上是将一次会话中形成的"隐含公理"萃取为显式的、可供下一段会话使用的知识。

三级：理论空间搜索。 当系统在多轮自修正后仍然无法维持 $M \models T$ 时，它需要搜索一个完全不同的理论 T' 。这是最激进的形态——系统在 T 空间中执行搜索，类似于遗传算法在基因组空间中搜索——保留 T 的最优公理，变异或重组次优公理，淘汰不一致的或产生不可接受模型的 T' 。这要求系统具备**元理论能力**：不仅知道自己正在使用哪个 T ，还能评估备选 T' 的预测力和一致性。目前没有任何 Harness 系统实现这一级——但它是指引方向的地平线。

8.1.2 自修正的核心约束：理论一致性

无论哪个等级，自修正必须遵守一条铁律： **T 必须在每一步都保持可满足**。如果 T 在修改后变成了不一致的（包含矛盾公理），所有 $M \models T$ 都是空集——系统在模型论意义上“不存在”了，行为完全不可预测。

形式上： $T_n \rightarrow T_{n+1}$ 的每次跃迁必须满足 $\text{Consistent}(T_{n+1})$ 。这要求系统在每次自修正前执行一致性预检——相当于一个**元级守卫**。DS-TUI 的 `LayerOrder` 在编译时提供了这样的预检：由于分层且优先级覆盖的语义是固定的，添加新公理到某一层不会制造层间矛盾（但可能制造层内矛盾，这是未来的风险评估点）。Oh My Pi 的模块化公理化则依赖命名空间隔离来避免矛盾——不同技能的工具定义不重叠，因此不会产生涉及同一谓词符号的冲突公式。两种策略都不完美——但在运行时一致性检查成为标准工程实践之前，这是最接近的保护。

模型论的视角揭示了更深层的约束：**自修正不仅要保持一致性，还要保持理论塔的保守扩展性质**。高层 T 在低层 T 上扩展时，不应在低层语言中推导出原本不存在的句子。如果低层提示词规定“工具调用必须包含参数 `path`”，高层自修正添加的规则不应该让某些 `path` 参数变为可选——否则就改变了低层理论的语义契约，两个之前等效的模型会突然不等效。

8.2 Agent 生态：从静态编排到动态自组织

当前的 Agent 编排是**静态声明式**的。用户在 YAML 文件中预先定义了 agent 的角色、依赖关系和执行模式（`pipeline / sequential / parallel`）。Oh My Pi 的 `swarm-extension` 的 YAML 配置是最典型的例子：

```
agents:
  - id: reader
    model: deepseek-chat
    system_prompt: "你是文件阅读专家"
    waits_for: []
    reports_to: analyst
  - id: analyst
    model: gpt-4o
    system_prompt: "你是分析专家"
    waits_for: [reader]
```

这种配置在运行前就完全固定了 agent 集合、通信拓扑和执行顺序。在模型论的语言中：**Agent 生态的理论 T_{swarm} 的符号集和公理在编译时完全固定**。每个 agent 是理论塔的一个独立理论 T_i ，它们之间通过 `waits_for` 关系形成偏序，通过 `reports_to` 关系形成分层——但任何 agent 的创建、销毁或重新配置都需要开发者修改配置文件。

8.2.1 动态编排的萌芽

Oh My Pi 的 `swarm-extension` 已经在三个层面上提供了动态编排的萌芽。

dag.ts 的拓扑排序引擎是一个关键支点。它的 `buildExecutionWaves()` 接受用户定义的 `waits_for` / `reports_to` 依赖，自动转换为执行波次。如果这个引擎不仅能处理用户定义的显式依赖，还能**自动发现 agent 之间的隐式依赖**——比如 Agent A 总是等待 Agent B 写入 `signals/finder_out.txt`，引擎自动添加一条 `Agent_A waits_for Agent_B` 的边——就获得了动态依赖发现能力。这等价于在运行时检测到两个理论 T_i 和 T_j 之间的逻辑依赖（ T_i 的推论需要 T_j 提供的某个句子作为前提），并在理论塔的层间自动添加连接桥。

pipeline.ts 的 target_count 模式是另一个关键机制。在 pipeline 模式下，同一组 agent 被重复执行 N 次，每次建立在上一次的结果之上。这本质上是一个**迭代改进循环**——每次迭代的结构 M_k 在 T 的指导下产生输出，下一次迭代的 M_{k+1} 使用前一轮的 M_k 的输出作为 T 的附加公理。如果把这个循环的逻辑从“固定的 N 次”改为“持续执行直到收敛条件满足”——比如直到两个相邻迭代的输出差异低于某个阈值——就获得了**自终止的迭代改进**。从模型论看，这对应着一系列理论扩张：

$$T \rightarrow T \cup \text{Sentence}(M_1) \rightarrow T \cup \text{Sentence}(M_1) \cup \text{Sentence}(M_2) \rightarrow \dots$$

当 $\text{Sentence}(M_k) \approx \text{Sentence}(M_{k-1})$ 时，Model 变得稳定。这就是**收敛**。

8.2.2 元 Agent 与自组织

更激进的方向是引入**元 agent**——一个独立的模型实例，其语言和理论位于“超结构”中，负责监控所有子 agent 的运行状态并动态调整 agent 生态。

元 agent 的核心能力是**理论比较**——它需要判断两个子 agent 的理论 T_i 和 T_j 之间的关系：是相容的（可以合并）、是互补的（需要定义通信协议）、还是矛盾的（需要仲裁）。在模型论中，这对应着判断 $\text{Mod}(T_i) \cap \text{Mod}(T_j)$ 是否为空——两个理论共享的模型集合。这个判断在经典模型论中是不可判定的（Church 定理），但在 LLM 系统的“软模型论”中，可以通过语义嵌入空间的相似度来近似——如果 T_i 和 T_j 的输出在嵌入空间中的距离小于一个阈值，它们很可能在解决同一类问题。

元 agent 可以执行的操作包括：

1. **Agent 创建**：当检测到当前 agent 集合无法覆盖某个子系统或子问题时，元 agent 生成新的 T_{new} ，包含该子系统的工具公理和约束公理，并实例化一个新的模型 M_{new} 。
2. **Agent 销毁**：当某个 agent 持续产生低质量输出（即 $M_i \not\models T_i$ 的偏离超过容忍度），或者该 agent 的任务已经完成，元 agent 将其从 Agent 生态中移除。
3. **Agent 重配置**：当发现某个 T_i 的约束对当前任务过于严格或过于宽松，元 agent 修改 T_i 的公理——对应于前述的自修正。

8.2.3 钱学森综合集成研讨厅

钱学森在上世纪 90 年代提出的**综合集成研讨厅**（Hall for Workshop of Metasynthetic Engineering）理念，为 Agent 生态的远期形态提供了一个引人深思的参考模型。

研讨厅的核心思想是：将不同领域的专家——人类专家 + 机器模型——围坐在"会议桌"旁，每个专家贡献自己的领域模型，通过不断的知识交互（讨论、辩论、综合）达到对复杂问题的共识。钱学森特别强调，这个过程中**没有一个人或机器拥有全局知识**——共识是从局部知识的迭代交互中涌现的。

映射到 Agent 生态：

- **每个 Agent 是一个领域专家**，其系统提示词 T_i 编码了一个特定领域的知识公理集和行为约束
- **Agent 之间的通信**（reports_to、waits_for、共享文件、MCP 协议桥）对应于专家之间的论据交换
- **结果聚合**不是简单的投票或求和——而是从多组输出中构造一个"综合"的结构 $M_{\text{composite}}$ ，它不应该与任何子 agent 的 T_i 产生矛盾（一致性约束）
- **研讨厅的迭代性质**——一轮讨论产生新的论据，新论据修改下一轮的知识基础——正好对应于 pipeline 的迭代改进循环，在理论上等价于一个逐轮逼近的固定点过程

这个类比揭示了当前 swarm-extension 的核心限制：**没有元级协调机制**来管理专家之间的"讨论"（如谁在什么时间发言、当前议题是什么、何时达成共识）。元 agent 就是研讨厅的主持人——它不贡献领域知识，但负责议程管理、知识流动、收敛判断。在 Oh My Pi 的 `executor.ts` 中，结果聚合是简单的级联传递——每个 agent 的输出做后一个 agent 的输入。真正的研讨厅需要更复杂的交互协议：多轮辩论、条件性询问、角色切换。

从理论上，Agent 生态的终极形态是一个**动态理论图**——节点是各自独立但可交互的理论 T_i ，边是通信协议（共享语言）和依赖关系（偏序），元 agent 在这个图上执行拓扑重排和节点增删操作，以维持整体结构的合理性和多目标优化——而这在系统论中，对应于一个**自组织的层次结构**。

8.3 预测式控制：从单步预测到全局轨迹

当前 Harness 系统的控制论主要是**反应式**的：- 上下文膨胀后才压缩（Oh My Pi 的 `shouldCompact()` 触发的负反馈）- 工具执行失败后才调整策略（DS-TUI 的 `capacity_controller` 仅在失效概率模型触发后选择干预）- Harmony 泄漏进入工具参数后才检测并截断（Oh My Pi 的 `HarmonyLeakInterruption`）

这些反应式机制是有用的。但它们有一个根本限制：**它们只能在问题已经发生或即将发生时才采取行动**。类比于自动驾驶，反应式控制相当于看到障碍物后才刹车——对于已知的场景，足以避免事故；但对于复杂的场景，它缺少预测和规划能力。

8.3.1 现有的预测雏形

系统中的非反应式（预测式）控制已经存在，但数量有限且范围狭窄。

DS-TUI 的 `capacity_controller` 是现有最成熟的预测式控制机制。它构建了一个简单的失效概率模型——基于最近请求的失败率、响应延迟、重试次数——预测下一次 API 调用可能失败的概率，然后在概率超过阈值时选择干预动作。这是**单步预测**：它只预测当前 turn 的失败概率，不预测整个会话的未来路径。

DS-TUI 的 `PrefixStabilityManager` 同样是单步预测。它通过计算相邻 turn 之间的前缀 SHA-256 哈希差异，预测当前前缀相对于上一个前缀的稳定性——如果哈希变化超过阈值，预测下一个 turn 的缓存命中率会下降。但这也是单步的：它只比较相邻的两个 turn，不建立前缀变化的长期趋势。

Oh My Pi 的 `shouldCompact()` 是另一种单步预测。它基于当前 token 计数和上下文窗口大小，预测超过窗口容量的时间点——但预测粒度是"下一个 turn 是否会溢出"的布尔决策，而不是"会话在什么时候以什么速率达到什么级别的压缩需求"。

8.3.2 全局预测式控制的架构

全局预测式控制的核心是：系统不仅预测下一个事件，而且**构建一个会话轨迹模型**——将当前会话映射到一个抽象的"状态空间"中，在这个状态空间中预测未来的状态演化路径。

DS-TUI 的 `working_set.rs` (57.1KB) 已经为这类预测提供了数据基础。它追踪 agent 访问了哪些文件、在哪些操作上花费了时间、哪些文件是被读取而非写入的。如果将这些追踪数据与会话的 token 消耗速率、任务复杂度的语义度量、历史会话数据交叉分析，系统可以构建一个**会话效率模型**。这个模型可以回答以下问题：

- 基于当前 token 消耗速率和任务复杂度，本次会话需要在什么时候压缩？答案不是"token 到阈值时"——而是"以这个速率，在 300 个 turn 后，即使压缩也只能恢复到窗口的 60%"，系统可以提前规划更激进的压缩策略。

- 基于模型的选择和任务类型，当前思考级别是最合适的吗？如果分析历史数据发现，"文件重构"类任务使用 `ReasoningEffort::High` 时准确率提升 12%，但成本增加 300%——系统可以作出成本与质量的权衡决策。
- 基于过去 1000 次类似会话，当前任务的预期成本是多少？用户可以提前知道"这类任务通常需要 50K token、30 轮 turn、耗费 \\$0.03"——而不是在会话结束时才知道费用。

在模型论语境中，全局预测式控制对应着**为会话过程建立一个更高阶的理论** T_{pred} 。 T_{pred} 以会话的初始状态（用户查询、系统配置、可用工具集）为输入，输出会话在时间 t 的预期模型状态 $M(t)$ 的分布。这不是在系统提示词的层次上工作——而是在元系统层次上： T_{pred} 描述了"系统在提示词 T 指导下处理查询 Q 时，通常产生的结构序列 $\{M_1, M_2, \dots, M_n\}$ 会是什么样子"。

8.3.3 预测式控制的闭环集成

全局预测不仅可以提前通知用户或开发者。它的真正价值在于**闭环**：预测结果直接驱动控制决策。

如果一个 10 轮前的预测显示"在第 200 轮附近会出现严重的上下文退化"，系统可以在第 150 轮主动开始"软压缩"——比如逐步降低工具输出的详细程度（通过修改解释函数 I 的详细程度参数），而不是等到第 200 轮才触发 `shouldCompact()` 的紧急压缩。类比于现代操作系统中的内存管理：系统不是在内存耗尽时才触发 OOM killer——它通过页面置换算法预测内存压力，提前将不常用页交换到磁盘。

在控制论的语言中：**前馈控制取代了反应式反馈**。反馈控制的公式是 $u(t) = K \cdot e(t)$ ——控制量基于当前误差 $e(t)$ ；前馈控制的公式是 $u(t) = K_{\text{ff}} \cdot r(t) + K_{\text{fb}} \cdot e(t)$ ——控制量基于预设的参考轨迹 $r(t)$ 加上误差修正。如果 Harness 系统能在会话开始时设定一个 token 消耗的参考轨迹（"理想情况下，前 100 轮应该消耗 30K token，全部在缓存窗口内"），然后通过前馈补偿来沿着这个轨迹行驶，系统的可预测性和稳定性将显著提升。

8.4 高阶自观察：观察的递归阶梯

二阶控制论的关键洞察（Heinz von Foerster）是：**观察者也是系统的一部分。系统对自身的认知不是外部的，而是系统行为本身的构成要素**。系统观察自己时，它观察的结果会改变它后续的行为。

将自观察按层次分解，我们得到一条阶梯：

- **一阶观察**（系统观察世界）：系统调用 `read_file` 后读取文件内容，感知外部环境。这就是 Harness 系统每次工具调用的执行——解释函数 I 将符号映射到外部操作。
- **二阶观察**（系统观察自身行为）：系统调用 `PrefixStabilityManager` 检测前缀是否变化，感知自身提示词的稳定性。这是当前系统已实现的最强自观察层次。

- **三阶观察**（系统观察自己的观察过程）：系统不只是检测前缀变化，还评估检测本身的开销、频率、准确性——"我的 PrefixStabilityManager 每轮运行消耗了 50ms 的 CPU 时间，是否有必要每轮都运行？"
- **四阶观察**（系统优化自己的观察策略）：系统不只是调整观察频率，还重新设计观察策略——"我检测前缀变化的方式（SHA-256 全程比较）对检测缓存无效类的变化没有意义，我应该改用结构化比较——只比较工具注册表的排序指纹，忽略用户输入导致的自然变化。"

8.4.1 已实现的二阶观察

当前 Harness 系统的二阶观察集中在三个维度的自观测：

前缀稳定性自观察。 DS-TUI 的 PrefixStabilityManager 通过跨 turn 比较前缀指纹的变化来检测提示词的"漂移程度"。如果缓存命中率下降，它可以追溯到特定的提示词段落的变更。这是对自身理论观察——PrefixStabilityManager 实际上在问："我的理论 T 变了吗？"

失效概率自观察。 DS-TUI 的 capacity_controller 追踪 API 调用的失败率、延迟分布和重试统计，构建失效概率模型。它问自己："我最近表现得好吗？"

协议异常自观察。 Oh My Pi 的 HarmonyLeakInterruption 通过七信号融合（标记词频率 M 、通道词 C 、Glitch Token 指令 G 、文本格式断裂 S 、字节反转 B 、低概率区域 R 、时间序列 T ）检测模型是否产生了协议泄漏。它问自己："我的输出中是否有不属于我的东西？"

这三类自观察覆盖了不同的观察对象：**理论维度的变化**（前缀稳定性）、**行为维度的表现**（失效概率）、**输出维度的异常**（协议泄漏）。但它们是孤立的——PrefixStabilityManager 不知道失效概率模型的结论，harmony-leak.ts 不考虑缓存命中率的变化。自观察的结果没有被整合到一个统一的状态表示中。

8.4.2 三阶和四阶观察的可能实现

三阶观察要求系统将自观察行为本身作为观察对象。一个具体场景：

PrefixStabilityManager 每轮计算一次前缀的 SHA-256 指纹——这是一个 $O(n)$ 操作，其中 n 是前缀长度。如果前缀长度为 100K tokens，计算指纹需要数十毫秒。如果每轮执行一次，1000 轮的会话就是数秒的 CPU 时间消耗。系统可以追踪自观察的成本——记录每次指纹计算的耗时——然后将这些成本纳入决策。当系统发现自观察本身消耗了超过 1% 的总运行时间时，它应降低观察频率：从每轮一次改为每 10 轮一次。

但这也是二阶的——它只是观察到一个新维度（自观察的开销）。真正的三阶观察是：**系统观察到自己观察到了自观察开销的变化，并将这个发现用于优化观察策略的适应率**。具体来说：如果系统发现"当我降低观察频率时，自观察开销下降了 90%，但缓存命中率没有变化"，它就知道当前的自观察频率是过度设计的——它获得了一个关于观察策略的元知识。

四阶观察则更进一步：系统优化的不只是频率或开销，而是**自观察的整个方法论**。Oh My Pi 的 docs/ERRATA-GPT5-HARMONY.md 文件完美地展示了这个过程的人类版本：开发者收集了 1.05M 次工具调用的数据，发现了 56 次泄漏，分析了泄漏的机制（glitch token → 空嵌入 → 先验坍缩 → logit mask 重新分配），然后设计了七信号融合检测器。如果这个过程可以被自动化——如果系统自己能从 stats.db 中的 ss_tool_calls 表提取异常模式、分析根因、设计一个新的检测信号——系统就达到了四阶自观察。

四阶观察的门槛极高。它要求系统不仅具备自观察能力，还具备**自观察的元理论**——即关于“我应该如何观察自己”的理论 T_{meta} 。 T_{meta} 包含了关于观察策略的知识：在什么条件下使用什么观察手段、如何验证观察结果的可靠性、如何在不同的观察维度之间分配预算。这是理论塔结构向更高层的自然延伸：理论 T （系统提示词）之上是元理论 T_{meta} （自观察策略），之上是元元理论 $T_{\text{meta}2}$ （观察策略优化策略），此过程可以无限继续——虽然实际上三层就足够了，每增加一层收益递减。

8.4.3 从人类操作到系统自动化

目前，三阶和四阶自观察完全由人类开发者执行——我们阅读日志 → 发现模式 → 修改配置 → 验证效果。按 von Foerster 的二阶控制论，这些人类操作不是“外部调试”——它们是系统认知功能的一部分，只是通过人类神经系统而非软件模块来执行。

自动化的关键在于：**将这些人类操作显式化为系统可以调用的控制回路**。如果系统自身能够：

1. **收集足够维度的自观察数据**（不仅是前缀稳定性，还有工具调用模式、输出质量、收敛速度、用户反馈）
2. **建立自观察之间的关联模型**（“当前缀稳定性下降时，工具调用的失败率也会上升——延迟时间约 5 轮”）
3. **评估自己的观察策略**（“我花了 30% 的 CPU 时间在前缀稳定性监测上，但这个维度对预测失败率只有 0.1 的贡献值——我应该将大部分预算转移到工具调用成功率监测上”）

——它就接近了三阶自观察的门槛。从三阶到四阶的跨越需要系统具备**策略的自我否定能力**：不是简单地调整预算分配，而是放弃当前的方法论框架，重新设计观察策略本身。这在当前是一个 open problem——类似于科学革命中的范式转移。

8.5 对实践者的启示

前四节指向了 Harness 工程的远期方向。以下三条启示是当前就可以应用的实践准则。

8.5.1 公理选择约束一切

系统提示词（理论 T ）的第一条公理定义了 Agent 的本体论——“你是谁”。这个选择约束了后续的一切设计。它不是一个“先写一个大概，以后再优化”的问题——它是系统架构的根节点。第一条公理的微小差异会像蝴蝶效应一样，在系统的每个工程决策中放大。

DS-TUI 的本体论公理是“你已经在运行在 TUI 中了”（“You're already running inside it”）。这意味着 agent **不需要知道启动细节**——不需要了解 PTY 分派、事件循环、渲染周期。它的世界就是工具接口——对它来说，`read_file`、`write_file`、`exec_shell` 就是“直接操作世界”的方式。这个选择直接决定了 DS-TUI 的编译时工具注册策略和静态前缀布局——因为所有工具在系统启动时就已知，不需要运行时发现。

Oh My Pi 的本体论公理更抽象——agent 是一个“编码代理”（coding agent），它的身份由模式（mode）定义。不同模式有不同的公理集：在交互模式下，agent 需要知道 TUI 的存在；在无头模式下，agent 只关心输入输出的契约。这个选择直接导致了 Oh My Pi 的模块化公理组装（`skills × rules × goals × memories`）和运行时提示词拼接——因为 agent 的身份需要在运行时动态调整。

工程检查：在写下第一条公理之前，问自己三个问题：

1. **Agent 需要知道多少自己的实现细节？** 如果太多，agent 的行为会与实现耦合，修改实现就会破坏 agent 的心智模型。如果太少，agent 会对自己能做什么产生错误认知。
2. **同一套公理能适用于所有运行模式吗？** 如果不同模式需要不同公理，就需要在系统提示词中建立“模式感知”的公理选择机制——相当于一个条件性理论的扩展。
3. **公理之间的依赖关系是什么？** 改变一条公理会不会隐式地改变另一条公理的推论？当你新添加一个工具时，工具列表中工具的排序可能影响前缀的稳定性——这种公理之间的隐式依赖是系统中最容易被忽视的耦合点。

8.5.2 先建立负反馈，再加正面功能

大多数构建 Harness 系统的人从“加更多工具”开始——增加能力，扩展语言，让 agent 能做更多事情。但这放大了 Ashby 的必要多样性律的问题：控制器的复杂度必须至少等于被控系统的复杂度。每增加一个工具，agent 的行为空间就扩大一个维度——负反馈机制必须相应地扩展以覆盖新增的失效模式。

DS-TUI 和 Oh My Pi 的经验表明，**应该先建立负反馈，再加正面功能：**

- 在添加 `edit_file` 工具之前，先有 `LspManager` 在编辑后提供诊断反馈（DS-TUI）
- 在添加 `agent_swarm` 之前，先有 `task_manager` 管理任务生命周期（DS-TUI）
- 在扩展上下文之前，先有 `shouldCompact()` 和 `pruning.ts` 管理上下文（Oh My Pi）

Oh My Pi 的 `harmony-leak.ts` 是一个反面教材：Harmony 泄漏检测是在问题出现之后才添加的。如果 GPT-5 的 Harmony 协议泄漏在项目设计阶段就被预见，检测器和恢复机制会从第一天就集成到 `agent-loop.ts` 中，而不是作为事后的补丁。这不是对 Oh My Pi 的批评——事实上，预见模型特定的异常模式几乎不可能——但它是一个深刻的教训：控制机制不应该在事故发生后"加装"，而应该在添加正面能力的同时配套实现。

实践检查清单：

- 敏感操作（写文件、执行命令）是否有审批机制？
- 长会话是否有自动压缩？
- 模型输出是否有格式验证？
- 工具调用参数是否在解析时检查（而非在执行时崩溃）？
- 是否有机检测模型的"异常行为模式"（重复失败、无意义输出、协议泄漏）？

如果以上任何一个问题的答案是"否"，那么在添加下一个工具之前，先解决这个控制缺口。

8.5.3 能够观察自己的系统才能改进自己

这是二阶控制论的核心教训，也是本章其余所有预测的元前提。一个系统如果不知道自己的前缀缓存命中率下降到了 40%，就不会去调查为什么；如果不知道自己的错误率在上升，就不会去调整策略；如果不知道 Harmony 泄漏正在发生，就不会去截断和恢复。

两个项目的实践体现了不同的自观察策略：

- **DS-TUI**：在代码中嵌入自观察（`PrefixStabilityManager`、`CapacityController`、`CycleManager`），将观察结果暴露给 TUI 和用户，使自观察成为**实时反馈**的一部分。用户能看到系统的性能指标，但 agent 自身不能（观察结果只在 UI 层，不注入理论 T ）。
- **Oh My Pi**：通过 `telemetry.ts` 的 `OpenTelemetry` 集成和 `stats.db` 的数据库记录，将观察结果持久化，供离线分析。这些数据不参与实时决策——它们是开发者追踪的，而不是系统自用的。

两种策略互补：实时自观察适用于需要即时响应的异常（如泄漏检测），离线自观察适用于需要统计积累才能发现的模式（如长期性能趋势）。但从自修正和预测式控制的角度看，**最关键的是将这两条观察流合并**——观察结果不仅对人类可见，也以某种形式对 agent 自身可见。这是从二阶观察走向三阶观察的第一步。

实践检查清单：

- 系统能否回答"我运行得怎么样？"（成本、延迟、成功率）
- 系统能否回答"我变了吗？"（提示词变化、工具列表变化、行为模式变化）

- 系统能否回答"我应该改变吗？"（是否到了压缩、归档、切换策略的时间点）
- 这些自观察信息是只对人类可见，还是也对 Agent 自身可见？如果 Agent 自身看不到，它无法基于这些数据调整行为。

8.6 结束语：一切理论都是模型

2017 年，N. J. A. Sloane 在回答"数学中最深刻的定理是什么"时，提到了模型论的一个基本结果——洛文海姆-斯科伦定理：如果一个一阶理论有无限模型，那么它在任意更大的基数上也有模型。这个定理揭示了理论的不完备性根植于逻辑本身：任何足够丰富的理论都无法唯一决定它的模型。

Harness 工程同样如此。你的系统提示词 T 描述了一个理想的 Agent。但在这个 T 下存在无数可能的模型 M ——好的、坏的、正确的、偏离的、合规的、异常的。我们的系统提示词（理论）永远无法完善到只唯一决定最优行为。模型论告诉我们，这是逻辑本身的限制，不是工程能力的限制。

这让我们回到本书的开篇命题：**一切理论都是模型**。模型论不是一个关于"终极真理"的框架——它是一个关于"如何在有限理论下管理无穷模型"的框架。

在 Harness 工程中，这意味着三件事：

第一，**放弃对"完美提示词"的追求**。不存在一个 T ，能涵盖所有可能场景而无需运行时调整。每一章展示的系统——DS-TUI 的 264 行公理系、Oh My Pi 的 108 条技能规则——都是"足够好"的理论，不是"完备"的理论。

第二，**在运行时维护而不只是编译时优化**。前缀缓存优化、容量控制、压缩策略、协议泄漏检测——这些都不是设计时的理论问题，而是运行时的一致性问题。理论的价值在执行中实现——不是在设计中评估的。

第三，**拥抱自观察闭路**。在模型论中，一个系统只有具备"观察自身模型"的能力，才能通过调整理论来缩小模型类。不具备自观察能力的系统是开环的——它在理论空间中飞行的路径由开发者预设，与运行时环境的反馈无关。具备自观察能力的系统是闭环的——它通过比较实际模型 M 与理想模型 M_{ideal} 来修正理论，使下一轮出现更接近理想的模型。

这一路走下去，Harness 系统将从"人类写提示词，模型执行"的静态分层，演化为"模型在执行过程中写自己的提示词，并观察自己执行得如何"的动态闭路。这不再是今天的 Harness——但它是模型论预设的方向。

而这，也许是 Harness 工程真正的"第一原理"：一个好的 Harness，不是一个告诉模型该做什么的程序——程序可以做到这一点。一个好的 Harness，也不只是一个帮助模型记住和执行的软件系统——软件工程每天都在做这件事。

一个好的 Harness，是一个帮助模型变得更好的系统。它帮助模型理解自己的局限性，帮助模型从错误中学习，帮助模型在未来做出更好的决策。如果模型有一天能自我改进，那么 Harness 应该已经为这一天做好了准备——它提供了形式化的语言、可验证的一致性约束、以及一个可以容纳自我修改的理论框架。

这是 Harness 工程的使命——不是控制，是相互超越。

附录：原始导读

Harness 工程的形式化基础（模型论 · 系统论双框架）

模型论提供形式化的分析语言（工具），系统论提供宏观的架构原则（组织）。读者维护两个世界观，但模型论作为"通用接口"统一了所有微观分析，系统论作为"架构原则"统一了所有章节组织。

篇	#	章	框架定位
第一篇 形式语言	1	模型论基础	模型论作为分析工具
	2	Harness 即模型论	模型论作用于 Harness
第二篇 系统论架构	3	整体性：理论的相容性与合并	系统论组织 × 模型论分析
	4	层次性：理论塔	系统论组织 × 模型论分析
	5	开放性：语言扩展	系统论组织 × 模型论分析
	6	目的性：目标函数驱动理论选择	系统论组织 × 控制论分析（子框架）
第三篇 综合	7	比较分析与统一定义	全综合
	8	展望	—

认知负担分析

读者需要同时维护两个世界观：

- **模型论**作为规范分析语言——所有微观机制（工具签名、提示词公理、缓存指纹）都用"理论 T "结构 M 解释函数 I 满足关系 $\models$ 来表述
- **系统论**作为章节组织原则——第3-6章按四大原理编排

双框架增加了认知负担，也提高了分析精度。模型论的语言工具使得微观机制的描述更加精确（"提示词即公理集"比"提示词是系统约束"更精确），同时模型论的"软满足"概念自然处理了LLM的不确定性。

章节文件

第一篇：形式语言

- 01-model-theory-foundations.md — 模型论基础
- 02-harness-as-model-theory.md — Harness 即模型论

第二篇：系统论架构

- 03-wholeness.md — 整体性
- 04-hierarchy.md — 层次性
- 05-openness.md — 开放性
- 06-purpose.md — 目的性

第三篇：综合

- 07-comparative-synthesis.md — 比较分析与统一定义
- 08-prospect.md — 展望